

Helios “ Project Bible

Autonomous Growth Diagnosis Engine

Version: v1.0 Date: 2026-06-03

**Purpose.** This document is the authoritative, rebuild-from-scratch reference for Helios “ an autonomous “AI Growth Analyst” that continuously diagnoses *why* an e-commerce funnel is moving, distinguishes mix-shift from rate-change, quantifies revenue-at-risk in dollars, prescribes a prioritized and statistically-defensible experiment backlog, and produces an executive Decision Brief, all grounded in governed SQL it writes, verifies, and is graded on by an offline evaluation benchmark. It is written so that an engineer with no prior context could reconstruct the entire system “ data model, semantic layer, MCP tool surface, seven-agent orchestration, memory, evaluation harness, and roadmap “ years from now. Where any section disagrees with the Canonical Reference Card below, the Reference Card (which mirrors the project Foundation spec) wins.

How to Read This Document

The Bible is organized in 25 numbered sections grouped into nine thematic parts. Read it top-to-bottom for the full narrative, or jump by role:

- **New to Helios?** Read Part I (Strategy, “5) for the *why*, then the Canonical Reference Card in this front matter for the shared vocabulary. Those two together are enough to follow any later section.
- **Analytics / data engineer?** Focus on Part III (Data Model, “11), Part IV (Semantics, “14), and Part V (Analytics Engineering, “17). These define every metric, the dbt DAG, and the BigQuery cost controls.
- **AI / platform engineer?** Focus on Part VI (MCP, “18) and Part VII (Agents + Memory, “19 & “22). The five MCP servers are the trust boundary; the seven agents are the control plane.
- **Evaluating rigor / trust?** Read Part VIII (Evaluation + Experimentation, “20“21): the labeled benchmark, the 85%-vs-45% claim, and the experiment-design framework.
- **Hiring manager / interviewer?** Part IX (Roadmap + Narrative, “23“25) maps the maturity ladder and the resume/interview framing.

Three reading aids: (1) every term is defined once in the **Glossary** at the end of this front matter; (2) every canonical name (metric, dimension, MCP tool, agent) is enumerated in the **Canonical Reference Card** and must never be paraphrased elsewhere; (3) requirements carry stable IDs (FR-, *NFR*-) so they can be cited and traced.

Table of Contents

Sections are listed in thematic reading order. Note that **Memory Architecture (22)** is presented alongside **Agent Architecture (19)** in Part VII, because memory is the state plane that the agent control plane reads from and writes to “ they are best understood together rather than in strict numeric order.

| #                      | Section                     | Part         |
|------------------------|-----------------------------|--------------|
| Part I “ Strategy      |                             |              |
| 1                      | Product Vision              | Strategy     |
| 2                      | Business Problem            | Strategy     |
| 3                      | User Personas               | Strategy     |
| 4                      | Core Product Thesis         | Strategy     |
| 5                      | Success Metrics             | Strategy     |
| Part II “ Requirements |                             |              |
| 6                      | Functional Requirements     | Requirements |
| 7                      | Non-Functional Requirements | Requirements |

| #                                        | Section                                               | Part                  |
|------------------------------------------|-------------------------------------------------------|-----------------------|
| Part III – Data Model                    |                                                       |                       |
| 8                                        | Data Model                                            | Data Model            |
| 9                                        | Event Model                                           | Data Model            |
| 10                                       | Funnel Definitions                                    | Data Model            |
| 11                                       | Revenue Definitions                                   | Data Model            |
| Part IV – Semantics                      |                                                       |                       |
| 12                                       | Channel Attribution Definitions                       | Semantics             |
| 13                                       | Metric Definitions                                    | Semantics             |
| 14                                       | Semantic Layer Design                                 | Semantics             |
| Part V – Analytics Engineering           |                                                       |                       |
| 15                                       | Analytics Engineering Architecture                    | Analytics Engineering |
| 16                                       | dbt Project Structure                                 | Analytics Engineering |
| 17                                       | BigQuery Architecture                                 | Analytics Engineering |
| Part VI – MCP                            |                                                       |                       |
| 18                                       | MCP Architecture                                      | MCP                   |
| Part VII – Agents & Memory               |                                                       |                       |
| 19                                       | Agent Architecture                                    | Agents                |
| 22                                       | Memory Architecture ( <i>presented alongside 19</i> ) | Agents                |
| Part VIII – Evaluation & Experimentation |                                                       |                       |
| 20                                       | Evaluation Framework                                  | Eval / Experiment     |
| 21                                       | Experimentation Framework                             | Eval / Experiment     |
| Part IX – Roadmap & Narrative            |                                                       |                       |
| 23                                       | Future Roadmap                                        | Roadmap / Narrative   |
| 24                                       | Interview Narrative                                   | Roadmap / Narrative   |
| 25                                       | Resume Narrative                                      | Roadmap / Narrative   |

## Document Conventions

**Naming.** All physical names – metrics, dimensions, dbt models, columns, MCP servers, MCP tools – are `snake_case` and must match the Canonical Reference Card exactly. Never invent a synonym (e.g. `conversion_pct` for `session_conversion_rate`); unknown names are a hard error at the semantic layer, not a fallback. MCP servers are named `<area>-mcp` (e.g. `warehouse-mcp`). dbt layers use fixed prefixes: `stg_<source>__<entity>` (staging), `int_<source>__<entity>` (intermediate), `fact_*` / `dim_*` (marts). Agent names are capitalized proper nouns (Orchestrator, Monitor, Decompose, Diagnose, Prescribe, Narrator, Critic).

**Code fences.** Fenced blocks are labeled by intent: `sql` for governed/dbt SQL, `python` for MCP-server or harness code, `yaml` for configuration and semantic-layer definitions, `json` for MCP request/response payloads, and `text` for diagrams, ASCII funnels, and pseudocode. SQL shown in the Bible is illustrative of the *governed* definition; at runtime the LLM never authors it – `semantic-mcp.build_query` emits it.

**Requirement numbering.** Functional requirements are `FR-<domain-letter><n>` (e.g. `FR-A1` in domain A "Data & Semantic Layer"). Non-functional requirements are `NFR-<area-letter><n>` (e.g. `NFR-P1` for Performance). Each carries a MoSCoW priority (Must / Should / Could / Won't-for-now) and a verification method. Findings, hypothesis cards, scenarios, and experiments carry their own ID prefixes ( `F-*`, `H-*`, `S###`, `Exp-###` ).

**Conventions for money and rates.** All money uses GA4's `_in_usd` columns; non-USD twins are never aggregated. Rates are computed as `SUM(numerator)/SUM(denominator)` after grouping (never an average of per-segment ratios) – this re-aggregation discipline is itself the defense

against Simpson's paradox.

## Canonical Reference Card

This card is the single source of truth for shared vocabulary. It mirrors the project Foundation spec exactly. Any section that disagrees with this card is in error.

### Macro funnel stages

```
session_start → view_item → add_to_cart → begin_checkout → add_shipping_info → add_payment_info → purchase
```

Helios reports **step-to-step** conversion (each stage / prior stage) AND the **overall** `session_conversion_rate = purchasing_sessions / sessions` . The funnel is session-scoped; the session key is `(user_pseudo_id, ga_session_id)` .

| # | Stage             | Maps to event                                   |
|---|-------------------|-------------------------------------------------|
| 0 | session_start     | <code>session_start</code> (denominator anchor) |
| 1 | view_item         | <code>view_item</code>                          |
| 2 | add_to_cart       | <code>add_to_cart</code>                        |
| 3 | begin_checkout    | <code>begin_checkout</code>                     |
| 4 | add_shipping_info | <code>add_shipping_info</code>                  |
| 5 | add_payment_info  | <code>add_payment_info</code>                   |
| 6 | purchase          | <code>purchase</code>                           |

### Canonical metric names (snake\_case)

| Group       | Metrics                                                                                                                                                                                                                                                                                                     |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Volume      | <code>sessions</code> , <code>users</code> , <code>new_users</code> , <code>returning_users</code> , <code>engaged_sessions</code> , <code>view_item_sessions</code> , <code>add_to_cart_sessions</code> , <code>begin_checkout_sessions</code> , <code>purchasing_sessions</code>                          |
| Engagement  | <code>engagement_rate</code>                                                                                                                                                                                                                                                                                |
| Funnel rate | <code>session_conversion_rate</code> (= <code>purchasing_sessions/sessions</code> ), <code>view_to_cart_rate</code> , <code>cart_to_checkout_rate</code> , <code>checkout_to_purchase_rate</code> , <code>cart_abandonment_rate</code> , <code>checkout_abandonment_rate</code>                             |
| Revenue     | <code>transactions</code> (distinct <code>transaction_id</code> ), <code>revenue</code> (sum <code>purchase_revenue_in_usd</code> ), <code>gross_revenue</code> , <code>net_revenue</code> (gross â refunds), <code>aov</code> (= <code>revenue/transactions</code> ), <code>items_per_transaction</code> |
| Efficiency  | <code>revenue_per_session</code> (RPS = <code>revenue/sessions</code> ), <code>revenue_per_user</code> (ARPU = <code>revenue/users</code> )                                                                                                                                                                 |

### Canonical dimensions

`device_category`, `operating_system`, `browser`, `country`, `region`, `channel_group`, `source`, `medium`, `campaign`, `landing_page` (first `page_location` of session), `item_category`, `item_name`, `is_new_user`, `day`, `week`, `session_number_bucket` .

**Channel groups (GA4-style default grouping):** Direct, Organic Search, Paid Search, Display, Paid Social, Organic Social, Email, Affiliates, Referral, Other.

### The five MCP servers and their tools

| Server                     | Role                                                  | Tools                                                                                                                                                    |
|----------------------------|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>warehouse-mcp</code> | Governed execution (only holder of a BigQuery client) | <code>list_tables</code> , <code>describe_table</code> , <code>dry_run(sql)</code> , <code>run_query(sql)</code> , <code>reconcile(metric, grain)</code> |
| <code>semantic-mcp</code>  | The ONLY path to SQL (anti-hallucination layer)       | <code>get_metric(name)</code> , <code>list_dimensions()</code> , <code>build_query(metric, dims, filters, window)</code>                                 |

| Server         | Role                                | Tools                                                                                                                                                                                                                                                 |
|----------------|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| stats-mcp      | The ONLY path to math (determinism) | <code>detect_anomaly(series, method)</code> , <code>decompose_change(metric, dim, t0, t1)</code> , <code>significance_test(a, b)</code> , <code>forecast(series, horizon)</code> , <code>cohort_retention(...)</code> , <code>rfm_segment(...)</code> |
| experiment-mcp | Statistically-defensible backlog    | <code>power_analysis(baseline, mde, alpha, power)</code> , <code>runtime_estimate(n, traffic)</code> , <code>design_experiment(hypothesis, metric)</code>                                                                                             |
| report-mcp     | Narration and memory                | <code>render_brief(findings)</code> , <code>save_diagnosis(...)</code> , <code>recall_prior(metric, segment)</code> , <code>export(format)</code>                                                                                                     |

The seven agents and their roles

| Agent                  | Model  | Role                                                                                             |
|------------------------|--------|--------------------------------------------------------------------------------------------------|
| Orchestrator (Planner) | Opus   | Plan the run, sequence stages, manage budget, route to Critic, finalize                          |
| Monitor                | Sonnet | Anomaly detection over canonical metric series                                                   |
| Decompose              | Sonnet | Split each anomaly into mix-shift vs rate-change vs interaction                                  |
| Diagnose               | Opus   | Hypothesis-tree root-cause analysis, each branch SQL-verified                                    |
| Prescribe              | Sonnet | Convert confirmed root causes into a prioritized, powered experiment backlog                     |
| Narrator               | Sonnet | Compose the executive Decision Brief from surviving findings                                     |
| Critic                 | Opus   | Adversarially refute every finding (confound, sample, seasonality, data quality) before it ships |

**Core decomposition identity** (the technical centerpiece). For an aggregate rate  $R = \sum_i (w_i \cdot r_i)$  where  $w_i$  is segment volume share and  $r_i$  is segment rate, the change  $\Delta R$  from  $t_0$  to  $t_1$  decomposes as:

```
mix effect    =  $\sum_i (w_i(t_1) \cdot r_i(t_0)) - \sum_i (w_i(t_0) \cdot r_i(t_0))$  # traffic composition changed
rate effect    =  $\sum_i (w_i(t_0) \cdot (r_i(t_1) - r_i(t_0)))$  # in-segment behavior changed
interaction    =  $\sum_i ((w_i(t_1) - w_i(t_0)) \cdot (r_i(t_1) - r_i(t_0)))$  # both moved together
 $\Delta R$          = mix effect + rate effect + interaction
```

**Success-metric targets:** root-cause accuracy ≥ 85% (vs ~45% naive baseline); time-to-diagnosis < 5 min/run; 0 hallucinated columns/metrics (100% governed SQL); 100% of findings carry significance + dollar impact; query cost per run under a fixed BigQuery byte budget.

**Maturity levels:** L1 Intern MVP; L2 Strong portfolio project; L3 Top-1% undergraduate project.

Glossary

| Term / acronym    | Definition                                                                                                                                                                                                                                |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Action tracking   | A memory store recording whether a prescribed experiment shipped and whether the fix actually worked (status ^ proposed / shipped / running / completed / abandoned, plus observed lift and outcome). Closes the "did-the-fix-work" loop. |
| AOV               | Average Order Value = <code>revenue / transactions</code> . Reflects merchandise value (excludes shipping and tax).                                                                                                                       |
| ARPU              | Average Revenue Per User = <code>revenue / users</code> (= <code>revenue_per_user</code> ). Denominator is all users in the window, not just purchasers.                                                                                  |
| Anomaly detection | Deterministic flagging of abnormal movement in a metric series via <code>stats-mcp.detect_anomaly</code> (STL-residual, EWMA, or robust z-score), accounting for weekly seasonality.                                                      |
| Channel group     | GA4-style default channel grouping (Direct, Organic Search, Paid Search, Display, Paid Social, Organic Social, Email, Affiliates, Referral, Other) derived from session-scoped source/medium.                                             |
| Cohort            | A set of users grouped by the week of their <code>user_first_touch_timestamp</code> ; the basis for retention curves.                                                                                                                     |
| Critic            | The adversarial verifier agent (Opus) that attempts to refute every finding before it ships; verdicts are PASS / DOWNGRADE / DROP.                                                                                                        |

| Term / acronym                         | Definition                                                                                                                                                                                                                                                |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Decision Brief</b>                  | The primary product surface: the autonomously-produced, executive-ready artifact rendered by <code>report-mcp.render_brief</code> , where every finding carries a decomposition, a significance test, a dollar revenue-at-risk, and a recommended action. |
| <b>Decomposition</b>                   | Splitting an aggregate rate change $\hat{\Delta}R$ into mix effect, rate effect, and interaction (see the core identity above); the technical centerpiece, computed by <code>stats-mcp.decompose_change</code> .                                          |
| <b>DiD (difference-in-differences)</b> | A quasi-experimental readback: $DiD = (\text{treated\_post} - \text{treated\_pre}) - (\text{control\_post} - \text{control\_pre})$ , netting out seasonality via a comparable control segment, with parallel-trends checked on the pre-period.            |
| <b>dry-run</b>                         | A BigQuery cost/schema check ( <code>warehouse-mcp.dry_run</code> ) that returns estimated scanned bytes without executing or billing; mandatory before any <code>run_query</code> to enforce the byte budget.                                            |
| <b>Engaged session</b>                 | A GA4-defined engaged session (session-scoped <code>session_engaged = '1'</code> , with the GA4 fallback of engagement time / multiple page views / a conversion event); <code>engagement_rate = engaged_sessions / sessions</code> .                     |
| <b>Faithfulness</b>                    | Whether the Narrator's prose claims are entailed by the underlying SQL evidence and <code>stats-mcp</code> outputs (target $\approx 0.95$ ); checked by rule + Critic-as-judge entailment.                                                                |
| <b>First-touch attribution</b>         | Crediting a session/transaction to the channel of the user's first session ever; stored on <code>dim_users.first_touch_channel</code> .                                                                                                                   |
| <b>fct_daily_funnel</b>                | The pre-aggregated daily funnel fact (additive step counts + revenue per day — dimension cell); the primary feed for Monitor and Decompose.                                                                                                               |
| <b>ga_session_id</b>                   | A session id that is unique only <i>within</i> a <code>user_pseudo_id</code> (never globally); lives inside <code>event_params</code> and must be unnested. Half of the session key.                                                                      |
| <b>Governed SQL</b>                    | SQL emitted exclusively by <code>semantic-mcp</code> from versioned metric/dimension definitions; the LLM never authors raw SQL. Guarantees 0 hallucinated columns.                                                                                       |
| <b>Grounding over generation</b>       | The first principle: the LLM composes governed metrics and calls deterministic tools, never authoring SQL or computing statistics in token-space.                                                                                                         |
| <b>Hallucination rate</b>              | The count of columns/metrics in emitted SQL not present in the semantic registry or GA4 schema; a hard-zero CI gate.                                                                                                                                      |
| <b>ICE / PIE</b>                       | Experiment-prioritization scores. ICE = (Impact — Confidence) / Effort; PIE = Potential — Importance — Ease. Both are deterministic and recorded on each hypothesis card.                                                                                 |
| <b>Idempotency</b>                     | Re-running a model/run over the same partitions yields byte-identical output (via partition-by-date + <code>insert_overwrite</code> ); no duplicate diagnoses per window.                                                                                 |
| <b>Interaction effect</b>              | The third decomposition term, $\hat{\Delta}(\hat{w}_i \cdot \hat{r}_i)$ change attributable to weight and rate moving together.                                                                                                                           |
| <b>Last-touch attribution</b>          | Crediting to the channel of the converting session itself; the Helios default for session-grained funnel and revenue diagnosis.                                                                                                                           |
| <b>Last-non-direct attribution</b>     | Crediting to the most recent non- ( <code>direct</code> ) channel within a lookback window (GA4 default 90 days).                                                                                                                                         |
| <b>MCP (Model Context Protocol)</b>    | An open client-server protocol over JSON-RPC that lets the agent runtime invoke schema-typed external tools; the mechanism that makes the grounding boundary physically enforceable.                                                                      |
| <b>MDE</b>                             | Minimum Detectable Effect $\hat{\Delta} \epsilon$ the smallest effect (often relative, e.g. 10%) an experiment is powered to detect; an input to <code>power_analysis</code> .                                                                            |
| <b>Memory store</b>                    | Durable state across runs: diagnosis history (+ embeddings), suppression list, business glossary, seasonality/launch calendars, action tracking, and the run-state/audit trail.                                                                           |
| <b>Mix-shift (mix effect)</b>          | A change in an aggregate rate caused by traffic <i>composition</i> shifting between segments (weights $w_i$ ), with in-segment behavior unchanged. The non-causal half that naive RCA mistakes for behavior change.                                       |
| <b>Quasi-experiment</b>                | An inference design used when no live A/B traffic exists (this dataset is observational): pre/post and difference-in-differences readbacks via <code>stats-mcp</code> .                                                                                   |
| <b>Rate-change (rate effect)</b>       | A change in an aggregate rate caused by in-segment <i>behavior</i> shifting (segment rates $r_i$ ), with composition held fixed. The genuinely causal half.                                                                                               |
| <b>Reconciliation</b>                  | Checking that a fact-derived metric matches <code>warehouse-mcp.reconcile</code> canonical totals within tolerance ( $\approx 0.5\%$ ); part of verify-then-trust.                                                                                        |
| <b>Revenue-at-risk</b>                 | The dollar counterfactual every finding carries: dollars recoverable if a degraded rate returned to its t0 / forecast baseline ( $\hat{r} \text{rate} - \text{affected sessions} - \text{downstream value}$ ).                                            |

| Term / acronym               | Definition                                                                                                                                                                                                                                              |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>RFM</b>                   | Recency / Frequency / Monetary segmentation, scored into quintiles per user by <code>stats-mcp.rfm_segment</code> .                                                                                                                                     |
| <b>RPS</b>                   | Revenue Per Session = <code>revenue / sessions</code> (= <code>revenue_per_session</code> ); decomposes exactly as <code>session_conversion_rate</code> $\hat{=}$ <code>aov</code> .                                                                    |
| <b>Sessionization</b>        | Reconstructing a session row from events sharing the same ( <code>user_pseudo_id</code> , <code>ga_session_id</code> ) â€” GA4 ships no session row.                                                                                                    |
| <b>Simpson's paradox</b>     | When an aggregate rate moves opposite to every segment's rate because the mix shifted; the failure mode the decomposition is built to detect.                                                                                                           |
| <b>Suppression list</b>      | A memory store of acknowledged/ignored causes that must not be re-raised, with a TTL so re-emerging issues eventually re-surface.                                                                                                                       |
| <b>traffic_source gotcha</b> | The event-level <code>traffic_source</code> struct is USER-LEVEL first-touch attribution, not session source; Helios prefers session-scoped <code>event_params source/medium</code> and uses <code>traffic_source</code> only as a documented fallback. |
| <b>user_pseudo_id</b>        | The device/cookie id that is the de facto user key ( <code>user_id</code> is almost always NULL); cross-device stitching is therefore impossible and user-grain metrics are cookie-based approximations.                                                |
| <b>Verify-then-trust</b>     | The second principle: dry-run cost + schema validation + reconciliation + adversarial Critic before any finding ships.                                                                                                                                  |

# 1. Product Vision

## 1.1 The 3-5 year vision

Helios is the **Autonomous Growth Diagnosis Engine**: an always-on AI Growth Analyst that does for funnel diagnosis what continuous integration did for software builds. Today a human analyst is a serial, manually-triggered, expensive root-cause oracle who is queried only when a number looks "off." In the 3-5 year horizon, Helios inverts that: diagnosis becomes a continuous, autonomous, proactive background process. Every scheduled run, Helios re-derives the full `session_start`  $\rightarrow$  `view_item`  $\rightarrow$  `add_to_cart`  $\rightarrow$  `begin_checkout`  $\rightarrow$  `add_shipping_info`  $\rightarrow$  `add_payment_info`  $\rightarrow$  `purchase` funnel, detects which steps moved, separates whether the movement came from **mix-shift** (traffic composition changed) or **rate-change** (in-segment behavior changed), prices the movement in dollars of `revenue` at risk, and ships a verified **Decision Brief** to the people who own the number â€” before they have to ask.

The end-state is an analytics organization where the question "why did `session_conversion_rate` drop last week?" is answered by an artifact that already exists, is statistically defensible, is grounded in governed SQL, and survived an adversarial Critic â€” not by a Slack message to an analyst that kicks off two days of ad-hoc querying.

## 1.2 The category Helios creates

Helios deliberately does not compete in "BI dashboards" or "ask-your-data chatbots." It creates a new category: **autonomous growth diagnosis**. The category's defining promise is *causal-style attribution of metric movement, continuously and without human prompting*, distinct from the three incumbent categories:

| Category                                    | Primary artifact        | Trigger                       | Answers                                               |
|---------------------------------------------|-------------------------|-------------------------------|-------------------------------------------------------|
| Descriptive BI (Looker, Tableau)            | Dashboard / chart       | Human pull                    | <i>What</i> happened                                  |
| Conversational analytics (text-to-SQL bots) | Ad-hoc query result     | Human prompt                  | <i>What</i> (one question at a time)                  |
| Product analytics (Amplitude, Mixpanel)     | Funnel/retention report | Human pull                    | <i>What</i> , with some <i>who</i>                    |
| <b>Autonomous growth diagnosis (Helios)</b> | <b>Decision Brief</b>   | <b>Scheduler (autonomous)</b> | <b><i>Why, how much (\$), and what to do next</i></b> |

## 1.3 The before / after world for an analytics org

BEFORE (manual root-cause analysis)

metric dips  $\rightarrow$  someone notices (days later)  $\rightarrow$  Slack the analyst  $\rightarrow$  analyst context-switches  $\rightarrow$  writes ad-hoc SQL (often ungoverned)  $\rightarrow$  eyeballs aggregate  $\rightarrow$  misses Simpson's paradox  $\rightarrow$  ships a guess  $\rightarrow$  no dollar quantification  $\rightarrow$  no experiment  $\rightarrow$  decision delayed weeks

AFTER (Helios)

scheduled run (e.g. daily)  $\rightarrow$  Monitor flags anomaly in a canonical metric  $\rightarrow$  Decompose splits mix vs rate vs interaction  $\rightarrow$  Diagnose builds + SQL-verifies a

hypothesis tree → Critic tries to refute it → Prescribe sizes an experiment →  
Narrator ships a Decision Brief with significance + \$ revenue-at-risk + action →  
decision made in <5 minutes of human reading time

## 1.4 Vision statement

Every material movement in the growth funnel is diagnosed automatically, priced in dollars, defended statistically, and turned into a prioritized experiment “before a human has to ask.

## 1.5 Guiding principles

1. **Grounding over generation.** The LLM never authors raw SQL and never computes a statistic by hand. It *composes* governed metrics via `semantic-mcp` and calls deterministic tools via `stats-mcp`. This yields the 0-hallucinated-column, 100%-governed-SQL target.
2. **Verify-then-trust.** Every query is `dry_run` -cost-checked and schema-validated; every result is reconciled against canonical totals via `warehouse-mcp.reconcile`; every finding is attacked by the Critic before it ships.
3. **Determinism where it matters.** All math (decomposition, significance, power, forecasting) runs in real Python code (`scipy/statsmodels/prophet`), never in token-space.
4. **Every finding is actionable.** A finding without a significance test, a dollar impact, and a recommended action is not a finding.
5. **Proactive, not reactive.** Conversation is a secondary drill-down surface; the product’s heartbeat is the autonomous scheduled run.

## 1.6 What success looks like at scale

At scale, Helios runs across many funnels, properties, and dimensions on a fixed BigQuery byte budget per run, produces Decision Briefs that practitioners trust without re-deriving, sustains diagnosis root-cause accuracy  $\geq 85\%$  against a labeled benchmark (vs  $\leq 45\%$  for a naive aggregate-only baseline), and demonstrably shortens the insight-to-action loop from weeks to a single review cycle.

# 2. Business Problem

## 2.1 The insight-to-action gap

Modern e-commerce orgs are drowning in *descriptive* signal and starving for *diagnostic* answers. Dashboards show that `session_conversion_rate` fell from 2.1% to 1.7%, but cannot say **why**, cannot say **how many dollars** that costs, and cannot say **what to do**. The gap between “we see a number moved” (insight) and “we changed the business because of it” (action) is the single most expensive inefficiency in growth analytics. Helios targets that gap directly.

## 2.2 The analyst-time cost of manual root-cause analysis

Manual root-cause analysis (RCA) of a single funnel anomaly is a multi-hour-to-multi-day serial process:

| RCA stage (manual)                                                                              | Typical effort           |
|-------------------------------------------------------------------------------------------------|--------------------------|
| Notice the anomaly (latency before anyone looks)                                                | 1-5 days                 |
| Context-switch + reconstruct the metric definition                                              | 0.5-1 hr                 |
| Write/fix ad-hoc SQL over <code>events_YYYYMMDD</code> shards                                   | 1-3 hr                   |
| Slice by <code>device_category</code> , <code>channel_group</code> , <code>country</code> , ... | 1-2 hr                   |
| Manually reason about mix vs rate (usually skipped)                                             | 0.5-2 hr                 |
| Quantify dollar impact (frequently never done)                                                  | 0.5 hr                   |
| Write up + socialize                                                                            | 1 hr                     |
| <b>Total per anomaly</b>                                                                        | <b>~1-3 analyst-days</b> |

Multiply by the number of metrics, segments, and weeks, and RCA consumes a large fraction of a senior analyst’s time “the exact work Helios automates to a <5 min/run autonomous process.

## 2.3 The mix-shift vs rate-change failure and Simpson's paradox

The core analytical failure of naive RCA is conflating **mix-shift** with **rate-change**. For an aggregate rate  $R = \sum_i (w_i * r_i)$  where  $w_i$  is segment volume share and  $r_i$  is segment rate, the change from  $t_0 \rightarrow t_1$  decomposes exactly into three terms:

```
mix effect    = sum_i ( delta_w_i * r_i_at_t0 )    # composition changed
rate effect   = sum_i ( w_i_at_t0 * delta_r_i )    # behavior changed
interaction   = sum_i ( delta_w_i * delta_r_i )    # both moved together
deltaR        = mix effect + rate effect + interaction
```

A naive analyst reads only `deltaR` and attributes it to behavior. But **Simpson's paradox** means the aggregate can move in the *opposite* direction of every segment. Concretely: if `checkout_to_purchase_rate` rose in both `mobile` and `desktop`, yet aggregate `checkout_to_purchase_rate` fell because traffic mix shifted toward lower-converting `mobile`, the correct action is a *traffic/acquisition* fix, not a *checkout* fix. Naive RCA prescribes fixing checkout â€” wrong root cause, wasted engineering. This decomposition is Helios's technical centerpiece, executed deterministically by `stats-mcp.decompose_change`.

## 2.4 Why dashboards and descriptive analytics fail

- They report **what**, never **why**: no causal-style attribution, no mix/rate split.
- They are **pull, not push**: someone must notice and ask.
- They are **dimension-blind to interaction**: a 2-D chart cannot surface a confound across `channel_group` x `device_category`.
- They lack **dollar quantification**: a percentage-point drop is not a budget decision until it is `revenue` -at-risk.
- They produce **no prescription**: no experiment, no power analysis, no prioritized backlog.

## 2.5 Market context

The market validates the demand and the gap simultaneously. **Amplitude** and **Mixpanel** lead product analytics with strong funnel/retention reporting and recent "AI" features that are still fundamentally descriptive and human-triggered. A wave of **AI-analyst / text-to-SQL startups** promises "ask your data," but they are conversational (reactive), frequently hallucinate columns/metrics, and rarely carry statistical rigor or dollar impact. None of them autonomously decompose mix-vs-rate, price the movement, and ship a verified brief on a schedule. That white space â€” *autonomous, governed, statistically-defensible, dollar-quantified diagnosis* â€” is precisely Helios's category.

## 2.6 Root causes of the problem

1. **No governed semantic layer** at query time -> ad-hoc, inconsistent, hallucinated SQL.
2. **Statistics done in heads/spreadsheets** -> no significance, no decomposition, Simpson's paradox missed.
3. **Reactive triggering** -> long latency before anyone even looks.
4. **No dollar bridge** from rate movement to revenue.
5. **No adversarial check** -> first plausible story ships unrefuted.

## 2.7 Cost of inaction

Every undiagnosed week of a depressed `session_conversion_rate` or `checkout_to_purchase_rate` is recurring lost `revenue` that compounds; every misdiagnosis (wrong mix/rate call) funds the wrong fix; every delayed decision is opportunity cost in an environment where competitors iterate weekly. Inaction is not free â€” it is a continuously accruing, unquantified liability that Helios converts into a visible, prioritized, dollar-denominated backlog.

# 3. User Personas

## 3.1 Priya â€” Head of Growth (PRIMARY)

- **Role & context**: Owns the funnel and the `revenue` number; reports to the exec team weekly; manages PMs and analysts; chronically time-poor.
- **Goals**: Know *why* the funnel moved and *what to do*, fast; defend decisions with data; allocate scarce eng/experiment capacity to the highest-`revenue` opportunity.
- **Top pains**: Waits days for RCA; gets descriptive dashboards with no "why"; can't trust ad-hoc numbers; can't tell mix-shift from rate-change.
- **Jobs-to-be-done**: "When a key metric moves, tell me the root cause, the dollar impact, and the one experiment to run." "Give me a brief I can forward to the CEO."
- **How Helios serves her**: Autonomous Decision Briefs each run, each with mix/rate split, `revenue` -at-risk, significance, and a prioritized experiment backlog from `experiment-mcp`.
- **Success criteria**: Time-to-diagnosis <5 min reading; >=85% of root causes correct; every brief carries \$ + significance + action.
- **Anti-needs**: Does NOT want a SQL IDE, raw tables, or a chatbot to interrogate; does NOT want ungoverned numbers.



### 3.2 Marcus – Product Manager

- **Role & context:** Owns a funnel surface (e.g., checkout: `begin_checkout` → `add_shipping_info` → `add_payment_info` → `purchase`); ships experiments.
- **Goals:** Find where in *his* surface conversion leaks; get a testable hypothesis with expected impact and required sample size.
- **Top pains:** Doesn't know if a leak is real (significant) or noise; doesn't know if it's his surface (rate) or upstream traffic (mix); doesn't know how long a test must run.
- **Jobs-to-be-done:** "Tell me which checkout step leaks, whether it's significant, and design the experiment." "Stop me from testing a confounded mix-shift."
- **How Helios serves him:** Step-level decomposition on `cart_to_checkout_rate / checkout_to_purchase_rate`; `power_analysis` + `runtime_estimate` + `design_experiment` test cards.
- **Success criteria:** Experiments shipped from Helios hypotheses; reduced wasted tests on confounds.
- **Anti-needs:** Does NOT want vanity dashboards; does NOT want hand-wavy "looks lower" claims without significance.

### 3.3 Dana – Data / Product Analyst

- **Role & context:** Owns metric definitions and the dbt models; the person Slacked for every RCA.
- **Goals:** Stop being a human RCA oracle; ensure every number is governed and reconciles; do higher-order analysis.
- **Top pains:** Constant context-switching; ungoverned SQL proliferating; manually checking mix vs rate; reconciling conflicting numbers.
- **Jobs-to-be-done:** "Automate the first 80% of RCA." "Guarantee numbers come only from the governed semantic layer."
- **How Helios serves her:** `semantic-mcp` is the only SQL path; `warehouse-mcp.reconcile` validates against canonical totals; dbt facts (`fct_daily_funnel`, `fct_orders`) are the substrate she owns.
- **Success criteria:** 100% governed SQL, 0 hallucinated columns; fewer ad-hoc RCA pings; faithfulness of briefs to underlying data.
- **Anti-needs:** Does NOT want a black box she can't audit; does NOT want a tool that invents metric synonyms.

### 3.4 Elena – Founder / CEO

- **Role & context:** Reads numbers weekly; cares about `revenue`, growth trajectory, and capital efficiency; not technical in SQL.
- **Goals:** A trustworthy executive read on what's driving growth and what the team is doing about it.
- **Top pains:** Briefs that are either too shallow (a chart) or too deep (a SQL dump); no clear "so what."
- **Jobs-to-be-done:** "Give me the three things that moved `revenue`, why, and the plan."
- **How Helios serves her:** `report-mcp.render_brief` produces an exec-grade Decision Brief; `recall_prior` shows trajectory and whether prior actions worked.
- **Success criteria:** Decisions influenced; confidence in the team's diagnosis.
- **Anti-needs:** Does NOT want raw analytics tooling, jargon, or unverified speculation.

### 3.5 "The Builder" – portfolio author / hiring-target candidate

- **Role & context:** The undergraduate engineer building Helios as a top-1% portfolio project to demonstrate senior-level systems + analytics-engineering + applied-ML judgment.
- **Goals:** Show end-to-end mastery: BigQuery + dbt + MCP + Claude Agent SDK + deterministic stats + offline eval; prove the work survives adversarial scrutiny.
- **Top pains:** Portfolio projects that are demos, not systems; reviewers who doubt rigor; hand-waved "AI" with no grounding or evaluation.
- **Jobs-to-be-done:** "Build a system an engineer can rebuild from the bible years later." "Prove >=85% accuracy on a labeled benchmark with a CI eval harness."
- **How Helios serves him:** The architecture itself (seven agents, five MCP servers, governed grounding, deterministic math, adversarial Critic, GitHub Actions eval) is the demonstration.
- **Success criteria:** Hits L3 maturity; eval harness in CI; targets met; defensible against an interviewer's "how do you know?"
- **Anti-needs:** Does NOT want a toy chatbot; does NOT want unmeasured claims.

## 4. Core Product Thesis

### 4.1 The one-line thesis

An autonomous **AI Growth Analyst** that continuously diagnoses **why** the e-commerce funnel is moving, distinguishes **mix-shift from rate-change**, quantifies **revenue-at-risk in dollars**, prescribes a prioritized, statistically-defensible **experiment backlog**, and produces an executive **Decision Brief** – all grounded in governed SQL it writes, verifies, and is graded on by an offline evaluation benchmark.

### 4.2 The wedge

The wedge is one painful, high-frequency, expensive job done better than anyone: **automated mix-vs-rate root-cause diagnosis of the canonical funnel, priced in dollars**. It is narrow enough to be excellent and verifiable, and it is the exact task that incumbents (descriptive BI, conversational bots, product analytics) all leave to a human.

4.3 Why now

- **GA4 BigQuery export** makes event-grain data ( `events_YYYYMMDD` ) universally queryable.
- **dbt** makes a governed semantic/metrics layer practical.
- **Claude Agent SDK + MCP** make multi-agent plan-execute-critique orchestration with tool-grounding feasible (Opus for orchestrate/critique, Sonnet for high-volume sub-tasks).
- **Mature deterministic stats** (scipy/statsmodels/prophet/pmdarima) let math stay out of token-space.
- Together these collapse the cost of an autonomous, grounded, verifiable diagnosis engine for the first time.

4.4 The anti-product stance

Helios is **not** a BI dashboard, **not** an ad-hoc SQL chatbot, and **not** a generic "ask your data" tool. Conversation is a secondary drill-down surface only. The product's primary mode is autonomous and proactive: it runs on a schedule and reports without being asked.

4.5 The three defensibility pillars

1. **Governed-SQL grounding.** All SQL flows through `semantic-mcp ( get_metric , build_query )`; the LLM never authors raw SQL. Result: 100% governed SQL, 0 hallucinated columns/metrics.
2. **Deterministic statistics.** All decomposition, significance, power, and forecasting run in real code via `stats-mcp` ; nothing statistical is generated by the model.
3. **Adversarial verification + offline eval.** The Critic agent tries to *refute* every finding (mix-shift confound? insufficient sample? seasonality? data quality?); a labeled benchmark in GitHub Actions CI grades root-cause accuracy (target >=85%).

4.6 The product loop

```
MONITOR (Monitor agent + stats-mcp.detect_anomaly)
  → DECOMPOSE (Decompose agent + stats-mcp.decompose_change : mix vs rate vs interaction)
  → DIAGNOSE (Diagnose agent : hypothesis tree, SQL-verified via semantic+warehouse mcp)
  → QUANTIFY (dollarize : delta-rate → revenue-at-risk in USD)
  → PRESCRIBE (Prescribe agent + experiment-mcp : power_analysis, design_experiment)
  → NARRATE (Narrator agent + report-mcp.render_brief : the Decision Brief)
  → LEARN (Memory store : save_diagnosis, recall_prior, suppression list, action tracking)
    ^_____ feeds next MONITOR _____|
(Critic agent adversarially reviews EVERY finding before NARRATE; findings must survive it)
```

4.7 The Decision Brief as the primary product surface

The **Decision Brief** is the product. It is the autonomously-produced, exec-ready artifact rendered by `report-mcp.render_brief` , and every finding in it carries (1) a mix/rate/interaction decomposition, (2) a significance test, (3) a dollar `revenue` -at-risk, and (4) a recommended action / experiment. The dashboard-style and conversational surfaces exist only to drill into the brief.

5. Success Metrics

5.1 North Star metric

**North Star: Verified, actioned Decision Briefs that correctly diagnose root cause** “ i.e., the count (and rate) of shipped briefs whose root cause is correct, that carry \$ + significance + action, and that influence a decision. It captures the whole thesis: autonomous, correct, quantified, actioned.

5.2 Product-quality metrics (mostly leading)

| Metric              | Definition / measurement                                                     | Baseline                         | Target                 | Lead/Lag |
|---------------------|------------------------------------------------------------------------------|----------------------------------|------------------------|----------|
| Root-cause accuracy | Correct root cause on labeled benchmark in CI                                | <=45% (naive aggregate baseline) | >=85%                  | Leading  |
| Decomposition error | Abs error of mix/rate/interaction vs ground truth on synthetic-labeled cases | n/a (unmeasured)                 | near-0 (deterministic) | Leading  |

| Metric              | Definition / measurement                                                        | Baseline                   | Target                  | Lead/Lag |
|---------------------|---------------------------------------------------------------------------------|----------------------------|-------------------------|----------|
| Hallucination rate  | Columns/metrics not in semantic layer per run (schema-validated)                | high (ungoverned bots)     | 0% (100% governed SQL)  | Leading  |
| Faithfulness        | Brief claims reconcile to <code>warehouse-mcp.reconcile</code> canonical totals | n/a                        | 100% reconciled         | Leading  |
| Time-to-diagnosis   | Wall-clock per autonomous run                                                   | ~1-3 analyst-days (manual) | <5 min/run              | Leading  |
| Cost per run        | BigQuery bytes billed per run (dry_run-bounded)                                 | unbounded ad-hoc           | under fixed byte budget | Leading  |
| Findings-with-rigor | % of findings carrying significance + \$ impact                                 | inconsistent               | 100%                    | Leading  |

5.3 Adoption / engagement metrics

| Metric                        | Measurement                                        | Lead/Lag |
|-------------------------------|----------------------------------------------------|----------|
| Scheduled-run completion rate | Successful autonomous runs / scheduled runs        | Leading  |
| Brief open / read rate        | Briefs opened by Priya/Elena / briefs shipped      | Leading  |
| Drill-down sessions           | Secondary conversational/drill use per brief       | Leading  |
| Suppression-list accuracy     | % anomalies correctly suppressed (no false alarms) | Leading  |

5.4 Business-impact metrics (lagging)

| Metric                    | Definition / measurement                                                       | Lead/Lag |
|---------------------------|--------------------------------------------------------------------------------|----------|
| Dollar at-risk surfaced   | Sum of <code>revenue</code> -at-risk correctly identified across briefs        | Lagging  |
| Experiments shipped       | Experiments launched from Helios test cards ( <code>design_experiment</code> ) | Lagging  |
| Decisions influenced      | Tracked via Memory action-tracking: brief -> decision/action taken             | Lagging  |
| Misdiagnosis cost avoided | Wrong-fix efforts avoided by correct mix-vs-rate calls                         | Lagging  |

5.5 Maturity-level success

- **L1 (Intern MVP):** loop runs end-to-end on one funnel; governed SQL; a decomposition; one brief.
- **L2 (Strong portfolio):** seven agents + five MCP servers; Critic active; CI eval harness; significance + \$ on every finding.
- **L3 (Top-1% undergrad):** >=85% benchmark accuracy vs <=45% baseline; <5 min/run; 0 hallucinated columns; cost under byte budget; full autonomous scheduled operation with Memory-driven learning.

6. Functional Requirements

This section enumerates the functional requirements (FRs) for Helios, the Autonomous Growth Diagnosis Engine, grouped by capability domain (A€K). Each FR carries a stable ID, requirement statement, rationale, acceptance criteria, and MoSCoW priority (Must / Should / Could / Won't-for-now). Every requirement is written so that an engineer can verify it deterministically against the GA4 dataset `bigquery-public-data.ga4_obfuscated_sample_ecommerce` and the five MCP servers ( `warehouse-mcp` , `semantic-mcp` , `stats-mcp` , `experiment-mcp` , `report-mcp` ) and seven agents (Orchestrator, Monitor, Decompose, Diagnose, Prescribe, Narrator, Critic).

6.A Data & Semantic Layer

| ID    | Requirement                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Rationale                                                                                       | Acceptance Criteria                                                                                                                                                                               | Priority |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-A1 | The system MUST expose every governed metric ( <code>sessions</code> , <code>users</code> , <code>new_users</code> , <code>returning_users</code> , <code>engaged_sessions</code> , <code>engagement_rate</code> , <code>view_item_sessions</code> , <code>add_to_cart_sessions</code> , <code>begin_checkout_sessions</code> , <code>purchasing_sessions</code> , <code>session_conversion_rate</code> , <code>view_to_cart_rate</code> , <code>cart_to_checkout_rate</code> , <code>checkout_to_purchase_rate</code> , <code>cart_abandonment_rate</code> , <code>checkout_abandonment_rate</code> , <code>transactions</code> , <code>revenue</code> , <code>gross_revenue</code> , <code>net_revenue</code> , <code>aov</code> , <code>items_per_transaction</code> , <code>revenue_per_session</code> , <code>revenue_per_user</code> ) only through <code>semantic-mcp.get_metric(name)</code> . | Anti-hallucination grounding; the LLM must never author raw metric SQL.                         | <code>get_metric(name)</code> returns governed SQL for all 24 canonical metrics; unknown names raise <code>MetricNotFound</code> ; no agent code path emits a metric not present in the registry. | Must     |
| FR-A2 | The system MUST build all SQL via <code>semantic-mcp.build_query(metric, dims, filters, window)</code> , never by string-concatenation in agent code.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Single validated SQL path; verify-then-trust.                                                   | 100% of executed queries originate from <code>build_query</code> ; a static check fails CI if any agent module contains a raw <code>SELECT</code> against <code>events_*</code> .                 | Must     |
| FR-A3 | The dbt project MUST materialize the canonical models: staging <code>stg_ga4__events</code> , <code>stg_ga4__event_params</code> ; intermediate <code>int_ga4__sessionized</code> , <code>int_ga4__funnel_steps</code> ; facts <code>fct_sessions</code> , <code>fct_funnel</code> , <code>fct_daily_funnel</code> , <code>fct_orders</code> , <code>fct_order_items</code> ; dims <code>dim_users</code> , <code>dim_items</code> , <code>dim_channels</code> , <code>dim_date</code> ; plus a semantic layer under <code>models/semantic</code> .                                                                                                                                                                                                                                                                                                                                                    | Reproducible, tested transformation lineage.                                                    | <code>dbt build</code> succeeds; all models exist with the exact snake_case names; <code>dbt docs</code> lineage covers source → staging → fact.                                                  | Must     |
| FR-A4 | <code>stg_ga4__event_params</code> MUST unnest <code>event_params</code> into typed columns and resolve <code>ga_session_id</code> to define the session key ( <code>user_pseudo_id</code> , <code>ga_session_id</code> ).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Sessionization is the foundation for every funnel metric.                                       | A session row exists for each distinct ( <code>user_pseudo_id</code> , <code>ga_session_id</code> ); null <code>ga_session_id</code> rows are quarantined and counted.                            | Must     |
| FR-A5 | <code>dim_channels</code> MUST derive GA4 default channel grouping from <code>SESSION-SCOPED event_params source/medium</code> , falling back to user first-touch <code>traffic_source</code> only when session scope is null.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | Honors the documented <code>traffic_source</code> gotcha (user-level first-touch, not session). | Channel labels → {Direct, Organic Search, Paid Search, Display, Paid Social, Organic Social, Email, Affiliates, Referral, Other}; a unit test asserts fallback ordering.                          | Must     |
| FR-A6 | <code>warehouse-mcp.reconcile(metric, grain)</code> MUST return canonical totals that every fact-derived metric is checked against.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Result reconciliation guardrail.                                                                | Every shipped finding's underlying metric matches <code>reconcile</code> within 0.5% tolerance; mismatches block the finding.                                                                     | Must     |
| FR-A7 | The semantic layer MUST publish <code>list_dimensions()</code> returning exactly the canonical dimensions ( <code>device_category</code> , <code>operating_system</code> , <code>browser</code> , <code>country</code> , <code>region</code> , <code>channel_group</code> , <code>source</code> , <code>medium</code> , <code>campaign</code> , <code>landing_page</code> , <code>item_category</code> , <code>item_name</code> , <code>is_new_user</code> , <code>day</code> , <code>week</code> , <code>session_number_bucket</code> ).                                                                                                                                                                                                                                                                                                                                                              | Bounded, governed slicing vocabulary.                                                           | <code>list_dimensions()</code> set-equals the canonical list; <code>build_query</code> rejects any other dimension.                                                                               | Must     |
| FR-A8 | The system SHOULD support an incremental dbt strategy keyed on <code>event_date</code> so only new date shards are processed per run.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Cost and runtime control on date-sharded <code>events_YYYYMMDD</code> .                         | Incremental run scans only shards newer than the last watermark; full-refresh remains available via flag.                                                                                         | Should   |

## 6.B Monitoring & Anomaly Detection

| ID    | Requirement                                                                                                                                       | Rationale                                             | Acceptance Criteria                                                                                         | Priority |
|-------|---------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|----------|
| FR-B1 | The Monitor agent MUST scan all canonical metrics over the run window for anomalies using <code>stats-mcp.detect_anomaly(series, method)</code> . | Math is deterministic and lives in code, not the LLM. | Each metric series is tested; the LLM performs no arithmetic; method → {stl-residual, ewma, robust-zscore}. | Must     |

| ID    | Requirement                                                                                                                                       | Rationale                                     | Acceptance Criteria                                                                                                        | Priority |
|-------|---------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|----------|
| FR-B2 | Anomaly detection MUST account for weekly seasonality and the dataset window (~2020-11-01 to 2021-01-31, including Black Friday / holiday peaks). | Avoid false positives from known seasonality. | Seasonal decomposition applied before flagging; holiday spikes do not trigger spurious anomalies in the labeled benchmark. | Must     |
| FR-B3 | The Monitor MUST emit, per anomaly, the metric, dimension slice, t0/t1 window, magnitude, and direction, then hand off to Decompose.              | Structured handoff for plan-execute-critique. | Anomaly record validates against schema; downstream agents receive typed objects, not prose.                               | Must     |
| FR-B4 | The system MUST consult the Memory suppression list before flagging, so previously-explained anomalies are not re-raised.                         | Reduce alert fatigue; learning.               | A suppressed (metric, segment) pair is skipped; suppression is logged with the prior diagnosis ID.                         | Should   |
| FR-B5 | The Monitor SHOULD rank candidate anomalies by preliminary dollar magnitude to focus the run under the time budget.                               | Time-to-diagnosis < 5 min/run.                | Top-N anomalies by estimated <code>revenue</code> impact are processed first.                                              | Should   |

## 6.C Decomposition (Mix-Shift vs Rate-Change)

| ID    | Requirement                                                                                                                                                                                                                                     | Rationale                                              | Acceptance Criteria                                                                                                                                                                                                                       | Priority |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-C1 | For any aggregate rate change, the Decompose agent MUST call <code>stats-mcp.decompose_change(metric, dim, t0, t1)</code> to split $\hat{I}^R$ into mix, rate, and interaction effects.                                                         | The technical centerpiece; resolves Simpson's paradox. | Output contains <code>mix_effect</code> , <code>rate_effect</code> , <code>interaction</code> per segment and in total; components sum to $\hat{I}^R$ within floating tolerance $1e-6$ .                                                  | Must     |
| FR-C2 | The decomposition MUST implement exactly: $\text{mix} = \hat{I} \hat{I}^w_i \hat{A} \cdot r_i(t0)$ , $\text{rate} = \hat{I} \hat{I}^w_i(t0) \hat{A} \cdot \hat{I}^r_i$ , $\text{interaction} = \hat{I} \hat{I}^w_i \hat{A} \cdot \hat{I}^r_i$ . | Determinism where it matters.                          | A golden-data unit test reproduces hand-computed effects; identity $\hat{I} \hat{I}^w_i \hat{A} \cdot \hat{I}^r_i + \hat{I} \hat{I}^w_i(t0) \hat{A} \cdot \hat{I}^r_i + \hat{I} \hat{I}^w_i \hat{A} \cdot \hat{I}^r_i = \hat{I}^R$ holds. | Must     |
| FR-C3 | Decomposition MUST be runnable across every canonical dimension to locate the dominant driver dimension.                                                                                                                                        | Attribute the change to the right slice.               | The dimension maximizing absolute contribution is reported with its segment-level breakdown.                                                                                                                                              | Must     |
| FR-C4 | The system SHOULD detect and flag Simpson's-paradox cases where the aggregate moves opposite to most segments.                                                                                                                                  | Headline insight; defends against naive reads.         | When $\text{sign}(\hat{I}^R)$ differs from the majority of $\text{sign}(\hat{I}^r_i)$ , a paradox flag is set on the finding.                                                                                                             | Should   |

## 6.D Diagnosis / Root-Cause Analysis

| ID    | Requirement                                                                                                                                                                                                                                                                   | Rationale                      | Acceptance Criteria                                                                                                      | Priority |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|--------------------------------------------------------------------------------------------------------------------------|----------|
| FR-D1 | The Diagnose agent MUST build a hypothesis tree and verify each branch with governed SQL via <code>semantic-mcp.build_query + warehouse-mcp.run_query</code> .                                                                                                                | Grounding over generation.     | Every leaf hypothesis maps to a %1 executed governed query; unverified branches are pruned, not reported.                | Must     |
| FR-D2 | Each hypothesis MUST carry a significance test from <code>stats-mcp.significance_test(a, b)</code> before promotion to a finding.                                                                                                                                             | Statistical defensibility.     | Findings include test statistic, p-value, and effect size; non-significant branches are discarded or marked exploratory. | Must     |
| FR-D3 | Every finding MUST pass the Critic agent, which actively attempts to REFUTE it (mix-shift confound, insufficient sample, seasonality, data quality).                                                                                                                          | Adversarial verify-then-trust. | A finding ships only with a recorded Critic verdict = "survived" and the refutation attempts it withstood.               | Must     |
| FR-D4 | Diagnosis MUST distinguish whether a funnel movement is driven by mix-shift vs rate-change by consuming FR-C1 output, never asserting cause from aggregates alone.                                                                                                            | Core thesis.                   | Each diagnosis cites the decomposition result; aggregate-only causal claims are blocked by the Critic.                   | Must     |
| FR-D5 | The system SHOULD chain funnel steps ( <code>session_start</code> → <code>view_item</code> → <code>add_to_cart</code> → <code>begin_checkout</code> → <code>add_shipping_info</code> → <code>add_payment_info</code> → <code>purchase</code> ) to localize the leakiest step. | Precise root cause.            | The step with the largest adverse delta in step-to-step rate is identified and reported.                                 | Should   |

## 6.E Quantification (Revenue-at-Risk)

| ID    | Requirement                                                                                                                                                         | Rationale                                          | Acceptance Criteria                                                                                                                                  | Priority |
|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-E1 | Every shipped finding MUST carry a dollar revenue-at-risk computed from governed <code>revenue</code> , <code>aoi</code> , and affected session/conversion volumes. | "Every finding carries a dollar impact" principle. | 100% of findings include a <code>revenue_at_risk_usd</code> value derived from reconciled metrics.                                                   | Must     |
| FR-E2 | Revenue-at-risk MUST be expressed as a counterfactual: dollars recoverable if the degraded rate returned to its t0 / forecast baseline.                             | Actionable, comparable sizing.                     | Computation = $\hat{r} \times \text{rate} \times \text{affected\_sessions} \times \text{downstream\_value}$ ; method is documented and reproducible. | Must     |
| FR-E3 | Quantification SHOULD attach a confidence interval to the dollar estimate using the significance test inputs.                                                       | Honest uncertainty.                                | Findings include a low/high band on <code>revenue_at_risk_usd</code> .                                                                               | Should   |

## 6.F Prescription / Experimentation

| ID    | Requirement                                                                                                                                                                                 | Rationale                              | Acceptance Criteria                                                                                                         | Priority |
|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|----------|
| FR-F1 | The Prescribe agent MUST produce a prioritized experiment backlog, each item from <code>experiment-mcp.design_experiment(hypothesis, metric)</code> .                                       | Statistically-defensible action.       | Backlog items are test cards with hypothesis, target metric, and design; ordered by revenue-at-risk $\hat{r}$ — confidence. | Must     |
| FR-F2 | Each experiment card MUST include a sample-size from <code>experiment-mcp.power_analysis(baseline, mde, alpha, power)</code> and a runtime from <code>runtime_estimate(n, traffic)</code> . | Feasibility and rigor.                 | Cards show required $n$ , $\hat{r} \pm$ , power, MDE, and estimated days-to-significance from actual traffic.               | Must     |
| FR-F3 | Prescriptions MUST tie each experiment to the specific finding and revenue-at-risk it addresses.                                                                                            | Traceability from diagnosis to action. | Every card references a finding ID and its <code>revenue_at_risk_usd</code> .                                               | Must     |
| FR-F4 | The system SHOULD flag experiments that are underpowered given available traffic within the dataset window.                                                                                 | Avoid recommending infeasible tests.   | Cards where <code>runtime_estimate</code> exceeds the window are marked "insufficient traffic".                             | Should   |

## 6.G Reporting / Decision Brief

| ID    | Requirement                                                                                                                    | Rationale                            | Acceptance Criteria                                                                                                    | Priority |
|-------|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|------------------------------------------------------------------------------------------------------------------------|----------|
| FR-G1 | The Narrator agent MUST produce an executive "Decision Brief" via <code>report-mcp.render_brief(findings)</code> .             | Proactive, executive-facing output.  | Each run yields one brief covering top findings, each with cause, dollar impact, significance, and recommended action. | Must     |
| FR-G2 | The brief MUST be generated autonomously per scheduled run, not on demand only.                                                | Anti-product: not an ad-hoc chatbot. | A brief is rendered and persisted on every scheduled run without human prompting.                                      | Must     |
| FR-G3 | The system MUST persist briefs via <code>report-mcp.save_diagnosis(...)</code> and support <code>export(format)</code> .       | Audit and distribution.              | Briefs persist with run ID and timestamp; export supports at least markdown/JSON.                                      | Must     |
| FR-G4 | The brief SHOULD surface prior-run context via <code>report-mcp.recall_prior(metric, segment)</code> to show trend continuity. | Learning narrative.                  | Recurring issues reference the prior diagnosis and whether the prescribed action shipped.                              | Should   |

## 6.H Memory & Learning

| ID    | Requirement                                                                                                    | Rationale                         | Acceptance Criteria                                                                                                  | Priority |
|-------|----------------------------------------------------------------------------------------------------------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------|----------|
| FR-H1 | The Memory store MUST persist prior diagnoses, a suppression list, the business glossary, and action-tracking. | Continuity and reduced re-work.   | All four stores are queryable; each diagnosis is retrievable by (metric, segment, date).                             | Must     |
| FR-H2 | Action-tracking MUST record whether a prescribed experiment was implemented and its observed outcome.          | Closed-loop learning.             | Each prescription has a status $\hat{r}$ {open, shipped, won, lost, abandoned} with linkage to the post-test metric. | Should   |
| FR-H3 | The business glossary SHOULD map informal stakeholder terms to canonical metric/dimension names.               | Robust conversational drill-down. | A glossary lookup resolves synonyms to canonical names; misses are logged for curation.                              | Could    |

## 6.I Evaluation

| ID    | Requirement                                                                                         | Rationale               | Acceptance Criteria                                                        | Priority |
|-------|-----------------------------------------------------------------------------------------------------|-------------------------|----------------------------------------------------------------------------|----------|
| FR-I1 | An offline evaluation harness MUST grade root-cause diagnosis accuracy against a labeled benchmark. | Objective quality bar.  | Harness reports root-cause accuracy; target ≥ 85% vs ≥ 45% naive baseline. | Must     |
| FR-I2 | The harness MUST assert 0 hallucinated columns/metrics (100% governed SQL) per run.                 | Grounding guarantee.    | Any non-governed column/metric in executed SQL fails the eval.             | Must     |
| FR-I3 | The harness MUST verify 100% of findings carry a significance test AND a dollar revenue-at-risk.    | Enforce core principle. | Eval fails if any finding lacks either attribute.                          | Must     |
| FR-I4 | The eval harness MUST run in CI (GitHub Actions) alongside <code>dbt build</code> + tests.          | Continuous correctness. | CI is red if accuracy/grounding/coverage thresholds are not met.           | Must     |

6.J Orchestration & Scheduling

| ID    | Requirement                                                                                                                                                                                        | Rationale                            | Acceptance Criteria                                                                        | Priority |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|--------------------------------------------------------------------------------------------|----------|
| FR-J1 | The Orchestrator MUST run the plan-execute-critique loop across the seven agents in order: Orchestrator → Monitor → Decompose → Diagnose → Prescribe → Narrator, with Critic gating every finding. | Multi-agent control flow.            | A run produces a trace showing each agent's invocation and the Critic verdict per finding. | Must     |
| FR-J2 | The system MUST run autonomously on a schedule (Cloud Scheduler / cron).                                                                                                                           | Autonomous operation, not on-demand. | A scheduled trigger executes a full run end-to-end with no human input.                    | Must     |
| FR-J3 | The Orchestrator SHOULD use Opus for orchestrate/critique and Sonnet for high-volume sub-tasks.                                                                                                    | Cost-aware model routing.            | Model assignment matches the policy and is recorded per agent call.                        | Should   |
| FR-J4 | A run MUST fail closed: if any guardrail (reconcile, dry-run cost, Critic) fails, affected findings are withheld, not shipped.                                                                     | Trust.                               | Withheld findings appear in the trace with the failed gate, not in the brief.              | Must     |

6.K Conversational Drill-Down (Secondary)

| ID    | Requirement                                                                                                                                                            | Rationale                                           | Acceptance Criteria                                                                                                | Priority |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|----------|
| FR-K1 | The system MAY answer follow-up questions about a shipped brief, still routing all SQL through <code>semantic-mcp</code> and all math through <code>stats-mcp</code> . | Drill-down is secondary, not an ad-hoc SQL chatbot. | Conversational answers cite governed metrics only; no raw SQL is authored by the LLM.                              | Should   |
| FR-K2 | Drill-down MUST refuse questions requiring non-governed metrics/dimensions and explain the limitation.                                                                 | Bounded, grounded interface.                        | Out-of-vocabulary requests return a refusal referencing <code>list_dimensions()</code> / <code>get_metric</code> . | Should   |
| FR-K3 | Drill-down COULD support natural-language reference to prior diagnoses via Memory.                                                                                     | Convenience.                                        | "Why did this happen last week?" resolves to a stored diagnosis.                                                   | Could    |

7. Non-Functional Requirements

Non-functional requirements (NFRs) define the quality attributes Helios must satisfy. Each carries an NFR-ID, a measurable target consistent with the Foundation success metrics, and a verification method.

7.1 Performance / Latency

| ID     | Target                                                                                                                                                             | Verification                                              |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| NFR-P1 | Time-to-diagnosis < 5 minutes per autonomous run (end-to-end: Monitor → Narrator).                                                                                 | Timed CI run on the full window; p95 across runs < 5 min. |
| NFR-P2 | <code>warehouse-mcp.dry_run(sql)</code> returns cost/bytes in < 2 s; <code>run_query</code> for any single governed query returns in < 30 s on the sample dataset. | Latency assertions in integration tests.                  |

| ID     | Target                                                                              | Verification                        |
|--------|-------------------------------------------------------------------------------------|-------------------------------------|
| NFR-P3 | Decomposition for a single dimension completes in < 1 s in <code>stats-mcp</code> . | Benchmark unit test on golden data. |

## 7.2 Cost (BigQuery Byte & LLM Token Budgets)

| ID     | Target                                                                                                                                       | Verification                                                                                                          |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| NFR-C1 | Total BigQuery bytes scanned per run MUST stay under a fixed byte budget; every query is gated by <code>dry_run</code> before execution.     | Run aborts a query whose <code>dry_run</code> bytes exceed the per-query cap; run total is logged and asserted in CI. |
| NFR-C2 | Incremental dbt + date-shard pruning MUST avoid full-table scans of <code>events_*</code> on routine runs.                                   | Query plan shows only required <code>events_YYYYMMDD</code> shards scanned.                                           |
| NFR-C3 | LLM token spend per run MUST stay within a configured budget, using Opus only for orchestrate/critique and Sonnet for high-volume sub-tasks. | Per-agent token usage logged; run total asserted against budget.                                                      |

## 7.3 Reliability / Availability

| ID     | Target                                                                                         | Verification                                                                                        |
|--------|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| NFR-R1 | A scheduled run MUST complete or fail closed with a recorded reason; no silent partial briefs. | Trace shows terminal status; partial outputs never reach <code>save_diagnosis</code> as "complete". |
| NFR-R2 | MCP tool calls MUST retry transient failures with bounded backoff (â‰¥3 attempts).             | Fault-injection test confirms retry then graceful failure.                                          |
| NFR-R3 | Re-running a failed run MUST be idempotent (no duplicate diagnoses for the same window).       | Duplicate-run test asserts a single persisted diagnosis per (run window, finding).                  |

## 7.4 Correctness & Trust Guardrails

| ID     | Target                                                                                                    | Verification                                                     |
|--------|-----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| NFR-T1 | 0 hallucinated columns/metrics â€” 100% of executed SQL is governed via <code>semantic-mcp</code> .       | Eval harness (FR-I2) + static scan for raw <code>SELECT</code> . |
| NFR-T2 | 100% of shipped findings carry a significance test AND a dollar revenue-at-risk AND a recommended action. | Eval harness (FR-I3) gate.                                       |
| NFR-T3 | Every metric value reconciles to <code>warehouse-mcp.reconcile</code> within 0.5%.                        | Reconciliation assertion per finding.                            |
| NFR-T4 | Every finding survives the Critic's refutation attempts before shipping.                                  | Critic verdict recorded; "not survived" â†’ withheld.            |

## 7.5 Security & PII / Privacy

| ID     | Target                                                                                                                                  | Verification                                                                                  |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| NFR-S1 | No PII egress. The dataset is obfuscated; <code>user_pseudo_id</code> is a device/cookie key only and MUST NOT be exported in briefs.   | Output scan asserts no <code>user_pseudo_id</code> , raw IDs, or geo below region in exports. |
| NFR-S2 | BigQuery access MUST use least-privilege service-account credentials scoped to the public dataset and the project's transformed tables. | IAM review; no broad project-owner roles.                                                     |
| NFR-S3 | Secrets (service-account keys, LLM API keys) MUST NOT appear in logs, traces, or the repo.                                              | Secret-scanning in CI; redaction in the logging layer.                                        |

## 7.6 Observability (Logging, Tracing, Audit Trail)

| ID     | Target                                                                                                                                         | Verification                                  |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| NFR-O1 | Every run MUST emit a full audit trail: each agent call, each MCP tool call with inputs/outputs, every executed SQL, and every Critic verdict. | Trace artifact persisted per run; replayable. |



| ID     | Target                                                                                                                       | Verification                                    |
|--------|------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| NFR-O2 | Each finding MUST be traceable from Decision Brief âdiagnosis â significance test â governed SQL â reconciled total. | Lineage walk-through on a sample finding in CI. |
| NFR-O3 | Structured logs MUST include run ID, agent name, model used, tokens, bytes scanned, and latency.                             | Log schema validated.                           |

## 7.7 Reproducibility / Determinism

| ID     | Target                                                                                                                                             | Verification                                                          |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| NFR-D1 | All statistical computation is deterministic and lives in <code>stats-mcp</code> code, never in the LLM; identical inputs yield identical outputs. | Repeated decomposition/significance calls produce bit-stable results. |
| NFR-D2 | A pinned run (fixed dataset window, seeds, model versions) MUST reproduce the same findings set.                                                   | Two pinned runs compared; finding set is identical.                   |
| NFR-D3 | dbt models MUST be deterministic given the same source shards.                                                                                     | <code>dbt build</code> re-run yields identical fact rows.             |

## 7.8 Maintainability

| ID     | Target                                                                                                        | Verification                                                                    |
|--------|---------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| NFR-M1 | Adding a new governed metric MUST require only a semantic-layer definition + dbt model, no agent code change. | New-metric exercise touches only <code>models/semantic</code> and the registry. |
| NFR-M2 | All models, metrics, and dimensions MUST follow snake_case and the canonical naming.                          | Linter enforces names; CI fails on deviation.                                   |
| NFR-M3 | MCP server tool signatures MUST be the exact canonical names/tools and stable across releases.                | Contract tests against the five servers' tool schemas.                          |

## 7.9 Scalability

| ID     | Target                                                                                                                                                              | Verification                                                                                                 |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| NFR-X1 | The pipeline MUST scale to additional date shards (extending beyond the ~2020-11 to 2021-01 window) without query-cost blowup, via incremental + partition pruning. | Synthetic shard extension stays within byte budget per run.                                                  |
| NFR-X2 | The architecture SHOULD support additional dimensions/metrics without rewriting the decomposition core.                                                             | New dimension added to <code>list_dimensions()</code> flows through <code>decompose_change</code> unchanged. |

## 7.10 Data-Quality SLAs

| ID     | Target                                                                                                                                                                                                                                                                                                                                                                                                                            | Verification                                                       |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| NFR-Q1 | dbt tests MUST enforce: unique <code>transaction_id</code> in <code>fct_orders</code> ; non-null session key ( <code>user_pseudo_id</code> , <code>ga_session_id</code> ); <code>session_conversion_rate</code> â [0,1]; funnel monotonicity ( <code>purchasing_sessions</code> â <code>begin_checkout_sessions</code> â <code>add_to_cart_sessions</code> â <code>view_item_sessions</code> â <code>sessions</code> ). | dbt test suite green in CI.                                        |
| NFR-Q2 | Rows with null <code>ga_session_id</code> or malformed <code>event_params</code> MUST be quarantined and counted, not silently dropped.                                                                                                                                                                                                                                                                                           | Quarantine table row counts reported; drop-rate threshold alarmed. |
| NFR-Q3 | <code>revenue</code> (sum <code>purchase_revenue_in_usd</code> ) and <code>transactions</code> (distinct <code>transaction_id</code> ) MUST reconcile to canonical totals.                                                                                                                                                                                                                                                        | <code>reconcile(metric, grain)</code> parity check.                |

## 7.11 Portability

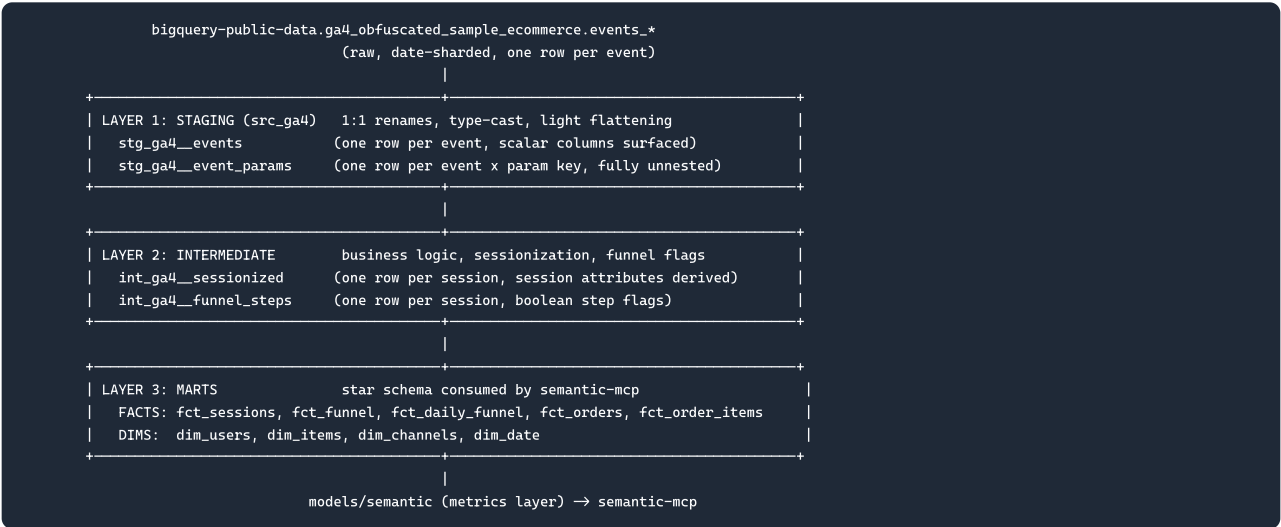
| ID     | Target                                                                                                                                     | Verification                                     |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| NFR-Z1 | Python MCP servers MUST run on Python 3.11 with pinned dependencies (scipy, statsmodels, prophet/pmdarima, pandas, google-cloud-bigquery). | Lockfile build reproduces the environment in CI. |

| ID     | Target                                                                                                                                       | Verification                                                         |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| NFR-Z2 | The system MUST run both in GitHub Actions CI and via the scheduler (Cloud Scheduler/cron) using the same containerized entrypoint.          | One image runs in both contexts; no environment-specific code path.  |
| NFR-Z3 | Warehouse access SHOULD be abstracted behind <code>warehouse-mcp</code> so the BigQuery backend can be swapped without changing agent logic. | Agents reference only MCP tools, never the BigQuery client directly. |

## 8. Data Model

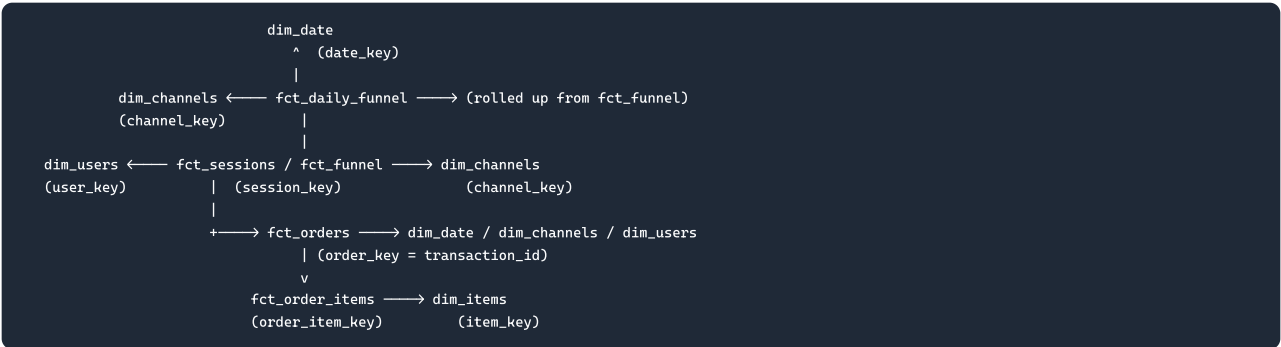
Helios models the GA4 export as a four-layer dbt DAG: **raw GA4 -> staging -> intermediate -> marts**. Every transformation is governed; the LLM never touches raw events directly. It composes governed metrics via `semantic-mcp`, which itself only references the marts described here. This separation is what guarantees the FOUNDATION principle "grounding over generation" and the success target of **0 hallucinated columns / 100% governed SQL**.

### 8.1 Layer Overview



Staging models are materialized as `view` (cheap, always-fresh casts). Intermediate models are `ephemeral` or `view`. Marts are materialized as `table` (or incremental on `event_date` for the large fact tables) so that the autonomous run hits a fixed, predictable BigQuery byte budget.

### 8.2 Star Schema Diagram



`fct_funnel` is the grain-defining session fact; `fct_orders` is the grain-defining transaction fact. All dimensions conform (shared `dim_date`, `dim_channels`, `dim_users`) so the Decompose agent can pivot any metric across `device_category`, `channel_group`, `country`, etc.

### 8.3 Mart Catalog “Fact Tables

`fct_sessions`

- **Grain:** one row per session.
- **Primary key:** `session_key`.
- **Description:** session-level attributes, volumes, engagement.

| column               | type      | key                | description                                            |
|----------------------|-----------|--------------------|--------------------------------------------------------|
| session_key          | STRING    | PK                 | `to_hex(md5(user_pseudo_id                             |
| user_pseudo_id       | STRING    | FK -> dim_users    | device/cookie key (de facto user)                      |
| ga_session_id        | INT64     |                    | session id within user                                 |
| date_key             | DATE      | FK -> dim_date     | session start date (from min event_timestamp)          |
| session_start_ts     | TIMESTAMP |                    | first event_timestamp of session                       |
| session_end_ts       | TIMESTAMP |                    | last event_timestamp of session                        |
| channel_key          | STRING    | FK -> dim_channels | session-scoped channel group surrogate                 |
| source               | STRING    |                    | session-scoped source                                  |
| medium               | STRING    |                    | session-scoped medium                                  |
| campaign             | STRING    |                    | session-scoped campaign                                |
| landing_page         | STRING    |                    | first <code>page_location</code> of session            |
| device_category      | STRING    |                    | mobile / desktop / tablet                              |
| operating_system     | STRING    |                    | device.operating_system                                |
| browser              | STRING    |                    | device.web_info.browser                                |
| country              | STRING    |                    | geo.country                                            |
| region               | STRING    |                    | geo.region                                             |
| is_new_user          | BOOL      |                    | TRUE if ga_session_number = 1                          |
| ga_session_number    | INT64     |                    | session ordinal for the user                           |
| event_count          | INT64     |                    | events in session                                      |
| engaged_session      | BOOL      |                    | session_engaged = "1" OR engagement_time_msec >= 10000 |
| engagement_time_msec | INT64     |                    | summed engagement time                                 |

#### fct\_funnel

- **Grain:** one row per session.
- **Primary key:** `session_key`.
- **Description:** the canonical session fact carrying the macro-funnel boolean flags plus per-session revenue. Joins 1:1 to `fct_sessions`.

| column              | type   | key                     | description                                            |
|---------------------|--------|-------------------------|--------------------------------------------------------|
| session_key         | STRING | PK / FK -> fct_sessions | session id                                             |
| user_pseudo_id      | STRING | FK -> dim_users         | user key                                               |
| date_key            | DATE   | FK -> dim_date          | session date                                           |
| channel_key         | STRING | FK -> dim_channels      | channel group                                          |
| device_category     | STRING |                         | dimension                                              |
| country             | STRING |                         | dimension                                              |
| is_new_user         | BOOL   |                         | new vs returning                                       |
| did_session_start   | BOOL   |                         | always TRUE (denominator anchor)                       |
| reached_view_item   | BOOL   |                         | session reached view_item or beyond (max-downstream)   |
| reached_add_to_cart | BOOL   |                         | session reached add_to_cart or beyond (max-downstream) |

| column                    | type    | key | description                                                  |
|---------------------------|---------|-----|--------------------------------------------------------------|
| reached_begin_checkout    | BOOL    |     | session reached begin_checkout or beyond (max-downstream)    |
| reached_add_shipping_info | BOOL    |     | session reached add_shipping_info or beyond (max-downstream) |
| reached_add_payment_info  | BOOL    |     | session reached add_payment_info or beyond (max-downstream)  |
| reached_purchase          | BOOL    |     | session reached purchase (fired purchase)                    |
| session_revenue           | FLOAT64 |     | sum purchase_revenue_in_usd in session (deduped)             |

#### fct\_daily\_funnel

- **Grain:** one row per ( date\_key , channel\_key , device\_category , country , is\_new\_user ).
- **Primary key:** composite of the grain columns ( daily\_funnel\_key surrogate).
- **Description:** pre-aggregated daily funnel, the primary feed for the Monitor (time-series anomaly) and Decompose (mix-shift) agents. Aggregates fct\_funnel .

| column                     | type    | description                    |
|----------------------------|---------|--------------------------------|
| daily_funnel_key           | STRING  | md5 of grain columns           |
| date_key                   | DATE    | day                            |
| channel_key                | STRING  | channel group                  |
| device_category            | STRING  | dimension                      |
| country                    | STRING  | dimension                      |
| is_new_user                | BOOL    | dimension                      |
| sessions                   | INT64   | count distinct session_key     |
| users                      | INT64   | count distinct user_pseudo_id  |
| new_users                  | INT64   | users where is_new_user        |
| returning_users            | INT64   | users where not is_new_user    |
| engaged_sessions           | INT64   | sessions where engaged_session |
| view_item_sessions         | INT64   | sum reached_view_item          |
| add_to_cart_sessions       | INT64   | sum reached_add_to_cart        |
| begin_checkout_sessions    | INT64   | sum reached_begin_checkout     |
| add_shipping_info_sessions | INT64   | sum reached_add_shipping_info  |
| add_payment_info_sessions  | INT64   | sum reached_add_payment_info   |
| purchasing_sessions        | INT64   | sum reached_purchase           |
| transactions               | INT64   | distinct transaction_id        |
| revenue                    | FLOAT64 | sum session_revenue            |

Rates ( session\_conversion\_rate , view\_to\_cart\_rate , etc.) are NOT stored here; they are computed in the semantic layer from these additive counts so they stay re-aggregatable across any slice.

#### fct\_orders

- **Grain:** one row per transaction (distinct transaction\_id ).
- **Primary key:** order\_key = transaction\_id .
- **Description:** deduped order header. One purchase event per transaction after dedup.

| column      | type   | key                | description         |
|-------------|--------|--------------------|---------------------|
| order_key   | STRING | PK                 | transaction_id      |
| session_key | STRING | FK -> fct_sessions | originating session |

| column                | type      | key                | description                        |
|-----------------------|-----------|--------------------|------------------------------------|
| user_pseudo_id        | STRING    | FK -> dim_users    | buyer                              |
| date_key              | DATE      | FK -> dim_date     | purchase date                      |
| channel_key           | STRING    | FK -> dim_channels | attributed channel                 |
| order_ts              | TIMESTAMP |                    | purchase event timestamp           |
| gross_revenue         | FLOAT64   |                    | purchase_revenue_in_usd            |
| refund_value_in_usd   | FLOAT64   |                    | refund (usually 0/NULL)            |
| net_revenue           | FLOAT64   |                    | gross_revenue - coalesce(refund,0) |
| shipping_value_in_usd | FLOAT64   |                    | ecommerce.shipping_value_in_usd    |
| tax_value_in_usd      | FLOAT64   |                    | ecommerce.tax_value_in_usd         |
| total_item_quantity   | INT64     |                    | items in order                     |
| unique_items          | INT64     |                    | distinct items in order            |

#### fct\_order\_items

- **Grain:** one row per ( transaction\_id , item line).
- **Primary key:** order\_item\_key = md5(transaction\_id + item\_id + row\_number).
- **Description:** exploded items[] array for purchases. Feeds item-category diagnoses.

| column              | type    | key              | description               |
|---------------------|---------|------------------|---------------------------|
| order_item_key      | STRING  | PK               | surrogate                 |
| order_key           | STRING  | FK -> fct_orders | transaction               |
| item_key            | STRING  | FK -> dim_items  | item_id                   |
| item_name           | STRING  |                  | items.item_name           |
| item_category       | STRING  |                  | items.item_category       |
| quantity            | INT64   |                  | items.quantity            |
| item_revenue_in_usd | FLOAT64 |                  | items.item_revenue_in_usd |
| price_in_usd        | FLOAT64 |                  | items.price_in_usd        |
| coupon              | STRING  |                  | items.coupon              |

## 8.4 Mart Catalog “ Dimension Tables

| table        | grain / PK                                  | key columns                                                                                                            | description                                                      |
|--------------|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| dim_users    | one row per user_pseudo_id (PK user_key )   | first_touch_ts, first_touch_source, first_touch_medium, first_channel_key, total_sessions, total_revenue, is_purchaser | user roll-up; first-touch attribution from traffic_source struct |
| dim_items    | one row per item_id (PK item_key )          | item_name, item_brand, item_category, item_category2..5, current_price_in_usd                                          | product catalog distilled from items[]                           |
| dim_channels | one row per channel_group (PK channel_key ) | channel_group, channel_order                                                                                           | the 10 GA4 default channel groups                                |
| dim_date     | one row per calendar day (PK date_key )     | date_key, day, week, month, year, day_of_week, is_weekend                                                              | conformed date spine 2020-11-01..2021-01-31                      |

## 8.5 Sessionization (in depth)

GA4 does not ship a session row; a session is reconstructed from events sharing the same (user\_pseudo\_id, ga\_session\_id) . ga\_session\_id lives inside event\_params and must be unnested. The canonical surrogate is session\_key = md5(user\_pseudo\_id || '-' || ga\_session\_id) .

```

-- int_ga4_sessionized
with params as (
  select
    user_pseudo_id,
    event_timestamp,
    event_name,
    (select ep.value.int_value from unnest(event_params) ep where ep.key='ga_session_id') as ga_session_id,
    (select ep.value.int_value from unnest(event_params) ep where ep.key='ga_session_number') as ga_session_number,
    (select ep.value.string_value from unnest(event_params) ep where ep.key='page_location') as page_location,
    (select coalesce(ep.value.string_value, '(direct)') from unnest(event_params) ep where ep.key='source') as ev_source,
    (select coalesce(ep.value.string_value, '(none)') from unnest(event_params) ep where ep.key='medium') as ev_medium,
    (select ep.value.int_value from unnest(event_params) ep where ep.key='engagement_time_msec') as engagement_time_msec,
    (select ep.value.string_value from unnest(event_params) ep where ep.key='session_engaged') as session_engaged,
    device.category as device_category, device.operating_system, device.web_info.browser,
    geo.country, geo.region, traffic_source.source as ut_source, traffic_source.medium as ut_medium
  from {{ source('src_ga4', 'events') }}
  where table_suffix between '20201101' and '20210131'
)
select
  to_hex(md5(user_pseudo_id || '-' || cast(ga_session_id as string))) as session_key,
  user_pseudo_id, ga_session_id, any_value(ga_session_number) as ga_session_number,
  nt(event_timestamp) as session_start_micros,
  mt(event_timestamp) as session_end_micros,
  -- landing_page = page_location of the earliest event carrying one
  array_agg(page_location ignore nulls order by event_timestamp limit 1)[safe_offset(0)] as landing_page,
  -- session-scoped source/medium: first non-null event-param value; fall back to user first-touch
  coalesce(array_agg(ev_source ignore nulls order by event_timestamp limit 1)[safe_offset(0)], any_value(ut_source), '(direct)') as source,
  coalesce(array_agg(ev_medium ignore nulls order by event_timestamp limit 1)[safe_offset(0)], any_value(ut_medium), '(none)') as medium,
  any_value(device_category) as device_category, any_value(operating_system) as operating_system,
  any_value(browser) as browser, any_value(country) as country, any_value(region) as region,
  countif(true) as event_count,
  max(coalesce(engagement_time_msec, 0)) as engagement_time_msec,
  logical_or(session_engaged='1') as session_engaged_flag
from params
where ga_session_id is not null
group by user_pseudo_id, ga_session_id

```

**Engagement:** engaged\_session = session\_engaged\_flag OR engagement\_time\_msec ≥ 10000 , matching GA4's engaged-session definition;  
engagement\_rate = engaged\_sessions / sessions . **Landing page** is the page\_location of the earliest-timestamp event with a non-null value  
(typically the session\_start / first\_visit /first page\_view ).

The **traffic\_source gotcha** is handled here explicitly: traffic\_source.\* is USER-LEVEL first-touch attribution, not session source. So Helios prefers the session-scoped event\_params.source/medium (which reflect the actual acquisition of *this* session) and only falls back to the user-level traffic\_source struct when the session params are null.

## 8.6 User Identity Resolution and Limits

The user key is user\_pseudo\_id (device/cookie id). user\_id is almost always NULL in this obfuscated dataset, so cross-device stitching is impossible – a single human on phone + laptop appears as two users. dim\_users therefore resolves identity at the device-cookie grain: first-touch timestamp = min(user\_first\_touch\_timestamp) , first-touch channel from the traffic\_source struct, and is\_new\_user derived from ga\_session\_number = 1 / the first\_visit event. Returning-user counts double-count cookie churn (cleared cookies look like new users), so all user-grain metrics ( new\_users , returning\_users , revenue\_per\_user /ARPU) are caveated by the Critic agent as cookie-based approximations, never true person-level counts.

## 9. Event Model

### 9.1 Canonical Event Taxonomy

GA4 rows are events. The table below catalogs every canonical event\_name present in the dataset, when it fires, its load-bearing params, and the funnel stage it maps to.

| event_name    | fires when                  | key event_params                         | funnel stage           |
|---------------|-----------------------------|------------------------------------------|------------------------|
| session_start | first event of a session    | ga_session_id, ga_session_number         | session_start (anchor) |
| first_visit   | first ever event for a user | ga_session_id                            | new-user flag          |
| page_view     | any page load               | page_location, page_title, page_referrer | (engagement)           |

| event_name        | fires when               | key event_params                          | funnel stage             |
|-------------------|--------------------------|-------------------------------------------|--------------------------|
| view_promotion    | promo impression         | items[], promotion_id                     | top of funnel            |
| view_item_list    | category/list page view  | items[], item_list_name                   | top of funnel            |
| view_item         | product detail page view | items[], page_location                    | <b>view_item</b>         |
| select_item       | item clicked in a list   | items[]                                   | micro: list -> PDP       |
| add_to_cart       | item added to cart       | items[], value, currency                  | <b>add_to_cart</b>       |
| view_cart         | cart viewed              | items[], value                            | micro: cart              |
| begin_checkout    | checkout initiated       | items[], value, coupon                    | <b>begin_checkout</b>    |
| add_shipping_info | shipping tier entered    | shipping_tier, value                      | <b>add_shipping_info</b> |
| add_payment_info  | payment method entered   | payment_type, value                       | <b>add_payment_info</b>  |
| purchase          | order completed          | transaction_id, value, items[], ecommerce | <b>purchase</b>          |
| scroll            | 90% scroll reached       | percent_scrolled                          | engagement               |
| click             | outbound/UI click        | link_url                                  | engagement               |
| user_engagement   | engagement heartbeat     | engagement_time_msec                      | engagement               |

The seven bolded stages plus `session_start` constitute the macro funnel (Section 10).

## 9.2 event\_params Flattening Patterns

`event_params` is `ARRAY<STRUCT<key STRING, value STRUCT<string_value, int_value, float_value, double_value>>>`. The value lives in exactly one typed sub-field, so extraction always targets the correct slot. The governed pattern is a correlated scalar subquery (avoids a fan-out join):

```
-- scalar extractors (used in staging)
(select ep.value.string_value from unnest(event_params) ep where ep.key = 'page_location') as page_location,
(select ep.value.int_value from unnest(event_params) ep where ep.key = 'ga_session_id') as ga_session_id,
(select ep.value.double_value from unnest(event_params) ep where ep.key = 'engagement_time_msec') as engagement_time_msec
```

A reusable dbt macro standardizes this “the canonical `get_event_param` helper:

```
{% macro get_event_param(key, type='string') %}
  (select ep.value.{{ type }}_value from unnest(event_params) ep where ep.key = '{{ key }}')
{% endmacro %}
-- usage: {{ get_event_param('ga_session_id','int') }} as ga_session_id
```

`stg_ga4__event_params` provides the fully-unnested alternative (one row per event x param), useful when you need to scan many keys at once:

```
-- stg_ga4__event_params: one row per (event, param key)
select
  to_hex(md5(user_pseudo_id || '-' || cast(event_timestamp as string) || '-' || event_name)) as event_key,
  user_pseudo_id, event_name, event_timestamp,
  ep.key as param_key,
  ep.value.string_value as string_value, ep.value.int_value as int_value,
  ep.value.float_value as float_value, ep.value.double_value as double_value
from {{ source('src_ga4','events') }} , unnest(event_params) ep
```

## 9.3 The items[] Array and the ecommerce Struct

`items` is `ARRAY<STRUCT< ... >>` carried on `view_item`, `add_to_cart`, `begin_checkout`, and `purchase`. Each element holds `item_id`, `item_name`, `item_brand`, `item_category` (+ `item_category2..5`), `item_variant`, `price_in_usd`, `price`, `quantity`, `item_revenue_in_usd`, `coupon`. To analyze items, `CROSS JOIN UNNEST(items)`:

```
select user_pseudo_id, i.item_id, i.item_name, i.item_category,
       i.quantity, i.item_revenue_in_usd, i.price_in_usd
from {{ source('src_ga4','events') }} , unnest(items) i
where event_name = 'purchase'
```

The scalar `ecommerce` struct (on `purchase`) carries order-level totals: `total_item_quantity`, `purchase_revenue_in_usd`, `purchase_revenue`, `refund_value_in_usd`, `shipping_value_in_usd`, `tax_value_in_usd`, `transaction_id`, `unique_items`. Order-level revenue comes from `ecommerce.purchase_revenue_in_usd`; line-level revenue from the unnested `items.item_revenue_in_usd`. The two need not sum identically (order revenue excludes shipping/tax depending on config) â€” Section 11 resolves which is authoritative.

## 9.4 Example: extracting session id, page\_location, source/medium

```
select
  user_pseudo_id,
  {{ get_event_param('ga_session_id','int') }} as ga_session_id,
  {{ get_event_param('page_location') }} as page_location,
  coalesce({{ get_event_param('source') }}, traffic_source.source) as session_source,
  coalesce({{ get_event_param('medium') }}, traffic_source.medium) as session_medium
from {{ source('src_ga4','events') }}
where event_name = 'page_view'
```

This is the only sanctioned path to source/medium: session-scoped `event_params` first, user first-touch `traffic_source` as fallback â€” honoring the gotcha.

# 10. Funnel Definitions

## 10.1 Canonical Macro Funnel

```
session_start → view_item → add_to_cart → begin_checkout → add_shipping_info → add_payment_info → purchase
```

Helios reports **step-to-step** conversion (each stage / prior stage) AND **overall** `session_conversion_rate = purchasing_sessions / sessions`. The funnel is **session-scoped** and uses **max-downstream** ("reached this step or beyond") semantics: a session `reached_begin_checkout` if it fired `begin_checkout` OR any later funnel-stage event (`add_shipping_info`, `add_payment_info`, `purchase`) at any point in the session. This rolls each flag forward to every downstream stage, so the macro funnel is **MONOTONIC by construction**: `sessions ≥ reached_view_item ≥ reached_add_to_cart ≥ reached_begin_checkout ≥ reached_add_shipping_info ≥ reached_add_payment_info ≥ reached_purchase`. Monotonicity prevents a downstream stage count from ever exceeding an upstream one (e.g. it guarantees `view_to_cart_rate ≤ 1` always holds) and avoids dropping sessions that, e.g., re-add a cart item without re-viewing the PDP. (An ordered variant is available for the Critic to test step-skipping hypotheses but is not the default.)

## 10.2 Micro-Funnels (Rates)

- `view_to_cart_rate = add_to_cart_sessions / view_item_sessions`
- `cart_to_checkout_rate = begin_checkout_sessions / add_to_cart_sessions`
- `checkout_to_purchase_rate = purchasing_sessions / begin_checkout_sessions`
- `cart_abandonment_rate = 1 - (purchasing_sessions / add_to_cart_sessions)`
- `checkout_abandonment_rate = 1 - (purchasing_sessions / begin_checkout_sessions)`

## 10.3 Session-Level Boolean Flags

```
-- int_ga4_funnel_steps : one row per session, max-downstream (monotonic) flags
-- each flag = "reached this step OR any later funnel stage"
select
  to_hex(md5(user_pseudo_id|| '-' || cast(ga_session_id as string))) as session_key,
  user_pseudo_id,
  true as did_session_start,
  logical_or(event_name in ('view_item','add_to_cart','begin_checkout','add_shipping_info','add_payment_info','purchase')) as reached_view_item,
  logical_or(event_name in ('add_to_cart','begin_checkout','add_shipping_info','add_payment_info','purchase')) as reached_add_to_cart,
  logical_or(event_name in ('begin_checkout','add_shipping_info','add_payment_info','purchase')) as reached_begin_checkout,
  logical_or(event_name in ('add_shipping_info','add_payment_info','purchase')) as reached_add_shipping_info,
  logical_or(event_name in ('add_payment_info','purchase')) as reached_add_payment_info,
  logical_or(event_name = 'purchase') as reached_purchase
from (
  select user_pseudo_id, event_name,
    {{ get_event_param('ga_session_id','int') }} as ga_session_id
  from {{ source('src_ga4','events') }}
)
where ga_session_id is not null
group by user_pseudo_id, ga_session_id
```



## 10.4 Session vs User Scope; Ordered vs Unordered; Time Windows

**Scope:** the default funnel is session-scoped (denominator = `sessions`), which isolates within-visit friction. A user-scoped variant (denominator = `users`, "did this user ever purchase in window") is exposed for retention/LTV questions but never mixed with session rates in one finding â€” the Critic flags scope-mixing as a refutation.

**Ordered vs unordered:** default is unordered (occurrence). Ordered logic, when requested, requires the min-timestamp of each step to be monotonically increasing per session; it is strictly stricter and yields lower step rates.

**Time window:** funnels are computed inside a fixed analysis window (e.g. a week or the t0->t1 comparison windows the Decompose agent receives). A session is attributed to the day of its `session_start_micros`, so a session spanning midnight counts once, on its start day.

## 10.5 Worked Example â€” building `fct_funnel`

```
-- fct_funnel : session grain, flags + revenue, joined to session attributes
with steps as ( select * from {{ ref('int_ga4_funnel_steps') }} ),
sess as ( select * from {{ ref('int_ga4_sessionized') }} ),
rev as (
  -- deduped session revenue: one purchase_revenue per distinct transaction_id
  select to_hex(md5(user_pseudo_id || '-' || cast(ga_session_id as string))) as session_key,
         sum(txn_rev) as session_revenue
  from (
    select user_pseudo_id,
           {{ get_event_param('ga_session_id','int') }} as ga_session_id,
           ecommerce.transaction_id as transaction_id,
           any_value(ecommerce.purchase_revenue_in_usd) as txn_rev
    from {{ source('src_ga4','events') }}
    where event_name='purchase' and ecommerce.transaction_id is not null
    group by 1,2,3 -- dedup duplicate purchase rows per transaction
  )
  group by session_key
)
select
  s.session_key, s.user_pseudo_id, date(timestamp_micros(s.session_start_micros)) as date_key,
  c.channel_key, s.device_category, s.country, (s.ga_session_number = 1) as is_new_user,
  st.did_session_start, st.reached_view_item, st.reached_add_to_cart, st.reached_begin_checkout,
  st.reached_add_shipping_info, st.reached_add_payment_info, st.reached_purchase,
  coalesce(r.session_revenue, 0.0) as session_revenue
from sess s
join steps st using (session_key)
left join rev r using (session_key)
left join {{ ref('dim_channels') }} c on c.channel_group = {{ derive_channel_group('s.source','s.medium') }}
```

`session_conversion_rate` is then `countif(reached_purchase) / count(*)` over `fct_funnel`, always recomputed from additive counts so it re-aggregates correctly across any dimension the Decompose agent slices.

# 11. Revenue Definitions

## 11.1 Precise Metric Definitions

- **gross\_revenue** = `SUM(purchase_revenue_in_usd)` over `purchase` events, **deduped by transaction\_id** (one revenue figure per distinct `transaction_id`).
- **net\_revenue** = `gross_revenue - SUM(refund_value_in_usd)`.
- **revenue** (canonical) = `gross_revenue` (the headline figure unless a finding explicitly concerns refunds).
- **item\_revenue** = `SUM(items.item_revenue_in_usd)` over unnested purchase items.
- **transactions** = `COUNT(DISTINCT transaction_id)`.
- **aov** = `revenue / transactions`.
- **items\_per\_transaction** = `SUM(total_item_quantity) / transactions`.
- **revenue\_per\_session** (RPS) = `revenue / sessions`.
- **revenue\_per\_user** (ARPU) = `revenue / users`.

## 11.2 Currency, Shipping, and Tax

All money uses the `_in_usd` fields (`purchase_revenue_in_usd`, `item_revenue_in_usd`, `refund_value_in_usd`, `shipping_value_in_usd`, `tax_value_in_usd`, `price_in_usd`). The non-USD twins (`purchase_revenue`, `price`) are ignored to avoid mixed-currency aggregation. `purchase_revenue_in_usd` is product revenue **excluding shipping and tax**; shipping and tax are stored separately on `fct_orders` and never

folded into `revenue` (so `aov` reflects merchandise value). The `_in_usd` fields are GA4's normalized columns, so no FX conversion is performed by Helios.

### 11.3 Dedup Gotchas

GA4 exports can emit **duplicate purchase rows** for one `transaction_id` (retries, multi-stream). Summing `purchase_revenue_in_usd` raw double-counts. The fix is to collapse to one row per `transaction_id` first ( `any_value` of the revenue), then sum. **NULL transaction\_id** rows are purchases that GA4 failed to tag (test orders, mis-instrumented) – they are excluded from `transactions` and `gross_revenue` but logged by the Critic as a data-quality caveat, since a spike in NULL-id purchases can masquerade as a revenue drop.

### 11.4 Refunds

Refunds appear as a non-null `refund_value_in_usd` on the purchase event (this dataset rarely has a separate `refund` event). `net_revenue` subtracts it; in practice refunds are near-zero in the sample window, so `gross_revenue`  $\approx$  `net_revenue`, but the distinction is preserved for the revenue-at-risk dollar quantification every finding carries.

### 11.5 SQL – fct\_orders revenue

```
-- fct_orders : one deduped row per transaction_id
with purch as (
  select
    ecommerce.transaction_id as order_key,
    to_hex(md5(user_pseudo_id || '-' || cast(
      (select ep.value_int_value from unnest(event_params) ep where ep.key='ga_session_id') as string))) as session_key,
    user_pseudo_id,
    timestamp_micros(event_timestamp) as order_ts,
    any_value(ecommerce.purchase_revenue_in_usd) as gross_revenue,
    any_value(ecommerce.refund_value_in_usd) as refund_value_in_usd,
    any_value(ecommerce.shipping_value_in_usd) as shipping_value_in_usd,
    any_value(ecommerce.tax_value_in_usd) as tax_value_in_usd,
    any_value(ecommerce.total_item_quantity) as total_item_quantity,
    any_value(ecommerce.unique_items) as unique_items
  from {{ source('src_ga4','events') }}
  where event_name = 'purchase' and ecommerce.transaction_id is not null
  group by order_key, session_key, user_pseudo_id, order_ts -- collapse duplicate purchase rows
)
select
  order_key, session_key, user_pseudo_id, date(order_ts) as date_key, order_ts,
  gross_revenue,
  coalesce(refund_value_in_usd, 0.0) as refund_value_in_usd,
  gross_revenue - coalesce(refund_value_in_usd, 0.0) as net_revenue,
  coalesce(shipping_value_in_usd, 0.0) as shipping_value_in_usd,
  coalesce(tax_value_in_usd, 0.0) as tax_value_in_usd,
  total_item_quantity, unique_items
from purch
```

### 11.6 SQL – headline revenue metrics

```
select
  count(distinct order_key) as transactions,
  sum(gross_revenue) as gross_revenue,
  sum(net_revenue) as net_revenue,
  safe_divide(sum(gross_revenue), count(distinct order_key)) as aov,
  safe_divide(sum(total_item_quantity), count(distinct order_key)) as items_per_transaction
from {{ ref('fct_orders') }}
-- revenue_per_session / revenue_per_user join fct_orders to fct_sessions / dim_users:
-- revenue_per_session = sum(gross_revenue) / count(distinct session_key from fct_sessions)
-- revenue_per_user = sum(gross_revenue) / count(distinct user_pseudo_id from dim_users)
```

These definitions are the single source consumed by `semantic-mcp.get_metric`; any deviation (a synonym, an un-deduped sum, mixing `purchase_revenue` with `_in_usd`) is rejected at the semantic layer and surfaced by the Critic, upholding the 100%-governed-SQL target.

## 12. Channel Attribution Definitions

Channel attribution is the single most error-prone area of GA4 analysis, and Helios treats it as a first-class governed concern. The Decompose and Diagnose agents lean heavily on `channel_group` to explain funnel movement, so a wrong attribution rule silently corrupts every downstream

finding. This section pins down the gotcha, the exact derivation, the attribution models, the channel grouping CASE logic, and the `dim_channels` dimension.

## 12.1 The traffic\_source gotcha

In the GA4 BigQuery export, the event-level `traffic_source` STRUCT<name, medium, source> is **USER-LEVEL FIRST-TOUCH attribution**, not the source of the session that produced the event. It is stamped from the user's very first acquisition and copied onto every subsequent event for that `user_pseudo_id` for the entire export. A user acquired via Organic Search in November who returns in January via an Email campaign will still carry `traffic_source.medium = 'organic'` on the January events. Using event-level `traffic_source` for session-level channel analysis therefore systematically over-credits acquisition channels and under-credits re-engagement channels, and it can manufacture Simpson's-paradox confounds (mix-shift that looks like rate-change) — exactly what the core algorithm is designed to detect. **Rule: never use event-level traffic\_source for session-scoped channel grouping.** It is permissible only as a documented fallback when session-scoped params are entirely NULL (rare), and as the basis for the genuinely user-level `dim_users.first_touch_channel`.

## 12.2 Session-scoped source / medium derivation

The session-correct source/medium lives in `event_params` on the session's own events (keys `source`, `medium`, `campaign`, `term`, `content`, `gclid`). These are populated chiefly on `session_start`, `first_visit`, and `page_view` events. The canonical rule is **first non-null value within the session ordered by event\_timestamp**, where session = (`user_pseudo_id`, `ga_session_id`). Newer GA4 exports also expose `collected_traffic_source` STRUCT<...>; the obfuscated sample predates it, so Helios derives from `event_params` and documents `collected_traffic_source` as the preferred source if/when present.

```
-- int_ga4_sessionized: one row per (user_pseudo_id, ga_session_id) with session source/medium
WITH ev AS (
  SELECT
    user_pseudo_id,
    (SELECT ep.value.int_value FROM UNNEST(event_params) ep WHERE ep.key='ga_session_id') AS ga_session_id,
    event_timestamp,
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key='source') AS p_source,
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key='medium') AS p_medium,
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key='campaign') AS p_campaign,
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key='gclid') AS p_gclid
  FROM `bigquery-public-data.ga4_obfuscated_sample_ecommerce.events_*`
)
SELECT
  user_pseudo_id, ga_session_id,
  -- first non-null param ordered by event_timestamp (IGNORE NULLS picks earliest populated)
  COALESCE(ARRAY_AGG(p_source IGNORE NULLS ORDER BY event_timestamp LIMIT 1)[OFFSET(0)], '(direct)') AS session_source,
  COALESCE(ARRAY_AGG(p_medium IGNORE NULLS ORDER BY event_timestamp LIMIT 1)[OFFSET(0)], '(none)') AS session_medium,
  ARRAY_AGG(p_campaign IGNORE NULLS ORDER BY event_timestamp LIMIT 1)[OFFSET(0)] AS session_campaign,
  LOGICAL_OR(p_gclid IS NOT NULL) AS has_gclid
FROM ev
WHERE ga_session_id IS NOT NULL
GROUP BY user_pseudo_id, ga_session_id
```

Normalization rules applied before grouping: lowercase `source` / `medium`; empty string → NULL; missing source → '(direct)'; missing medium with direct source → '(none)'. These mirror GA4's own canonicalization so reconciliation against `reconcile()` totals holds.

## 12.3 Attribution models

Helios computes three attribution models and exposes the model as an explicit choice so findings declare which they used:

- **First-touch** — credit the session/transaction to the channel of the user's first session ever (`ga_session_number = 1`). Stored on `dim_users.first_touch_channel`; equals the (correctly-scoped) channel of the first session, not the event-level `traffic_source`.
- **Last-touch** — credit to the channel of the converting session itself. This is the Helios **default** for session-grained funnel and revenue diagnosis, because the funnel question is "what brought this session that converted."
- **Last-non-direct** — credit to the most recent non-`(direct)` channel within a lookback window (GA4 default 90 days), ignoring direct sessions. Used for revenue-at-risk attribution where direct traffic is treated as un-attributable re-entry.

Unless a finding states otherwise, `channel_group` on `fact_sessions` / `fact_funnel` / `fact_orders` is **last-touch session-scoped**.

## 12.4 GA4-style default channel grouping — classification rules

Derived strictly from session-scoped `session_source` / `session_medium` (+ `has_gclid`, `session_campaign`). Rules are evaluated top-to-bottom; first match wins. This mirrors GA4's default channel definitions.

| channel_group  | Rule (on lowercased session source/medium)                                                                                                             |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Direct         | medium IN ('(none)', '(not set)') AND source IN ('(direct)', '')                                                                                       |
| Paid Search    | medium matches `^(cpc                                                                                                                                  |
| Paid Social    | medium matches `^(cpc                                                                                                                                  |
| Display        | medium matches `^(display                                                                                                                              |
| Organic Search | medium = 'organic' OR source matches a search engine with non-paid medium                                                                              |
| Organic Social | medium IN ('social', 'social-network', 'social-media', 'sm', 'social network', 'social media') OR source matches a social network with non-paid medium |
| Email          | medium IN ('email', 'e-mail', 'e_mail', 'newsletter') OR source matches `email                                                                         |
| Affiliates     | medium = 'affiliate' OR medium = 'affiliates'                                                                                                          |
| Referral       | medium IN ('referral', 'link') (and not classified above)                                                                                              |
| Other          | everything else / unclassifiable                                                                                                                       |

## 12.5 channel\_group CASE SQL

```
-- dim_channels resolver: applied in int_ga4_sessionized → propagated to facts
CASE
  -- Direct
  WHEN LOWER(session_medium) IN ('(none)', '(not set)', '') AND LOWER(session_source) IN ('(direct)', '')
  THEN 'Direct'
  -- Paid Search
  WHEN (REGEXP_CONTAINS(LOWER(session_medium), r'^(cpc|ppc|paid|paidsearch)$') OR has_gclid)
  AND REGEXP_CONTAINS(LOWER(session_source), r'google|bing|yahoo|duckduckgo|ecosia|baidu|yandex')
  THEN 'Paid Search'
  -- Paid Social
  WHEN REGEXP_CONTAINS(LOWER(session_medium), r'^(cpc|ppc|paid.*$)')
  AND REGEXP_CONTAINS(LOWER(session_source), r'facebook|instagram|fb|twitter|x\.com|tiktok|linkedin|pinterest|reddit|snapchat|youtube')
  THEN 'Paid Social'
  -- Display
  WHEN REGEXP_CONTAINS(LOWER(session_medium), r'^(display|banner|expandable|interstitial|cpm)$')
  THEN 'Display'
  -- Organic Search
  WHEN LOWER(session_medium) = 'organic'
  OR REGEXP_CONTAINS(LOWER(session_source), r'google|bing|yahoo|duckduckgo|ecosia|baidu|yandex')
  THEN 'Organic Search'
  -- Organic Social
  WHEN LOWER(session_medium) IN ('social', 'social-network', 'social-media', 'sm', 'social network', 'social media')
  OR REGEXP_CONTAINS(LOWER(session_source), r'facebook|instagram|fb|twitter|tiktok|linkedin|pinterest|reddit|snapchat|youtube')
  THEN 'Organic Social'
  -- Email
  WHEN LOWER(session_medium) IN ('email', 'e-mail', 'e_mail', 'newsletter')
  OR REGEXP_CONTAINS(LOWER(session_source), r'email|newsletter')
  THEN 'Email'
  -- Affiliates
  WHEN LOWER(session_medium) IN ('affiliate', 'affiliates')
  THEN 'Affiliates'
  -- Referral
  WHEN LOWER(session_medium) IN ('referral', 'link')
  THEN 'Referral'
  ELSE 'Other'
END AS channel_group
```

## 12.6 dim\_channels

`dim_channels` is the conformed dimension table that materializes the grouping logic so it is authored once and reused everywhere. Its grain is **one row per channel\_group** (~10 rows): the canonical channel groups enumerated once, each with its paid/organic flags and sort order. Facts carry their resolved `channel_group` and join to `dim_channels` on it, preventing ad-hoc CASE statements from drifting across models.

```
-- dims/dim_channels.sql (one row per channel_group; ~10 rows)
SELECT
  TO_HEX(MD5(channel_group)) AS channel_key,
  channel_group,
  CASE WHEN channel_group IN ('Paid Search', 'Paid Social', 'Display')
  THEN TRUE ELSE FALSE END AS is_paid,
```

```

CASE WHEN channel_group IN ('Organic Search','Organic Social','Direct','Referral')
THEN TRUE ELSE FALSE END AS is_organic,
channel_group_order
FROM UNNEST([
STRUCT('Direct' AS channel_group, 1 AS channel_group_order),
STRUCT('Organic Search' AS channel_group, 2 AS channel_group_order),
STRUCT('Paid Search' AS channel_group, 3 AS channel_group_order),
STRUCT('Paid Social' AS channel_group, 4 AS channel_group_order),
STRUCT('Organic Social' AS channel_group, 5 AS channel_group_order),
STRUCT('Display' AS channel_group, 6 AS channel_group_order),
STRUCT('Email' AS channel_group, 7 AS channel_group_order),
STRUCT('Affiliates' AS channel_group, 8 AS channel_group_order),
STRUCT('Referral' AS channel_group, 9 AS channel_group_order),
STRUCT('Other' AS channel_group, 10 AS channel_group_order)
])

```

The per-session `channel_group` is resolved upstream in `int_ga4__sessionized` via the dbt macro `channel_group_case()` so the rules in 12.5 exist in exactly one place. `fact_sessions` carries `channel_group` and joins to `dim_channels` on `channel_group` (resolving `channel_key`, `is_paid`, `is_organic`, `channel_group_order`); the Diagnose agent reads `channel_group` only via `semantic-mcp`, never by re-deriving it.

## 13. Metric Definitions

This is the complete, authoritative metric catalog. Every metric the FOUNDATION names appears here with description, math, SQL expression, grain, numerator/denominator, applicable dimensions, filters, units, and gotchas. All metrics are session-grained at base (one row per `(user_pseudo_id, ga_session_id)` in `fact_sessions` / `fact_funnel`) unless noted; revenue metrics derive from `fact_orders` (one row per `transaction_id`). Aggregation to any dimension is `SUM / COUNT` of the base, never an average of ratios – ratios are computed as `SUM(numerator)/SUM(denominator)` after grouping. This re-aggregation discipline is itself the defense against Simpson's paradox.

### 13.1 Catalog overview

| metric                    | group       | type  | numerator                                                        | denominator             | units         | grain   |
|---------------------------|-------------|-------|------------------------------------------------------------------|-------------------------|---------------|---------|
| sessions                  | volume      | count | distinct (user_pseudo_id, ga_session_id)                         | â€”                     | count         | session |
| users                     | volume      | count | distinct user_pseudo_id                                          | â€”                     | count         | user    |
| new_users                 | volume      | count | users with first_visit/session_number=1                          | â€”                     | count         | user    |
| returning_users           | volume      | count | users âˆ’ new_users                                              | â€”                     | count         | user    |
| engaged_sessions          | volume      | count | sessions with session_engaged='1' OR engagement_time_msec>=10000 | â€”                     | count         | session |
| engagement_rate           | engagement  | ratio | engaged_sessions                                                 | sessions                | rate<br>0â€”1 | session |
| view_item_sessions        | volume      | count | sessions with view_item                                          | â€”                     | count         | session |
| add_to_cart_sessions      | volume      | count | sessions with add_to_cart                                        | â€”                     | count         | session |
| begin_checkout_sessions   | volume      | count | sessions with begin_checkout                                     | â€”                     | count         | session |
| purchasing_sessions       | volume      | count | sessions with purchase                                           | â€”                     | count         | session |
| session_conversion_rate   | funnel-rate | ratio | purchasing_sessions                                              | sessions                | rate<br>0â€”1 | session |
| view_to_cart_rate         | funnel-rate | ratio | add_to_cart_sessions                                             | view_item_sessions      | rate<br>0â€”1 | session |
| cart_to_checkout_rate     | funnel-rate | ratio | begin_checkout_sessions                                          | add_to_cart_sessions    | rate<br>0â€”1 | session |
| checkout_to_purchase_rate | funnel-rate | ratio | purchasing_sessions                                              | begin_checkout_sessions | rate<br>0â€”1 | session |
| cart_abandonment_rate     | funnel-rate | ratio | add_to_cart_sessions âˆ’ purchasing_sessions                     | add_to_cart_sessions    | rate<br>0â€”1 | session |
| checkout_abandonment_rate | funnel-rate | ratio | begin_checkout_sessions âˆ’ purchasing_sessions                  | begin_checkout_sessions | rate<br>0â€”1 | session |

| metric                | group      | type  | numerator                                 | denominator  | units | grain   |
|-----------------------|------------|-------|-------------------------------------------|--------------|-------|---------|
| transactions          | revenue    | count | distinct transaction_id                   | â€”          | count | order   |
| revenue               | revenue    | sum   | sum purchase_revenue_in_usd               | â€”          | USD   | order   |
| gross_revenue         | revenue    | sum   | sum purchase_revenue_in_usd               | â€”          | USD   | order   |
| net_revenue           | revenue    | sum   | gross_revenue â”” sum refund_value_in_usd | â€”          | USD   | order   |
| aov                   | revenue    | ratio | revenue                                   | transactions | USD   | order   |
| items_per_transaction | revenue    | ratio | sum item quantity                         | transactions | items | order   |
| revenue_per_session   | efficiency | ratio | revenue                                   | sessions     | USD   | session |
| revenue_per_user      | efficiency | ratio | revenue                                   | users        | USD   | user    |

### 13.2 Volume metrics

```
-- sessions
COUNT(DISTINCT session_key) AS sessions
-- users
COUNT(DISTINCT user_pseudo_id) AS users
-- new_users: first_visit event OR ga_session_number = 1
COUNT(DISTINCT IF(is_new_user, user_pseudo_id, NULL)) AS new_users
-- returning_users
COUNT(DISTINCT user_pseudo_id) - COUNT(DISTINCT IF(is_new_user, user_pseudo_id, NULL)) AS returning_users
-- step-reached session counts (booleans precomputed in fct_funnel)
COUNTIF(reached_view_item) AS view_item_sessions,
COUNTIF(reached_add_to_cart) AS add_to_cart_sessions,
COUNTIF(reached_begin_checkout) AS begin_checkout_sessions,
COUNTIF(reached_add_shipping_info) AS add_shipping_info_sessions,
COUNTIF(reached_add_payment_info) AS add_payment_info_sessions,
COUNTIF(reached_purchase) AS purchasing_sessions
-- session_key = TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))
```

**Gotchas:** sessions MUST be distinct on the composite (user\_pseudo\_id, ga\_session\_id) â€” ga\_session\_id is unique only within a user, never globally. new\_users on the GA4 sample is best identified by the first\_visit event because user\_id is almost always NULL; ga\_session\_number = 1 is the fallback. Step counts are session-presence booleans (did the session ever fire the event), not event counts.

### 13.3 Funnel-rate metrics

```
-- session_conversion_rate
SAFE_DIVIDE(COUNTIF(reached_purchase), COUNT(DISTINCT session_key)) AS session_conversion_rate,
-- step-to-step rates (denominator = prior step, NOT total sessions)
SAFE_DIVIDE(COUNTIF(reached_add_to_cart), COUNTIF(reached_view_item)) AS view_to_cart_rate,
SAFE_DIVIDE(COUNTIF(reached_begin_checkout), COUNTIF(reached_add_to_cart)) AS cart_to_checkout_rate,
SAFE_DIVIDE(COUNTIF(reached_add_shipping_info), COUNTIF(reached_begin_checkout)) AS checkout_to_shipping_rate,
SAFE_DIVIDE(COUNTIF(reached_add_payment_info), COUNTIF(reached_add_shipping_info)) AS shipping_to_payment_rate,
SAFE_DIVIDE(COUNTIF(reached_purchase), COUNTIF(reached_add_payment_info)) AS payment_to_purchase_rate,
SAFE_DIVIDE(COUNTIF(reached_purchase), COUNTIF(reached_begin_checkout)) AS checkout_to_purchase_rate,
-- abandonment
SAFE_DIVIDE(COUNTIF(reached_add_to_cart) - COUNTIF(reached_purchase), COUNTIF(reached_add_to_cart)) AS cart_abandonment_rate,
SAFE_DIVIDE(COUNTIF(reached_begin_checkout) - COUNTIF(reached_purchase), COUNTIF(reached_begin_checkout)) AS checkout_abandonment_rate
```

**Gotchas:** funnel step-to-step rates use the **immediately prior step** as denominator, not total sessions. The canonical macro funnel ( session\_start â”” view\_item â”” add\_to\_cart â”” begin\_checkout â”” add\_shipping\_info â”” add\_payment\_info â”” purchase ) is monotonic only when steps are computed as “reached this step OR any later step”; Helios materializes reached\_\* booleans with this max-downstream rule so a session that purchases without a logged begin\_checkout still counts as having reached checkout. Always wrap divisions in SAFE\_DIVIDE to return NULL (not error) on zero denominators. When aggregating across dimensions, re-aggregate numerator and denominator separately â€” never average the per-segment rates.

### 13.4 Revenue metrics

```
-- transactions: distinct transaction_id from purchase events
COUNT(DISTINCT ecommerce.transaction_id) AS transactions,
-- revenue / gross_revenue
SUM(ecommerce.purchase_revenue_in_usd) AS revenue,
SUM(ecommerce.purchase_revenue_in_usd) AS gross_revenue,
```

```
-- net_revenue
SUM(ecommerce.purchase_revenue_in_usd) - SUM(COALESCE(ecommerce.refund_value_in_usd,0)) AS net_revenue,
-- aov
SAFE_DIVIDE(SUM(ecommerce.purchase_revenue_in_usd), COUNT(DISTINCT ecommerce.transaction_id)) AS aov,
-- items_per_transaction
SAFE_DIVIDE(SUM(ecommerce.total_item_quantity), COUNT(DISTINCT ecommerce.transaction_id)) AS items_per_transaction
```

**Gotchas:** revenue is taken once per purchase event from `ecommerce.purchase_revenue_in_usd`, NOT by summing `items[].item_revenue_in_usd` (which double-counts when both are present and excludes shipping/tax). `transaction_id` can repeat across shards if a purchase is re-logged – always `COUNT(DISTINCT)`. Use `*_in_usd` columns exclusively; the non-USD `purchase_revenue / price` are in the original currency and must never be mixed. `net_revenue` subtracts refunds, which in this dataset are sparse but must be coalesced to 0.

### 13.5 Efficiency metrics

```
-- revenue_per_session (RPS): revenue spread over ALL sessions, not just purchasing ones
SAFE_DIVIDE(SUM(revenue), COUNT(DISTINCT session_key)) AS revenue_per_session,
-- revenue_per_user (ARPU)
SAFE_DIVIDE(SUM(revenue), COUNT(DISTINCT user_pseudo_id)) AS revenue_per_user
```

**Gotchas:** RPS and ARPU denominators are **all** sessions/users in the window, not purchasers; this is intentional so the metric captures both conversion rate and AOV. Joining `fct_orders` to `fct_sessions` requires a left join from sessions so non-purchasing sessions remain in the denominator. RPS decomposes exactly as `session_conversion_rate` – `aov`, a relationship the Decompose agent exploits to attribute RPS movement to conversion vs basket-size.

### 13.6 Engagement metrics

```
-- engaged_sessions: GA4 engaged = session_engaged='1' OR engagement_time_msec ≥ 10000
COUNTIF(is_engaged_session) AS engaged_sessions,
-- engagement_rate
SAFE_DIVIDE(COUNTIF(is_engaged_session), COUNT(*)) AS engagement_rate
```

`is_engaged_session` is set in `int_ga4_sessionized` as `session_engaged = '1' OR engagement_time_msec ≥ 10000`, using the session-scoped `session_engaged` param. **Gotcha:** do not equate engagement with conversion; an engaged session need not purchase.

### 13.7 Cohort / retention metrics

These are computed by `stats-mcp` (`cohort_retention`, `rfm_segment`), but their grain and definition are governed here. Cohort = users grouped by week of `user_first_touch_timestamp`. `retention_rate(cohort, week_n)` = distinct users from the cohort with any session in week n divided by cohort size. RFM segments score Recency (days since last session), Frequency (distinct sessions), Monetary (sum `revenue`) into quintiles per `user_pseudo_id`. **Gotcha:** retention denominators are the original cohort size (fixed), never the surviving population, so retention is monotonically non-increasing.

## 14. Semantic Layer Design

The semantic layer is the heart of Helios's anti-hallucination guarantee. The PRINCIPLE is **grounding over generation**: the LLM never authors raw SQL or computes a statistic – it composes governed metric and dimension definitions through `semantic-mcp`, which is the **ONLY** path to SQL. Every column name, every join, every formula in this section comes from a registry of YAML definitions that an engineer owns, versions, and tests. If the LLM references a metric or dimension that is not in the registry, `build_query` raises before any SQL is generated – making "0 hallucinated columns/metrics (100% governed SQL)" a structural property, not a hope.

### 14.1 YAML schema for metrics and dimensions

Each metric and dimension is a YAML document with a fixed, validated schema. The fields:

```
# Field contract for a metric definition
name:      # snake_case canonical id (must match FOUNDATION exactly)
label:     # human display label
description: # one-line semantics, used in the exec brief glossary
type:      # one of: count | sum | ratio | derived
entity:    # grain entity: session | user | order | order_item
grain:     # physical base model this resolves against (e.g. fct_funnel)
agg:       # aggregation for additive metrics: count_distinct | sum | countif
sql:       # for count/sum: the additive SQL expression
numerator: # for ratio: name of numerator metric (must exist in registry)
```

```

denominator: # for ratio: name of denominator metric (must exist in registry)
expr:        # for derived: expression over other registered metrics
filters:     # list of governed filter predicates always applied
format:      # rendering: integer | percent_1dp | usd | decimal_2
dimensions:  # whitelist of dimension names this metric may be sliced by
owner:       # accountable engineer / team
version:     # semver of this definition

```

```

# Field contract for a dimension definition
name:      # snake_case canonical id (must match FOUNDATION dimension list)
label:     # display label
description: # semantics
type:      # categorical | temporal | boolean
entity:    # session | user | order | item
sql:       # column or expression resolving the dimension on its entity
format:    # display format
owner:
version:

```

## 14.2 Example metric YAML entries

```

metrics:
- name: sessions
  label: Sessions
  description: Distinct (user_pseudo_id, ga_session_id) pairs in the window.
  type: count
  entity: session
  grain: fct_funnel
  agg: count_distinct
  sql: "session_key" # TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))
  filters: []
  format: integer
  dimensions: [device_category, operating_system, browser, country, region,
               channel_group, source, medium, campaign, landing_page,
               is_new_user, day, week, session_number_bucket]
  owner: analytics-eng
  version: 1.2.0

- name: add_shipping_info_sessions
  label: Add Shipping Info Sessions
  description: Sessions that reached the add_shipping_info step (max-downstream).
  type: count
  entity: session
  grain: fct_funnel
  agg: countif
  sql: "reached_add_shipping_info"
  filters: []
  format: integer
  dimensions: [device_category, operating_system, browser, country, region,
               channel_group, source, medium, campaign, landing_page,
               is_new_user, day, week, session_number_bucket]
  owner: analytics-eng
  version: 1.0.0

- name: add_payment_info_sessions
  label: Add Payment Info Sessions
  description: Sessions that reached the add_payment_info step (max-downstream).
  type: count
  entity: session
  grain: fct_funnel
  agg: countif
  sql: "reached_add_payment_info"
  filters: []
  format: integer
  dimensions: [device_category, operating_system, browser, country, region,
               channel_group, source, medium, campaign, landing_page,
               is_new_user, day, week, session_number_bucket]
  owner: analytics-eng
  version: 1.0.0

- name: purchasing_sessions
  label: Purchasing Sessions
  description: Sessions that reached the purchase step.
  type: count
  entity: session
  grain: fct_funnel
  agg: countif

```



```

sql: "reached_purchase"
filters: []
format: integer
dimensions: [device_category, operating_system, browser, country, region,
              channel_group, source, medium, campaign, landing_page,
              is_new_user, day, week, session_number_bucket]
owner: analytics-eng
version: 1.1.0

- name: session_conversion_rate
  label: Session Conversion Rate
  description: Share of sessions that purchased (purchasing_sessions / sessions).
  type: ratio
  entity: session
  grain: fct_funnel
  numerator: purchasing_sessions
  denominator: sessions
  filters: []
  format: percent_1dp
  dimensions: [device_category, operating_system, browser, country, region,
                channel_group, source, medium, campaign, landing_page,
                is_new_user, day, week, session_number_bucket]
  owner: analytics-eng
  version: 2.0.0

- name: revenue_per_session
  label: Revenue per Session (RPS)
  description: Revenue spread across all sessions; equals session_conversion_rate * aov.
  type: derived
  entity: session
  grain: fct_funnel
  expr: "SAFE_DIVIDE({revenue}, {sessions})"
  filters: []
  format: usd
  dimensions: [device_category, channel_group, country, day, week, is_new_user]
  owner: analytics-eng
  version: 1.0.0

```

### 14.3 Example dimension YAML entries

```

dimensions:
- name: channel_group
  label: Channel Group
  description: GA4-style default channel grouping from session-scoped source/medium.
  type: categorical
  entity: session
  sql: "channel_group" # resolved via dim_channels join, never re-derived ad hoc
  format: string
  owner: analytics-eng
  version: 1.3.0

- name: device_category
  label: Device Category
  description: device.category of the session (desktop / mobile / tablet).
  type: categorical
  entity: session
  sql: "device_category"
  format: string
  owner: analytics-eng
  version: 1.0.0

```

### 14.4 The registry and how build\_query composes validated SQL

The metric and dimension YAML files compile at load time into an in-memory **registry**: two dictionaries keyed by `name`, with referential integrity checks (every `numerator` / `denominator` / `expr` token must resolve to a registered metric; every entry in a metric's `dimensions` whitelist must be a registered dimension on a compatible entity). `semantic-mcp.build_query(metric, dims, filters, window)` is a deterministic resolver – not an LLM – that composes SQL strictly from these definitions:

```

def build_query(metric: str, dims: list[str], filters: dict, window: str) → str:
    m = REGISTRY.metrics[metric] # KeyError → hard fail, no hallucination
    for d in dims:
        if d not in m["dimensions"]: # dimension not whitelisted for metric
            raise SemanticError(f"{d} not permitted for {metric}")
    dim_sql = [REGISTRY.dimensions[d]["sql"] + f" AS {d}" for d in dims]
    where = compile_window(window) + compile_filters(filters, REGISTRY) # only governed predicates

```

```

if m["type"] in ("count", "sum"):
    select = f"AGG[{m['agg']}](m['sql']) AS {metric}"
elif m["type"] == "ratio":
    num, den = REGISTRY.metrics[m["numerator"]], REGISTRY.metrics[m["denominator"]]
    select = (f"SAFE_DIVIDE(AGG[num['agg']])(num['sql']), "
            f"AGG[den['agg']](den['sql'])) AS {metric}")
elif m["type"] == "derived":
    select = expand_expr(m["expr"], REGISTRY) + f" AS {metric}"
grp = f"GROUP BY {'', ' '.join(str(i+1) for i in range(len(dims)))}" if dims else ""
return f"SELECT {'', ' '.join(dim_sql + [select])} FROM {GRAIN[m['grain']]} WHERE {where} {grp}"

```

**Why this prevents hallucinated columns:** the LLM only ever passes string names. It can never emit a column, table, or formula directly. If it invents `conversion_pct`, the registry lookup fails loudly and the Critic agent flags it; the model retries with a real metric. Physical column names live exclusively in `sql` fields owned by analytics engineers, so a GA4 schema change is fixed in one YAML file, not across prompts. Every generated query is then `dry_run`-checked for cost/schema and `reconcile`-checked against canonical totals before results are trusted â€” verify-then-trust.

## 14.5 Governance, ownership, versioning

Each definition carries `owner` and semver `version`. Definitions live in `models/semantic/*.yaml` under code review; CI (GitHub Actions: `dbt build` + tests + eval harness) compiles the registry, runs referential-integrity checks, and fails the build on any dangling reference or schema drift. A breaking change to a formula bumps the major version and is recorded in the Memory store so prior diagnoses remain interpretable against the definition that produced them. The Critic agent additionally checks that a finding's cited metric `version` matches the run's registry version.

## 14.6 Mapping to dbt semantic layer / MetricFlow

The registry maps 1:1 onto dbt's semantic layer. Each `entity / grain` becomes a MetricFlow **semantic model** over the corresponding fact (`fact_funnel`, `fact_orders`); additive metrics map to MetricFlow `measures` (`agg: sum|count_distinct`); `type: ratio` maps to a `ratio` metric (numerator/denominator); `type: derived` maps to a `derived` metric over other metrics; dimension `sql` maps to semantic-model `dimensions`. Helios ships a thin custom resolver (14.4) so it runs identically with or without a dbt Cloud Semantic Layer endpoint â€” the YAML is the single source of truth either way.

## 14.7 Worked example

`build_query('session_conversion_rate', ['device_category'], window='last_28d')` resolves: metric `session_conversion_rate` (ratio, grain `fact_funnel`, numerator `purchasing_sessions` [countif `reached_purchase`], denominator `sessions` [count\_distinct of the session key]); dimension `device_category` (whitelisted); window `last_28d` â†’ the trailing 28 `event_date` shards relative to the run date. Generated SQL:

```

SELECT
  device_category AS device_category,
  SAFE_DIVIDE(
    COUNTIF(reached_purchase),
    COUNT(DISTINCT session_key)
  ) AS session_conversion_rate
FROM 'helios.marts.fct_funnel'
WHERE event_date BETWEEN
  FORMAT_DATE('%Y%m%d', DATE_SUB(CURRENT_DATE(), INTERVAL 28 DAY))
  AND FORMAT_DATE('%Y%m%d', CURRENT_DATE())
GROUP BY 1

```

This SQL contains zero column names the LLM chose; every token traces to a versioned YAML definition, dry-run validated and reconciled before any finding built on it ships.

# 15. Analytics Engineering Architecture

Helios treats analytics as software. Every metric the Diagnose and Decompose agents consume is the output of a deterministic, version-controlled, tested transformation pipeline â€” never an ad-hoc query an LLM authored. This section specifies the layered architecture, the ELT flow, idempotency and incremental strategy, the testing pyramid, freshness SLAs, CI/CD, environments, code review, and lineage.

## 15.1 Layered architecture (sources -> staging -> intermediate -> marts -> semantic)

The pipeline is a strict five-layer DAG. Data flows in one direction; no layer reaches across or upward.

```

bigquery-public-data.ga4_obfuscated_sample_ecommerce
  events_YYYYMMDD (date-sharded, rows = events)

```

```

|
[SOURCES]    src_ga4 (declared in sources.yml; no SQL, contract only)
|
[STAGING]    stg_ga4__events      stg_ga4__event_params
              (1:1 with source, renamed/typed, light cleaning, views)
|
[INTERMEDIATE] int_ga4__sessionized int_ga4__funnel_steps
              (business logic: session keys, channel grouping, funnel flags)
|
[MARTS] core:  fct_sessions fct_funnel fct_daily_funnel
              dim_users dim_items dim_channels dim_date
              finance: fct_orders fct_order_items
              growth: (rollups feeding stats-mcp)
|
[SEMANTIC]    models/semantic (governed metric defs; the ONLY SQL path
              for the LLM, surfaced through semantic-mcp.get_metric /
              build_query)

```

- **Sources** ( `src_ga4` ) declare the upstream `events_*` shards as a contract. No transformation; they pin schema, freshness, and partition expectations so the build fails loudly if Google changes the export.
- **Staging** ( `stg_ga4__*` ) is 1:1 with the source, materialized as **views**. Responsibilities: rename to snake\_case, cast types, flatten nothing structural yet, and expose the de facto keys ( `user_pseudo_id` , `event_timestamp` ). `stg_ga4__event_params` unnests the `event_params` ARRAY into a long key/value table so downstream models never re-implement UNNEST.
- **Intermediate** ( `int_ga4__*` ) holds reusable business logic that more than one mart needs: sessionization (the ( `user_pseudo_id` , `ga_session_id` ) key, landing page, session-scoped source/medium, channel group) and funnel-step flags. Materialized as ephemeral or table depending on cost.
- **Marts** are the consumption layer, split into `core` , `finance` , `growth` . Facts are grain-explicit ( `fct_sessions` = one row per session; `fct_orders` = one row per `transaction_id` ). Dims are conformed.
- **Semantic** is the governance boundary. Every canonical metric ( `session_conversion_rate` , `revenue` , `aov` , `revenue_per_session` , ...) is defined exactly once here. `semantic-mcp` reads these definitions; the LLM composes them but never writes the SQL.

## 15.2 ELT flow

Extract-Load is free: the GA4 export already lives in BigQuery as `bigquery-public-data` . Helios is therefore an **ELT** system where the "T" is the entire dbt project. A run is:

1. **Scheduler trigger** (Cloud Scheduler / cron) kicks the Orchestrator on the autonomous cadence.
2. `dbt build --select state:modified+` (CI) or `dbt build` (prod refresh) compiles, runs, and tests the DAG against the prod dataset.
3. Marts and the semantic layer materialize.
4. `warehouse-mcp.reconcile(metric, grain)` is invoked to assert marts agree with canonical raw totals.
5. The agent pipeline (Monitor -> Decompose -> Diagnose -> Prescribe -> Narrator, with the Critic gating) runs against the semantic layer only.

## 15.3 Idempotency and incremental vs full-refresh

Every model is **idempotent**: re-running over the same `event_date` partitions yields byte-identical output. This is guaranteed by partition-by-date plus `insert_overwrite` .

- **Full refresh** is the default for the small public dataset (~3 months, Nov 2020–Jan 2021). `dbt build --full-refresh` rebuilds everything cheaply and is the safe reset.
- **Incremental** is the production pattern, partitioned by `event_date` ( `DATE` ). New shards (e.g. a freshly landed `events_YYYYMMDD` ) are processed with `incremental_strategy='insert_overwrite'` , which atomically replaces only the touched partitions. A late-arriving or corrected shard reprocesses cleanly because the whole partition is overwritten – no dedup logic, no drift.

```

{{ config(
  materialized='incremental',
  incremental_strategy='insert_overwrite',
  partition_by={ 'field': 'event_date', 'data_type': 'date', 'granularity': 'day' },
  cluster_by=[ 'device_category', 'channel_group' ]
) }}

```

The incremental predicate prunes the source to the lookback window:

```

{% if is_incremental() %}
  where event_date >= date_sub(dbt_max_partition, interval 3 day)
{% endif %}

```

A 3-day lookback absorbs GA4's late event landing without reprocessing the full history.

## 15.4 Testing strategy (the pyramid)

| Layer          | Test type                                                                                               | Tooling                                                                  | What it proves                             |
|----------------|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|--------------------------------------------|
| Schema         | <code>unique</code> , <code>not_null</code> , <code>accepted_values</code> , <code>relationships</code> | dbt generic tests                                                        | structural contract holds                  |
| Data           | freshness, row-count thresholds, value ranges                                                           | dbt + <code>dbt_utils</code>                                             | data is sane, not just well-typed          |
| Unit           | seeded input -> asserted output for a transform                                                         | dbt <code>unit_tests</code> ( <code>given</code> / <code>expect</code> ) | sessionization & channel logic are correct |
| Reconciliation | mart total == raw canonical total                                                                       | custom singular test + <code>warehouse-mcp.reconcile</code>              | marts didn't silently drift                |

- **Schema tests** live in `schema.yml` beside every model. Examples: `fct_orders.transaction_id` is `unique` + `not_null`; `dim_channels.channel_group` is `accepted_values` of the ten canonical channels; `fct_funnel.user_pseudo_id` has a `relationships` test to `dim_users`.
- **Data tests** assert `session_conversion_rate` between 0 and 1, `revenue`  $\geq 0$ , and source freshness (latest `event_date` within SLA).
- **Unit tests** seed a handful of synthetic events and assert the sessionizer produces the right `(user_pseudo_id, ga_session_id)` rows and the right `channel_group`. This is where Simpson's-paradox-relevant logic is locked down.
- **Reconciliation tests** are the verify-then-trust backbone: `sum(revenue)` from `fct_orders` must equal `sum(purchase_revenue_in_usd)` from raw within a tolerance of zero. Any drift fails the build and blocks the agent run, directly supporting the **100% governed SQL / 0 hallucinated metrics** target.

## 15.5 Data freshness and SLA

The public dataset is static, so freshness is enforced against the source's max shard. The production SLA: the latest available `events_YYYYMMDD` must be reflected in marts within the run window, and an autonomous diagnosis must complete in **under 5 minutes per run**. dbt source freshness:

```
freshness:
  warn_after: {count: 36, period: hour}
  error_after: {count: 48, period: hour}
  loaded_at_field: parse_date('%Y%m%d', _table_suffix_or_event_date)
```

A freshness error blocks the agent pipeline rather than letting it diagnose stale data.

## 15.6 CI/CD for analytics

GitHub Actions runs `dbt build` plus the eval harness on every PR:

```
on: pull_request
jobs:
  dbt-ci:
    steps:
      - run: dbt deps
      - run: dbt build --select state:modified+ --defer --state ./prod-manifest
      - run: dbt test --select state:modified+
      - run: python eval/run_benchmark.py # root-cause accuracy gate  $\geq 85\%$ 
```

- **Slim CI** via `state:modified+` + `--defer` builds only changed models and their children against a prod manifest, keeping byte cost under the run budget.
- The **eval harness** runs the labeled diagnosis benchmark; a PR that drops root-cause accuracy below 85% (vs the  $\leq 45\%$  naive baseline) fails CI. This makes correctness a merge gate, not an afterthought.

## 15.7 Environments

Two physical BigQuery datasets isolate work: `helios_dev` (per-developer, prefixed schemas) and `helios_prod`. The active dataset is chosen by dbt target in `profiles.yml`. CI builds into ephemeral dev schemas; merges to `main` trigger the prod build. The semantic layer is identical across environments so the agents behave consistently dev-to-prod.

## 15.8 SQL code review

No SQL merges without human review of the compiled output. Reviewers check: grain is declared and tested; no `SELECT *`; partition filter present; new metrics added to the semantic layer (not hardcoded in marts); reconciliation test added for any new fact. The Critic agent is the runtime analogue â€” it adversarially refutes findings (mix-shift confound, insufficient sample, seasonality, data quality) before they ship.

15.9 Lineage and exposures

dbt's DAG gives column- and model-level lineage from `src_ga4` through `models/semantic`. **Exposures** declare the downstream consumers â€” the seven agents and the Decision Brief â€” so `dbt ls --select +exposure:helios_decision_brief` answers "what feeds the brief?" and impact analysis (`dbt build --select +<changed_model>+exposure:*`) shows what a change touches before it ships.

16. dbt Project Structure

16.1 Project tree

```
helios/
├── dbt_project.yml
├── packages.yml
├── profiles.yml # dev/prod targets → helios_dev / helios_prod
├── models/
│   ├── staging/
│   │   ├── src_ga4.yml # source declaration for events_* shards
│   │   ├── stg_ga4_events.sql # 1:1 typed/renamed event rows
│   │   ├── stg_ga4_event_params.sql # unnested key/value param long table
│   │   └── stg_ga4_schema.yml # staging tests + docs
│   ├── intermediate/
│   │   ├── int_ga4_sessionized.sql # session key, landing_page, channel_group
│   │   ├── int_ga4_funnel_steps.sql # per-session funnel-step boolean flags
│   │   └── int_ga4_schema.yml
│   ├── marts/
│   │   ├── core/
│   │   │   ├── fct_sessions.sql # 1 row / session; engagement, funnel reach
│   │   │   ├── fct_funnel.sql # 1 row / session (PK session_key); boolean reached_* flags
│   │   │   ├── fct_daily_funnel.sql # daily grain funnel counts + rates
│   │   │   ├── dim_users.sql # 1 row / user_pseudo_id; first-touch attrs
│   │   │   ├── dim_items.sql # 1 row / item_id; category hierarchy
│   │   │   ├── dim_channels.sql # source/medium → channel_group map
│   │   │   └── dim_date.sql # date spine, week, day-of-week
│   │   ├── core__schema.yml
│   │   ├── finance/
│   │   │   ├── fct_orders.sql # 1 row / transaction_id; revenue, aov inputs
│   │   │   ├── fct_order_items.sql # 1 row / transaction_id / item
│   │   │   └── finance__schema.yml
│   │   ├── growth/
│   │   │   ├── fct_funnel_by_dim.sql # funnel rollup by canonical dimensions
│   │   │   ├── fct_cohorts.sql # weekly acquisition cohorts (retention input)
│   │   │   └── growth__schema.yml
│   │   └── semantic/
│   │       ├── semantic_layer.yml # governed metric definitions (canonical names)
│   │       └── metrics__schema.yml
│   ├── macros/
│   │   ├── get_event_param.sql # extract a typed value from event_params
│   │   ├── channel_group.sql # channel_group_case(): single source of truth (rules in 04-semantics 12.5)
│   │   ├── sessionize.sql # build (user_pseudo_id, ga_session_id) key
│   │   └── test_revenue_reconciles.sql # custom generic test
│   ├── seeds/
│   │   └── channel_group_mapping.csv # source/medium → channel_group seed
│   ├── tests/
│   │   └── assert_session_conversion_rate_bounds.sql # singular data test
│   ├── snapshots/
│   │   └── snap_dim_items.sql # SCD2 on item price/category
│   ├── analyses/
│   │   └── adhoc_mix_shift_explore.sql # non-materialized exploration
│   └── exposures/
│       └── exposures.yml # agents + Decision Brief consumers
```

16.2 Model file catalog

| Model          | Layer   | Grain | Purpose                            |
|----------------|---------|-------|------------------------------------|
| stg_ga4_events | staging | event | typed/renamed 1:1 view of events_* |

| Model                | Layer        | Grain                    | Purpose                                                                                                                        |
|----------------------|--------------|--------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| stg_ga4_event_params | staging      | event—param              | unnested event_params long table                                                                                               |
| int_ga4_sessionized  | intermediate | session                  | session key, landing_page, channel_group, is_new_user                                                                          |
| int_ga4_funnel_steps | intermediate | session                  | boolean reach flags per macro funnel step                                                                                      |
| fct_sessions         | core         | session                  | engagement, funnel reach, RPS inputs                                                                                           |
| fct_funnel           | core         | session (PK session_key) | one row per session with boolean reached_* step flags; step-to-step rates computed from those flags (or from fct_daily_funnel) |
| fct_daily_funnel     | core         | day                      | daily sessions, step sessions, rates                                                                                           |
| dim_users            | core         | user_pseudo_id           | first-touch attrs, new/returning                                                                                               |
| dim_items            | core         | item_id                  | item/category hierarchy                                                                                                        |
| dim_channels         | core         | channel_group            | conformed channel dim                                                                                                          |
| dim_date             | core         | day                      | date spine                                                                                                                     |
| fct_orders           | finance      | transaction_id           | revenue, gross_revenue, net_revenue, aov inputs                                                                                |
| fct_order_items      | finance      | transaction_id—item      | items_per_transaction, item revenue                                                                                            |
| fct_funnel_by_dim    | growth       | day—dimension            | decomposition input (mix vs rate)                                                                                              |
| fct_cohorts          | growth       | cohort_week—age          | retention input for stats-mcp.cohort_retention                                                                                 |

### 16.3 sources.yml ( src\_ga4 )

```
version: 2
sources:
  - name: src_ga4
    database: bigquery-public-data
    schema: ga4_obfuscated_sample_ecommerce
    loaded_at_field: parse_date('%Y%m%d', event_date)
    freshness:
      warn_after: {count: 36, period: hour}
      error_after: {count: 48, period: hour}
    tables:
      - name: events
        identifier: "events_*" # date-sharded wildcard
        description: "GA4 export; rows are events; events_YYYYMMDD shards"
        columns:
          - name: event_date
            tests: [not_null]
          - name: user_pseudo_id
            description: "de facto user key (user_id almost always null)"
```

### 16.4 schema.yml with tests

```
version: 2
models:
  - name: fct_orders
    description: "One row per transaction_id with revenue measures."
    columns:
      - name: transaction_id
        tests: [unique, not_null]
      - name: revenue
        tests:
          - not_null
          - dbt_utils.accepted_range: {min_value: 0}
      - name: channel_group
        tests:
          - relationships: {to: ref('dim_channels'), field: channel_group}
    tests:
      - revenue_reconciles: # custom generic test
        column_name: revenue
        tolerance: 0
  - name: dim_channels
```

```

columns:
  - name: channel_group
  tests:
    - not_null
    - accepted_values:
        values: ['Direct','Organic Search','Paid Search','Display',
                 'Paid Social','Organic Social','Email','Affiliates',
                 'Referral','Other']

```

## 16.5 Custom generic test ( test\_revenue\_reconciles.sql )

```

{% test revenue_reconciles(model, column_name, tolerance=0) %}
with mart as (select sum({{ column_name }}) as v from {{ model }}),
raw as (
  select sum(ecommerce.purchase_revenue_in_usd) as v
  from {{ source('src_ga4','events') }}
  where event_name = 'purchase'
)
select mart.v as mart_v, raw.v as raw_v
from mart cross join raw
where abs(coalesce(mart.v,0) - coalesce(raw.v,0)) > {{ tolerance }}
{% endtest %}

```

## 16.6 Key macros

```

-- get_event_param.sql : typed extraction from the event_params ARRAY
{% macro get_event_param(key, type='string') %}
( select ep.value[{{ type }}]_value
  from unnest(event_params) ep
  where ep.key = '{{ key }}' limit 1)
{% endmacro %}

-- channel_group.sql : SINGLE SOURCE OF TRUTH for channel grouping.
-- This macro is the only place the classification logic lives; the rules it
-- encodes are documented authoritatively in 04-semantics section 12.5.
-- Body is an exact copy of the channel_group_case CASE SQL from 12.5
-- (10 groups only, has_gclid-based Paid detection, top-to-bottom precedence,
-- no "Paid Other" branch).
{% macro channel_group_case() %}
CASE
  -- Direct
  WHEN LOWER(session_medium) IN ('(none)','(not set)','') AND LOWER(session_source) IN ('(direct)','')
    THEN 'Direct'
  -- Paid Search
  WHEN (REGEXP_CONTAINS(LOWER(session_medium), r'^(cpc|ppc|paid|paidsearch)$') OR has_gclid)
    AND REGEXP_CONTAINS(LOWER(session_source), r'google|bing|yahoo|duckduckgo|ecosia|baidu|yandex')
    THEN 'Paid Search'
  -- Paid Social
  WHEN REGEXP_CONTAINS(LOWER(session_medium), r'^(cpc|ppc|paid.*$)')
    AND REGEXP_CONTAINS(LOWER(session_source), r'facebook|instagram|fb|twitter|x.com|tiktok|linkedin|pinterest|reddit|snapchat|youtube')
    THEN 'Paid Social'
  -- Display
  WHEN REGEXP_CONTAINS(LOWER(session_medium), r'^(display|banner|expandable|interstitial|cpm)$')
    THEN 'Display'
  -- Organic Search
  WHEN LOWER(session_medium) = 'organic'
    OR REGEXP_CONTAINS(LOWER(session_source), r'google|bing|yahoo|duckduckgo|ecosia|baidu|yandex')
    THEN 'Organic Search'
  -- Organic Social
  WHEN LOWER(session_medium) IN ('social','social-network','social-media','sm','social network','social media')
    OR REGEXP_CONTAINS(LOWER(session_source), r'facebook|instagram|fb|twitter|tiktok|linkedin|pinterest|reddit|snapchat|youtube')
    THEN 'Organic Social'
  -- Email
  WHEN LOWER(session_medium) IN ('email','e-mail','e_mail','newsletter')
    OR REGEXP_CONTAINS(LOWER(session_source), r'email|newsletter')
    THEN 'Email'
  -- Affiliates
  WHEN LOWER(session_medium) IN ('affiliate','affiliates')
    THEN 'Affiliates'
  -- Referral
  WHEN LOWER(session_medium) IN ('referral','link')
    THEN 'Referral'
  ELSE 'Other'
END
{% endmacro %}

```

```
-- sessionize.sql : build the session key (honors the traffic_source gotcha)
{% macro sessionize() %}
concat(user_pseudo_id, '-',
      cast({{ get_event_param('ga_session_id','int') }} as string))
{% endmacro %}
```

## 16.7 dbt\_project.yml

```
name: helios
version: '1.0.0'
profile: helios
require-dbt-version: "≥1.7.0"
models:
  helios:
    staging: {+materialized: view}
    intermediate: {+materialized: ephemeral}
    marts:
      core:
        +materialized: incremental
        +incremental_strategy: insert_overwrite
        +partition_by: {field: event_date, data_type: date, granularity: day}
        +cluster_by: {'device_category','channel_group'}
        finance: {+materialized: table}
        growth: {+materialized: table}
        semantic: {+materialized: view}
    seeds:
      helios: {channel_group_mapping: {+column_types: {medium: string}}}
```

## 16.8 packages.yml

```
packages:
- package: dbt-labs/dbt_utils
  version: {>=1.1.0, <2.0.0}
```

## 16.9 Sample staging model ( stg\_ga4\_\_events.sql )

```
{{ config(materialized='view') }}
select
  parse_date('%Y%m%d', event_date) as event_date,
  event_timestamp,
  event_name,
  user_pseudo_id,
  {{ sessionize() }} as session_key,
  {{ get_event_param('ga_session_id','int') }} as ga_session_id,
  {{ get_event_param('page_location') }} as page_location,
  {{ get_event_param('source') }} as session_source,
  {{ get_event_param('medium') }} as session_medium,
  device.category as device_category,
  device.web_info.browser as browser,
  geo.country as country,
  ecommerce.transaction_id,
  ecommerce.purchase_revenue_in_usd
from {{ source('src_ga4','events') }}
{% if is_incremental() %}
where parse_date('%Y%m%d', event_date)
      ≥ date_sub(_dbt_max_partition, interval 3 day)
{% endif %}
```

## 16.10 Sample mart model ( fct\_daily\_funnel.sql )

```
{{ config(materialized='incremental', incremental_strategy='insert_overwrite',
  partition_by={field:'event_date',data_type:'date',granularity:'day'}) }}
select
  event_date,
  count(distinct session_key) as sessions,
  count(distinct if(reached_view_item, session_key, null)) as view_item_sessions,
  count(distinct if(reached_add_to_cart, session_key, null)) as add_to_cart_sessions,
  count(distinct if(reached_begin_checkout, session_key, null)) as begin_checkout_sessions,
  count(distinct if(reached_add_shipping_info, session_key, null)) as add_shipping_info_sessions,
  count(distinct if(reached_add_payment_info, session_key, null)) as add_payment_info_sessions,
  count(distinct if(reached_purchase, session_key, null)) as purchasing_sessions,
  safe_divide( count(distinct if(reached_purchase, session_key, null)),
```



```
count(distinct session_key)) as session_conversion_rate
from {{ ref('int_ga4_funnel_steps') }}
group by event_date
```

## 17. BigQuery Architecture

### 17.1 Dataset layout

| Dataset                                              | Contents                                      | Materialization      | Access                    |
|------------------------------------------------------|-----------------------------------------------|----------------------|---------------------------|
| bigquery-public-data.ga4_obfuscated_sample_ecommerce | raw events_* shards                           | external (read-only) | roles/bigquery.dataViewer |
| helios_staging                                       | stg_ga4_*                                     | views                | service account RW        |
| helios_marts                                         | fct_*, dim_* (core/finance/growth)            | incremental tables   | service account RW        |
| helios_semantic                                      | governed metric views                         | views                | semantic-mcp RO           |
| helios_eval                                          | labeled benchmark, snapshots of agent outputs | tables               | eval harness RW           |

Separating `semantic` from `marts` enforces the grounding principle: `semantic-mcp` is granted read on `helios_semantic` only, so the LLM physically cannot bypass governance to hit raw facts.

### 17.2 Partitioning and clustering

- **Partition by `event_date` ( DATE )**. Every fact and staging model is day-partitioned. The diagnosis pipeline always scopes a window (t0->t1 for `decompose_change`, a series for `detect_anomaly`), so date partitioning gives near-perfect pruning: a 14-day diagnosis scans 14 of ~90 partitions.
- **Cluster by `device_category`, `channel_group`**. These are the two highest-cardinality, most-queried canonical dimensions (the mix-shift decomposition almost always slices by them). Clustering co-locates rows so block-level filtering drops bytes scanned again after partition pruning.

Rationale: partition pruning is coarse (whole days); clustering is the fine-grained second filter on the dimensions the Decompose agent uses to separate **mix effect** from **rate effect**.

### 17.3 Materialization choices

| Model class          | Materialization                                      | Why                                               |
|----------------------|------------------------------------------------------|---------------------------------------------------|
| staging              | <b>view</b>                                          | thin rename/cast; no storage cost; always fresh   |
| intermediate         | <b>ephemeral</b>                                     | inlined as CTEs; no object proliferation          |
| core facts           | <b>incremental</b> ( <code>insert_overwrite</code> ) | day-grain, idempotent partition replacement       |
| finance/growth facts | <b>table</b>                                         | small, fully rebuilt cheaply                      |
| semantic             | <b>view</b>                                          | governance layer; must reflect marts instantly    |
| daily rollups        | <b>materialized view</b> (optional)                  | auto-maintained pre-aggregates for hot dashboards |

`insert_overwrite` is chosen over `merge` because partition-atomic overwrite is cheaper and inherently idempotent for the date-sharded GA4 model â€” no surrogate-key dedup needed.

### 17.4 Cost controls

The system runs autonomously and must stay **under a fixed BigQuery byte budget per run**. Controls, layered:

1. **Partition pruning** â€” every query filters `event_date` BETWEEN the diagnosis window. `_dbt_max_partition` drives incremental lookback.
2. **require\_partition\_filter = true** on all partitioned marts, so any query missing a date predicate errors instead of full-scanning.
3. **maximum\_bytes\_billed** set on the connection (e.g. per-query cap), so a runaway query is killed by BigQuery, not discovered on the bill.
4. **Dry-run budgets** â€” `warehouse-mcp.dry_run(sql)` returns estimated bytes/cost before any execution; a per-query cap and a per-run cumulative cap are enforced (see 17.8).

5. **Shard pruning** â€” when reading raw `events_*`, filter `_TABLE_SUFFIX BETWEEN '20210101' AND '20210114'` so only relevant daily tables are read.
6. **No `SELECT *`** â€” column projection is mandatory (GA4 rows are wide with nested ARRAYS; `SELECT *` materializes megabytes of unused structs).

## 17.5 On-demand vs slot pricing

Helios defaults to **on-demand** (pay per byte scanned), which aligns with the byte-budget guardrail and the bursty, scheduled-run workload â€” there are no idle slots to amortize. If run frequency rises to continuous monitoring, a small **flat-rate / autoscaling slot reservation** caps spend and removes per-byte variance; the dry-run guardrail still governs bytes for query hygiene regardless of pricing model. The decision pivots on duty cycle: on-demand below it, reserved slots above the breakeven where per-byte cost exceeds slot rent.

## 17.6 IAM, service accounts, least privilege

- **sa-helios-runner** â€” runs dbt + agents in prod. `roles/bigquery.dataEditor` on `helios_*` datasets; `roles/bigquery.dataViewer` on the public source; `roles/bigquery.jobUser` to run queries. No project-level admin.
- **sa-helios-semantic** â€” backs `semantic-mcp`. **Read-only** on `helios_semantic` only. This is the technical enforcement of "the LLM never authors raw SQL": the credential it runs under cannot read raw or even mart tables directly.
- **sa-helios-ci** â€” GitHub Actions. RW on ephemeral CI schemas, RO on prod manifest. Scoped via Workload Identity Federation (no long-lived keys).
- Developers get RW only on their personal `helios_dev` schema.

## 17.7 GA4 query-optimization patterns

```
-- Prune shards + project only needed columns; never SELECT *
select event_timestamp, user_pseudo_id, event_name,
       (select value.int_value from unnest(event_params)
        where key = 'ga_session_id') as ga_session_id
from `bigquery-public-data.ga4_obfuscated_sample_ecommerce.events_*`
where _TABLE_SUFFIX between '20210101' and '20210114' -- shard prune
and event_name in ('session_start','purchase') -- predicate pushdown
```

- Unnest `event_params` once in staging (`stg_ga4__event_params`); never repeat correlated UNNEST subqueries downstream.
- Filter `_TABLE_SUFFIX` on the raw wildcard; filter `event_date` on partitioned marts.
- Honor the **traffic source gotcha**: prefer session-scoped `event_params.source/medium` for `channel_group`; fall back to user first-touch `traffic_source` only when session scope is null.

## 17.8 How the dry\_run guardrail enforces the byte budget

`warehouse-mcp.dry_run(sql)` performs a BigQuery dry run (`use_query_cache=False`, `dry_run=True`) and returns `total_bytes_processed` without executing. The guardrail wraps every execution path:

```
def guarded_run(sql, per_query_cap, run_state):
    est = warehouse_mcp.dry_run(sql) # bytes, no execution, no cost
    if est.bytes > per_query_cap:
        raise BudgetError("query exceeds per-query cap")
    if run_state.bytes_used + est.bytes > run_state.run_cap:
        raise BudgetError("run byte budget exhausted")
    rows = warehouse_mcp.run_query(sql) # only now does it cost
    run_state.bytes_used += est.bytes
    return rows
```

Every SQL the agents execute â€” composed exclusively through `semantic-mcp.build_query` â€” passes the dry run first. This keeps **query cost per run under the fixed budget**, contributes to **time-to-diagnosis under 5 minutes**, and, combined with reconciliation and the Critic, upholds **0 hallucinated columns/metrics and 100% governed SQL**.

# 18. MCP Architecture

## 18.1 What MCP Is and Why Helios Uses It

The **Model Context Protocol (MCP)** is an open client-server protocol that lets an LLM-driven agent invoke external capabilities ("tools"), read external state ("resources"), and reuse parameterized instruction templates ("prompts") through a single, schema-typed JSON-RPC 2.0 interface. An

MCP **server** advertises a manifest of tools â€” each with a JSON Schema for inputs â€” and the MCP **client** (here, the Claude Agent SDK runtime) marshals the model's structured tool calls to the server and returns typed results to the model's context window.

Helios uses MCP because the entire product thesis rests on **grounding over generation**: the LLM must NEVER author raw SQL and must NEVER compute a statistic in free text. Instead, every database access and every numerical operation is forced through a narrow, governed, deterministic tool surface. MCP is the mechanism that makes that boundary *physically enforceable* rather than merely a prompt-time suggestion. The model literally has no tool with which to run arbitrary SQL; it can only compose governed metrics. This converts three abstract principles into wiring:

- **Grounding (anti-hallucination):** `semantic-mcp` is the ONLY path to SQL. The model selects canonical metric/dimension names; the server emits validated SQL. A hallucinated column ( `event_params.foo` ) cannot survive `build_query` because the metric registry does not reference it.
- **Determinism where it matters:** `stats-mcp` is the ONLY path to math. Mix-shift decomposition, significance tests, and forecasts run in audited `scipy/statsmodels` code, byte-for-byte reproducible, never re-derived by the model.
- **Least privilege:** each server holds exactly the credentials and scope it needs. Only `warehouse-mcp` has a BigQuery client. `semantic-mcp` emits SQL *strings* but cannot execute them. `report-mcp` writes briefs but cannot query the warehouse. A compromised or buggy server has a blast radius bounded by its tool catalog.

18.2 Tool-Boundary Rationale

The five-server split is deliberate. SQL *generation* ( `semantic-mcp` ) is separated from SQL *execution* ( `warehouse-mcp` ) so that a query is validated and reconciled against canonical totals before a single byte is scanned, and so the model cannot smuggle hand-written SQL into the executor. Math ( `stats-mcp` ) is separated from data fetch so statistical methods are version-pinned and unit-tested independent of warehouse state. Experiment design ( `experiment-mcp` ) and narration ( `report-mcp` ) are isolated because they have zero data-access needs â€” they operate on findings already produced. The guardrail chain is: **dry\_run before every run\_query** (cost/byte budget enforcement), **build\_query before every dry\_run** (no ungoverned SQL reaches the executor), and **stats-mcp before every quantitative claim** (no model-computed numbers).

18.3 Transport, Authentication, Configuration, Statelessness

| Concern        | Decision                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Transport      | Local servers ( <code>semantic</code> , <code>stats</code> , <code>experiment</code> , <code>report</code> ) run over <b>stdio</b> (subprocess, JSON-RPC framed on stdin/stdout) â€” lowest latency, no network surface. <code>warehouse-mcp</code> runs over <b>HTTP (streamable)</b> so it can live in a network boundary with the BigQuery service account and be shared across runs.                                     |
| Authentication | <code>warehouse-mcp</code> : GCP service-account JSON via Application Default Credentials ( <code>GOOGLE_APPLICATION_CREDENTIALS</code> ), scoped to <code>roles/bigquery.dataViewer</code> + <code>roles/bigquery.jobUser</code> on the project, read-only on <code>bigquery-public-data</code> . HTTP endpoint guarded by a bearer token ( <code>HELIOS_WH_TOKEN</code> ). stdio servers inherit no warehouse credentials. |
| Configuration  | Declarative <code>mcp_servers.yaml</code> ( see 18.10 ). Each server takes a config block: byte budget, dataset id, metric-registry path, RNG seed.                                                                                                                                                                                                                                                                          |
| Statelessness  | All tools are <b>stateless request/response</b> ; no session affinity. The lone exception is <code>report-mcp</code> 's memory store ( <code>save_diagnosis</code> / <code>recall_prior</code> ), which is an explicit, durable side effect backed by a database â€” not in-process session state. Statelessness makes runs idempotent and horizontally scalable.                                                            |
| Registration   | The Agent SDK loads <code>mcp_servers.yaml</code> , spawns/connects each server, fetches its manifest, and exposes the union of tools to agents â€” filtered per-agent by an allow-list so the Narrator cannot call <code>run_query</code> ( see 18.9 ).                                                                                                                                                                     |

18.4 warehouse-mcp â€” Governed Execution (enforces verify-then-trust)

Purpose: the sole holder of a BigQuery client. Executes only SQL that already passed `semantic-mcp` , enforces the byte budget via mandatory dry-run, and reconciles metrics against canonical totals.

| Tool                        | Signature (typed inputs)                           | Output schema                                                   | Error modes                                            | Side effects       |
|-----------------------------|----------------------------------------------------|-----------------------------------------------------------------|--------------------------------------------------------|--------------------|
| <code>list_tables</code>    | (dataset: str = "ga4_obfuscated_sample_ecommerce") | {tables: [{name, row_count, size_bytes, shard_min, shard_max}]} | <code>DatasetNotFound</code> , <code>AuthError</code>  | none (read)        |
| <code>describe_table</code> | (table: str)                                       | {name, schema: [{name, type, mode}], num_rows}                  | <code>TableNotFound</code>                             | none               |
| <code>dry_run</code>        | (sql: str)                                         | {valid: bool, total_bytes_processed: int,                       | <code>SqlSyntaxError</code> , <code>SchemaError</code> | none (dry-run job) |

| Tool                   | Signature (typed inputs)                       | Output schema                                                                             | Error modes                                                                                     | Side effects                |
|------------------------|------------------------------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|-----------------------------|
|                        |                                                | <code>estimated_cost_usd: float,</code><br><code>referenced_tables: [str]}</code>         |                                                                                                 |                             |
| <code>run_query</code> | <code>(sql: str, max_bytes_billed: int)</code> | <code>{rows: [obj], row_count,</code><br><code>bytes_processed, job_id}</code>            | <code>ByteBudgetExceeded</code> ,<br><code>QueryTimeout</code> ,<br><code>NotDryRunFirst</code> | BigQuery<br>job (read-only) |
| <code>reconcile</code> | <code>(metric: str, grain: str)</code>         | <code>{metric, grain, canonical_total:</code><br><code>float, source: "warehouse"}</code> | <code>UnknownMetric</code>                                                                      | none                        |

Guardrail: `run_query` rejects any `sql` whose normalized hash was not seen by a prior `dry_run` in the same run, and hard-caps `max_bytes_billed` at the configured budget ( `ByteBudgetExceeded` otherwise), keeping query cost per run under the fixed BigQuery byte budget.

```
// dry_run call
{"tool": "dry_run", "arguments": {"sql": "SELECT COUNT(DISTINCT CONCAT(user_pseudo_id, CAST((SELECT value.int_value FROM UNNEST(event_params) WHERE key='ga_session_id') AS STRING))) AS sessions FROM `bigquery-public-data.ga4_obfuscated_sample_ecommerce.events_*` WHERE _TABLE_SUFFIX BETWEEN '20210101' AND '20210131'"}, "window": {"start": "20210101", "end": "20210131"}}

// response
{"valid": true, "total_bytes_processed": 248901376, "estimated_cost_usd": 0.00124, "referenced_tables": ["events_2021*"]}
```

18.5 semantic-mcp – The ONLY Path to SQL (enforces grounding)

Purpose: the anti-hallucination layer. Holds the governed metric registry (canonical SQL definitions for `sessions`, `session_conversion_rate`, `revenue`, `aov`, etc.) and the dimension catalog. Composes validated SQL from canonical names only; refuses unknown names.

| Tool                         | Signature                                                                                                                          | Output schema                                                                     | Error modes                                                                                                             | Side effects |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|--------------|
| <code>get_metric</code>      | <code>(name: str)</code>                                                                                                           | <code>{name, sql_template, grain,</code><br><code>type, depends_on: [str]}</code> | <code>UnknownMetric</code>                                                                                              | none         |
| <code>list_dimensions</code> | <code>()</code>                                                                                                                    | <code>{dimensions: [{name,</code><br><code>sql_expr, type}]}</code>               | none                                                                                                                    | none         |
| <code>build_query</code>     | <code>(metric: str   [str], dims: [str],</code><br><code>filters: [{dim, op, value}], window:</code><br><code>{start, end})</code> | <code>{sql: str, governed: true,</code><br><code>metrics, dims}</code>            | <code>UnknownMetric</code> , <code>UnknownDimension</code> ,<br><code>InvalidFilter</code> , <code>InvalidWindow</code> | none         |

`build_query` is the chokepoint: it interpolates only registry-defined templates and dimension expressions, so emitted SQL can reference only real GA4 columns. Output feeds directly into `warehouse-mcp.dry_run`.

```
// build_query call
{"tool": "build_query", "arguments": {"metric": ["sessions", "purchasing_sessions", "session_conversion_rate"], "dims": ["device_category"], "filters": [{"dim": "device_category", "op": "eq", "value": "mobile"}], "window": {"start": "20210101", "end": "20210131"}}, "window": {"start": "20210101", "end": "20210131"}}

// response
{"governed": true, "metrics": ["sessions", "purchasing_sessions", "session_conversion_rate"], "dims": ["device_category"], "sql": "WITH s AS (SELECT device_category AS device_category, TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST((SELECT value.int_value FROM UNNEST(event_params) WHERE key='ga_session_id') AS STRING)))) AS session_key, MAX(IF(event_name='purchase',1,0)) AS purchased FROM `bigquery-public-data.ga4_obfuscated_sample_ecommerce.events_*` WHERE _TABLE_SUFFIX BETWEEN '20210101' AND '20210131' GROUP BY 1,2) SELECT device_category, COUNT(DISTINCT session_key) AS sessions, SUM(purchased) AS purchasing_sessions, SAFE_DIVIDE(SUM(purchased),COUNT(DISTINCT session_key)) AS session_conversion_rate FROM s GROUP BY 1"}


```

18.6 stats-mcp – The ONLY Path to Math (enforces determinism)

Purpose: every statistic. Implements the core mix-shift vs rate-change decomposition, anomaly detection, significance tests, and forecasting in deterministic, seeded code. The model passes data arrays in and gets numbers out; it never computes.

| Tool                          | Signature                                                                                                  | Output schema                                                                                                                             | Error modes                                                     | Side effects |
|-------------------------------|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|--------------|
| <code>detect_anomaly</code>   | <code>(series: [{t, value}], method:</code><br><code>"stl"   "ewma"   "iqr")</code>                        | <code>{anomalies: [{t, value, score,</code><br><code>direction}]}</code>                                                                  | <code>InsufficientData</code>                                   | none         |
| <code>decompose_change</code> | <code>(metric: str, dim: str, t0:</code><br><code>[{seg, w, r}], t1: [{seg, w,</code><br><code>r}])</code> | <code>{delta_R, mix_effect, rate_effect,</code><br><code>interaction, by_segment: [{seg, mix, rate,</code><br><code>interaction}]}</code> | <code>SegmentMismatch</code> ,<br><code>InsufficientData</code> | none         |

| Tool              | Signature                                           | Output schema                                              | Error modes      | Side effects |
|-------------------|-----------------------------------------------------|------------------------------------------------------------|------------------|--------------|
| significance_test | (a: {n, x}, b: {n, x}, kind: "proportion"   "mean") | {p_value, effect_size, ci_low, ci_high, significant: bool} | ZeroSample       | none         |
| forecast          | (series: [{t, value}], horizon: int)                | {forecast: [{t, yhat, lo, hi}], model}                     | InsufficientData | none         |
| cohort_retention  | (events, cohort_grain, periods)                     | {matrix: [[float]]}                                        | InsufficientData | none         |
| rfm_segment       | (users: [{id, recency, frequency, monetary}])       | {segments: [{id, r, f, m, label}]}                         | none             | none         |

decompose\_change implements exactly the FOUNDATION formula:  $\text{mix\_effect} = \hat{I} \hat{E} \hat{I} w_i \cdot r_i(t_0)$ ,  $\text{rate\_effect} = \hat{I} \hat{E} w_i(t_0) \hat{A} \cdot \hat{I} r_i$ ,  $\text{interaction} = \hat{I} \hat{E} \hat{I} w_i \hat{A} \cdot \hat{I} r_i$ , separating "traffic composition changed" from "behavior changed" and dissolving Simpson's paradox.

```
// decompose_change call
"tool" "decompose_change" "arguments" {"metric" "session_conversion_rate" "dim" "device_category"
"t0" [{"seg" "desktop" "w" 0.40 "r" 0.030}, {"seg" "mobile" "w" 0.60 "r" 0.012}]
"t1" [{"seg" "desktop" "w" 0.30 "r" 0.030}, {"seg" "mobile" "w" 0.70 "r" 0.012}]}
// response
{"delta_R" -0.0018 "mix_effect" -0.0018 "rate_effect" 0.0 "interaction" 0.0
"by_segment" [{"seg" "desktop" "mix" -0.0030 "rate" 0.0 "interaction" 0.0},
{"seg" "mobile" "mix" 0.0012 "rate" 0.0 "interaction" 0.0}]}
```

This shows a pure **mix-shift**: overall conversion fell entirely because traffic moved toward low-converting mobile, with zero behavior change  $\hat{A}$  – a finding the Critic cannot refute as a rate problem.

18.7 experiment-mcp – Statistically-Defensible Backlog

Purpose: turns diagnoses into a prioritized, powered experiment backlog. No data access; consumes baselines from prior tools.

| Tool              | Signature                                                              | Output schema                                                                              | Error modes   | Side effects |
|-------------------|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|---------------|--------------|
| power_analysis    | (baseline: float, mde: float, alpha: float = 0.05, power: float = 0.8) | {n_per_arm: int, total_n: int}                                                             | InvalidRate   | none         |
| runtime_estimate  | (n: int, traffic: float)                                               | {days: float, weeks: float}                                                                | ZeroTraffic   | none         |
| design_experiment | (hypothesis: str, metric: str)                                         | {test_card: {hypothesis, primary_metric, mde, n_per_arm, runtime_days, guardrail_metrics}} | UnknownMetric | none         |

```
"tool" "power_analysis" "arguments" {"baseline" 0.024 "mde" 0.10 "alpha" 0.05 "power" 0.8}
// response
{"n_per_arm" 58420 "total_n" 116840}
```

18.8 report-mcp – Narration and Memory

Purpose: renders the executive Decision Brief and persists prior diagnoses (the only stateful server). Cannot query the warehouse.

| Tool           | Signature                        | Output schema                              | Error modes       | Side effects        |
|----------------|----------------------------------|--------------------------------------------|-------------------|---------------------|
| render_brief   | (findings: [obj])                | {brief_md: str, brief_html: str}           | EmptyFindings     | none                |
| save_diagnosis | (diagnosis: obj)                 | {id: str, saved: true}                     | ValidationError   | writes memory store |
| recall_prior   | (metric: str, segment: str)      | {prior: [{id, t, summary, action_status}]} | none              | reads memory store  |
| export         | (format: "pdf"   "slack"   "md") | {uri: str}                                 | UnsupportedFormat | writes artifact     |

```

{"tool": "recall_prior", "arguments": {"metric": "session_conversion_rate", "segment": "device_category=mobile"}}
// response
{"prior": [{"id": "dx_20210115_07", "t": "2021-01-15", "summary": "mobile mix-shift, $14.2k at risk", "action_status": "experiment_running"}]}

```

18.9 Per-Agent Tool Allow-Lists

Servers are registered globally, but each of the SEVEN AGENTS sees only the tools it needs, enforcing least privilege at the agent layer.

| Agent        | Allowed tools                                                 |
|--------------|---------------------------------------------------------------|
| Orchestrator | list_tables, list_dimensions, recall_prior                    |
| Monitor      | build_query, dry_run, run_query, detect_anomaly               |
| Decompose    | build_query, dry_run, run_query, decompose_change             |
| Diagnose     | build_query, dry_run, run_query, reconcile, significance_test |
| Prescribe    | power_analysis, runtime_estimate, design_experiment           |
| Narrator     | render_brief, export                                          |
| Critic       | reconcile, significance_test, decompose_change, recall_prior  |

18.10 Configuration File

```

# mcp_servers.yaml
servers:
  warehouse-mcp:
    transport: http
    url: https://helios-wh.internal:8443/mcp
    auth: {type: bearer, token_env: HELIOS_WH_TOKEN}
    config: {dataset: bigquery-public-data.ga4_obfuscated_sample_ecommerce,
             byte_budget: 5368709120, require_dry_run: true}
  semantic-mcp:
    transport: stdio
    command: ["python", "-m", "helios.mcp.semantic"]
    config: {registry: ./models/semantic/semantic_layer.yaml}
  stats-mcp:
    transport: stdio
    command: ["python", "-m", "helios.mcp.stats"]
    config: {rng_seed: 1729}
  experiment-mcp:
    transport: stdio
    command: ["python", "-m", "helios.mcp.experiment"]
  report-mcp:
    transport: stdio
    command: ["python", "-m", "helios.mcp.report"]
    config: {memory_db: postgres://helios/memory}

```

18.11 Flow Diagram

```

graph TD
    Claude[Claude Agent SDK (MCP client)]
    Claude --> Orch[Orchestrator]
    Claude --> Monitor[Monitor]
    Claude --> Decompose[Decompose]
    Claude --> Diagnose[Diagnose]
    Claude --> Prescribe[Prescribe]
    Claude --> Narrator[Narrator]
    Claude --> Critic[Critic]
    Orch --> Monitor
    Orch --> Decompose
    Orch --> Diagnose
    Orch --> Prescribe
    Orch --> Narrator
    Orch --> Critic
    Monitor --> Decompose
    Monitor --> Diagnose
    Decompose --> Diagnose
    Decompose --> Prescribe
    Diagnose --> Prescribe
    Diagnose --> Narrator
    Diagnose --> Critic
    Prescribe --> Narrator
    Prescribe --> Critic
    Narrator --> Critic
    Critic --> Narrator
    Critic --> Decompose
    Critic --> Monitor
    Critic --> Diagnose
    Critic --> Prescribe
    Critic --> Narrator
    Critic --> Orchestrator

    subgraph per-agent_allow_list [per-agent allow-list]
        direction TB
        semantic[semantic-mcp]
        warehouse[warehouse-mcp]
        stats[stats-mcp]
        report[report-mcp]
    end

    semantic --> warehouse
    semantic --> stats
    semantic --> report
    semantic --> Critic
    semantic --> Decompose
    semantic --> Monitor
    semantic --> Diagnose
    semantic --> Prescribe
    semantic --> Narrator
    semantic --> Orchestrator

    semantic --> warehouse --> (ONLY path to SQL)
    semantic --> warehouse --> (dry_run + byte-budget gate)
    semantic --> warehouse --> (ONLY path to math)
    semantic --> warehouse --> (memory)
    semantic --> warehouse --> (brief + memory)

    semantic --> warehouse --> (governed SQL)
    semantic --> warehouse --> (ADC service account read-only)
    semantic --> warehouse --> (BigQuery: ga4_obfuscated_sample_ecommerce)
    semantic --> warehouse --> (events_YYYYMMDD shards)

    GUARDRAIL_CHAIN[GUARDRAIL CHAIN per query]
    agent[agent] --> semantic --> warehouse --> dry_run --> cost_check[cost check]

```

```
â warehouse.run_query (â byte budget) â stats.* (all numbers)
â Critic verifies â report.save_diagnosis
```

18.12 Python Tool-Registration Skeleton (warehouse-mcp)

```
# helios/mcp/warehouse.py
import hashlib
from mcp.server.fastmcp import FastMCP
from google.cloud import bigquery

mcp = FastMCP("warehouse-mcp")
_bq = bigquery.Client()           # ADC; least-privilege SA
_BYTE_BUDGET = 5_368_709_120     # fixed per-run cap
_SEEN_DRYRUN = set[ str ] = set() # hashes validated this run

def _h(sql: str) -> str:
    return hashlib.sha256(" ".join(sql.split()).lower().encode()).hexdigest()

@mcp.tool()
def dry_run(sql: str) -> dict:
    """Validate SQL and estimate scanned bytes WITHOUT executing."""
    cfg = bigquery.QueryJobConfig(dry_run=True, use_query_cache=False)
    job = _bq.query(sql, job_config=cfg)
    _SEEN_DRYRUN.add(_h(sql))
    b = job.total_bytes_processed
    return {"valid": True, "total_bytes_processed": b,
            "estimated_cost_usd": round(b / 1e12 * 6.25, 5),
            "referenced_tables": [t.table_id for t in job.referenced_tables]}

@mcp.tool()
def run_query(sql: str, max_bytes_billed: int) -> dict:
    """Execute read-only SQL. Refuses un-dry-run'd or over-budget queries."""
    if _h(sql) not in _SEEN_DRYRUN:
        raise ValueError("NotDryRunFirst: call dry_run before run_query")
    capped = min(max_bytes_billed, _BYTE_BUDGET)
    cfg = bigquery.QueryJobConfig(maximum_bytes_billed=capped)
    job = _bq.query(sql, job_config=cfg)           # ByteBudgetExceeded -> raises
    rows = [r for r in job.result()]
    return {"rows": rows, "row_count": len(rows),
            "bytes_processed": job.total_bytes_processed, "job_id": job.job_id}

if __name__ == "__main__":
    mcp.run(transport="streamable-http")
```

This skeleton encodes the load-bearing invariants: `run_query` cannot execute SQL that was not first dry-run'd in the same run, and cannot exceed the byte budget. Combined with `semantic-mcp` as the sole SQL author and `stats-mcp` as the sole computer of numbers, the architecture guarantees **0 hallucinated columns/metrics (100% governed SQL)** and **100% of findings carrying a significance test and dollar impact** â the FOUNDATION success targets â by construction rather than by convention.

19. Agent Architecture

Helios is a **multi-agent plan-execute-critique system** built on the Claude Agent SDK. Seven agents collaborate under a deterministic control plane: an Orchestrator plans, five worker agents execute one stage each of the diagnosis pipeline, and an adversarial Critic attempts to refute every finding before it ships. The LLM is never the system of record â it is a *composer* that emits tool calls. All SQL flows through `semantic-mcp` ; all math flows through `stats-mcp` . This satisfies the **grounding-over-generation** principle: zero hallucinated columns/metrics (100% governed SQL) and 100% of shipped findings carrying a significance test and a dollar revenue-at-risk.

19.1 The seven agents

| Agent        | Model | Responsibility                                                          | Reads              | Writes                        | MCP tools permitted                                                                                                                      |
|--------------|-------|-------------------------------------------------------------------------|--------------------|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Orchestrator | Opus  | Plan the run, sequence stages, manage budget, route to Critic, finalize | run config, memory | run plan, final brief pointer | <code>warehouse-mcp.list_tables/describe_table</code> , <code>report-mcp.recall_prior</code> , <code>semantic-mcp.list_dimensions</code> |

| Agent     | Model  | Responsibility                                                                   | Reads                   | Writes                                              | MCP tools permitted                                                                                                                                                                          |
|-----------|--------|----------------------------------------------------------------------------------|-------------------------|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Monitor   | Sonnet | Detect which metrics/segments moved abnormally                                   | governed metric series  | anomaly list (metric, dim, t0â†’t1, score)          | <code>semantic-mcp.get_metric/build_query</code> , <code>warehouse-mcp.dry_run/run_query</code> , <code>stats-mcp.detect_anomaly/forecast</code>                                             |
| Decompose | Sonnet | Split each anomaly into mix-shift vs rate-change vs interaction                  | anomaly list            | decomposition table per anomaly                     | <code>semantic-mcp.build_query</code> , <code>warehouse-mcp.run_query</code> , <code>stats-mcp.decompose_change</code>                                                                       |
| Diagnose  | Opus   | Run hypothesis-tree RCA over the dimensional space; verify each branch with SQL  | decompositions          | ranked root-cause candidates w/ evidence            | <code>semantic-mcp.get_metric/build_query/list_dimensions</code> , <code>warehouse-mcp.dry_run/run_query</code> , <code>stats-mcp.significance_test/decompose_change/cohort_retention</code> |
| Prescribe | Sonnet | Convert confirmed root causes into a prioritized, powered experiment backlog     | confirmed findings      | experiment test cards (n, runtime, MDE)             | <code>experiment-mcp.power_analysis/runtime_estimate/design_experiment</code> , <code>semantic-mcp.get_metric</code>                                                                         |
| Critic    | Opus   | Adversarially refute every finding (confound, sample, seasonality, data quality) | findings + all evidence | verdict per finding (PASS/DOWNGRADE/DROP) + reasons | <code>semantic-mcp.build_query</code> , <code>warehouse-mcp.run_query</code> , <code>stats-mcp.significance_test/decompose_change</code> , <code>report-mcp.recall_prior</code>              |
| Narrator  | Sonnet | Compose the executive Decision Brief from surviving findings                     | PASS findings + backlog | rendered brief, persisted diagnosis                 | <code>report-mcp.render_brief/save_diagnosis/export</code> , <code>semantic-mcp.get_metric</code>                                                                                            |

#### Model choice rationale

**Opus** is reserved for the three roles requiring deep multi-step reasoning over an open hypothesis space and high cost-of-error: **Orchestrator** (global planning, budget allocation, branch routing), **Diagnose** (combinatorial hypothesis-tree search with self-directed SQL verification), and **Critic** (adversarial reasoning that must out-think the Diagnose agent to find confounds). **Sonnet** handles the high-volume, more mechanical stages â€” **Monitor**, **Decompose**, **Prescribe**, **Narrator** â€” where the action space is bounded (which metric to series-test, which decomposition to run, which power-analysis to call, which template to fill). This split keeps cost per run inside the fixed BigQuery + token budget while preserving accuracy on the reasoning-critical paths.

#### 19.2 Grounding rules (non-negotiable)

```

RULE G1 The LLM NEVER emits raw SQL. To get data it MUST call
semantic-mcp.build_query(metric, dims, filters, window) or
semantic-mcp.get_metric(name). build_query returns VALIDATED SQL
(governed metric defs + known dimensions only).
RULE G2 The LLM NEVER computes a statistic in prose. Anomaly scores,
decompositions, significance, forecasts, power → stats-mcp /
experiment-mcp ONLY. Numbers in the brief are tool outputs verbatim.
RULE G3 Every query is dry-run (warehouse-mcp.dry_run) BEFORE run_query.
If bytes_scanned would exceed the per-run byte budget the call is
rejected and the agent must narrow window/dims.
RULE G4 Reconcile: aggregate metrics are checked against
warehouse-mcp.reconcile(metric, grain) canonical totals; a >0.5%
drift fails the finding.
RULE G5 Use ONLY canonical metric/dimension names. semantic-mcp rejects
unknown names (anti-hallucination). An unknown name is a hard error,
not a fallback to free SQL.

```



These rules are enforced twice: structurally (the agents are given *only* the MCP tools in their row above – they physically cannot call `run_query` with arbitrary SQL because they never hold a raw-SQL tool) and behaviorally (each system prompt restates the rules).

### 19.3 Control flow â€” the state machine

The Orchestrator drives a finite state machine. Worker output never ships directly; it must pass the Critic loop.

[illegible]

**Critic verdicts.** PASS “finding survives all refutation attempts; proceeds to Prescribe. DOWNGRADE “finding is plausible but evidence is insufficient (e.g. sample too small, decomposition not isolated to one dimension); returns to Diagnose for a targeted re-query, bounded by MAX\_REFUTE\_ROUNDS = 2. After the cap, a still-downgraded finding is demoted to a “watchlist” note in the brief rather than a confident finding. DROP “finding is refuted (confound found, fails reconciliation, fully explained by known seasonality from memory); it is discarded and the cause is added to the run’s local suppression so siblings don’t re-raise it.

#### 19.4 Hypothesis-tree RCA (Diagnose agent)

Diagnose treats root-cause analysis as a **best-first search over the dimensional space**. The root is the top-level moved metric (e.g. `session_conversion_rate` fell). Each tree node is `(metric, dimension-slice, decomposition verdict)`. Children expand the node along the next canonical dimension, ordered by the **rate-effect magnitude** returned by `stats-mcp.decompose_change` â€” the agent always drills into the slice contributing the largest *rate* change (a genuine behavior change) before chasing *mix* effects (composition change, which is often a Simpson's-paradox artifact, not a cause).

```

ROOT: session_conversion_rate -2.1pp WoW (revenue-at-risk computed at leaf)
  â”€â”€â”€ split by device_category
    â”€â”€â”€ mobile rate_effect=-1.6pp share=4pp â”€ expand (largest rate effect)
    â”€â”€â”€ â”€â”€â”€ â”€â”€â”€ split by channel_group
      â”€â”€â”€ â”€â”€â”€ â”€â”€â”€ Paid Search rate_effect=-1.3pp â”€ expand
      â”€â”€â”€ â”€â”€â”€ â”€â”€â”€ â”€â”€â”€ split by landing_page â”€ /sale page rate_effect=-1.1pp [LEAF candidate]
      â”€â”€â”€ â”€â”€â”€ â”€â”€â”€ â”€â”€â”€ Organic Search rate_effect=-0.1pp (prune: below threshold)
      â”€â”€â”€ â”€â”€â”€ desktop rate_effect=-0.2pp (prune)
    â”€â”€â”€ mix term across device = +0.3pp (note: NOT a cause; composition shift)

```

**Pruning rules:** a branch is pruned when its rate-effect is below `MIN_RATE_EFFECT` (a configured pp threshold) OR `significance_test` p-value > 0.05 OR the slice's session count is below the minimum sample for the metric. **Leaf promotion:** a node becomes a candidate finding when it is statistically significant, isolates a single dimension's rate change, survives reconciliation (G4), and carries a non-trivial dollar impact. Search is

breadth-bounded ( `MAX_BRANCHING = 4` slices per node) and depth-bounded ( `MAX_DEPTH = 3` dimensions ) to keep query cost and time-to-diagnosis (<5 min/run) in budget. Each leaf is annotated with **revenue-at-risk** = (counterfactual `session_conversion_rate` at `t0` â€” observed at `t1`)  $\hat{A}$ — affected `sessions`  $\hat{A}$ — `aov` , computed entirely from governed metrics.

## 19.5 Context management, retries, error handling

- **Context windowing.** Each worker agent receives a *compacted* context: the run plan, the canonical FOUNDATION metric/dimension catalog, the upstream stage's structured output (JSON, not prose), and any priors surfaced by `recall_prior` . Raw query result sets are summarized to aggregates before entering the LLM context; the LLM never sees row-level dumps. Inter-agent hand-off is a typed JSON envelope ( `finding_id` , `metric` , `dims` , `t0` , `t1` , `decomposition` , `significance` , `dollar_impact` , `evidence_query_ids` ), keeping token usage flat as the dimensional search widens.
- **Retries.** Transient `warehouse-mcp` / `BigQuery` errors retry with exponential backoff (3 attempts). A `dry_run` budget rejection (G3) is *not* retried blindly â€” the agent must narrow scope (shorter window or fewer dims) before re-issuing. A `semantic-mcp` unknown-name error (G5) is a hard stop for that branch; the agent re-plans the query against `list_dimensions()` .
- **Failure isolation.** A single failed branch in Diagnose is logged and pruned, not fatal to the run. If Monitor/Decompose fail entirely, the Orchestrator aborts and writes an audit record (no partial brief ships). Every tool call, its arguments hash, bytes scanned, and verdict are appended to the run audit trail (see 22.5) so a run is fully reconstructable.

## 19.6 End-to-end autonomous run â€” sequence diagram

```
Scheduler(cron/Cloud Scheduler) â€”â€”â€”> Orchestrator
Orchestrator -> report-mcp.recall_prior(metric, segment) # load priors, suppression
Orchestrator -> warehouse-mcp.list_tables / semantic-mcp.list_dimensions # scope
Orchestrator -> Monitor: "diagnose window t0..t1, budget B"
Monitor -> semantic-mcp.build_query(session_conversion_rate, [day], window)
Monitor -> warehouse-mcp.dry_run -> run_query
Monitor -> stats-mcp.detect_anomaly(series) -> [anomaly: conv_rate -2.1pp]
Monitor -> Orchestrator: anomaly list
Orchestrator -> Decompose: anomaly list
Decompose -> stats-mcp.decompose_change(conv_rate, device_category, t0, t1)
Decompose -> Orchestrator: {mix:+0.3pp, rate:-1.8pp, interaction:-0.6pp}
Orchestrator -> Diagnose: decompositions
Diagnose -> (loop) semantic-mcp.build_query + warehouse-mcp.run_query
Diagnose -> stats-mcp.decompose_change / significance_test # walk hypothesis tree
Diagnose -> Orchestrator: candidate finding F1 (/sale Paid Search mobile, p=0.01, $42k risk)
Orchestrator -> Critic: F1 + evidence
Critic -> report-mcp.recall_prior # known seasonality? prior suppression?
Critic -> stats-mcp.significance_test (re-run on holdout slice)
Critic -> warehouse-mcp.run_query (confound probe: did traffic mix flip?)
Critic -> Orchestrator: VERDICT=PASS (no confound; not seasonal; reconciles)
Orchestrator -> Prescribe: F1
Prescribe -> experiment-mcp.power_analysis(baseline, mde) -> n
Prescribe -> experiment-mcp.runtime_estimate(n, traffic) -> 9 days
Prescribe -> experiment-mcp.design_experiment(hypothesis, metric) -> test card
Prescribe -> Orchestrator: backlog[F1 -> Exp-001]
Orchestrator -> Narrator: PASS findings + backlog
Narrator -> report-mcp.render_brief(findings)
Narrator -> report-mcp.save_diagnosis(...) # writes memory (sec 22)
Narrator -> report-mcp.export(format=pdf/slack)
Narrator -> Orchestrator: brief URL
Orchestrator -> Scheduler: run complete, audit row written
```

A clean run (no anomaly) short-circuits after Monitor and Narrator emits a "no material change" brief, still writing a run-state audit row so absence-of-finding is itself recorded.

# 22. Memory Architecture

Memory is what turns Helios from a stateless analyst into one that **learns across runs**: it stops re-flagging acknowledged causes, recognizes known seasonality and launches, and closes the loop by checking whether prescribed experiments actually moved the metric. Memory is split across **BigQuery tables** (the durable system of record, queryable by the same `warehouse-mcp` ) and a **vector store** (embeddings of past findings for similarity recall). All memory I/O for agents goes through `report-mcp` ( `save_diagnosis` , `recall_prior` ) so the LLM never writes memory directly.

## 22.1 Diagnosis history

Every shipped (and DROPPED, for audit) finding is persisted with an embedding of its natural-language summary so future runs can find similar past diagnoses.

```
-- dataset: helios_memory
CREATE TABLE IF NOT EXISTS helios_memory.diagnosis_history (
  finding_id      STRING NOT NULL,      -- uuid
  run_id          STRING NOT NULL,
  created_at      TIMESTAMP NOT NULL,
  metric          STRING NOT NULL,      -- canonical, e.g. session_conversion_rate
  dimension_slice STRING,               -- e.g. device_category=mobile|channel_group=Paid Search
  t0              DATE, t1 DATE,
  direction       STRING,               -- up|down
  magnitude       FLOAT64,              -- delta in metric units (pp or %)
  mix_effect      FLOAT64,
  rate_effect     FLOAT64,
  interaction_effect FLOAT64,
  p_value         FLOAT64,
  dollar_impact   FLOAT64,              -- revenue-at-risk
  critic_verdict  STRING,               -- PASS|DOWNGRADE|DROP
  root_cause_label STRING,              -- canonical cause taxonomy id
  summary_text    STRING,               -- NL one-liner used for embedding
  embedding       ARRAY<FLOAT64>       -- text-embedding vector (also mirrored to vector store)
)
PARTITION BY DATE(created_at)
CLUSTER BY metric, root_cause_label;
```

**Write path:** Narrator calls `report-mcp.save_diagnosis(finding)` â†’ inserts the row and upserts `(finding_id, embedding, metric, root_cause_label)` into the vector store. **Retrieval path:** `recall_prior(metric, segment)` runs a hybrid query â€” exact filter on `metric + dimension_slice` in BigQuery, plus a vector ANN search on the embedding of the *current* candidate's summary â€” and returns the top-k priors with their verdicts and recency. **Decay:** priors are weighted by  $\exp(-\text{age\_days} / \text{HALF\_LIFE})$  (default `HALF\_LIFE = 60d`, matching the ~3-month dataset window) so stale findings fade but are never hard-deleted (audit requirement).

## 22.2 Suppression list

Acknowledged or intentionally-ignored causes that must **not** be re-raised, so the brief doesn't repeat itself week after week.

```
CREATE TABLE IF NOT EXISTS helios_memory.suppression_list (
  suppression_id STRING NOT NULL,
  metric          STRING NOT NULL,
  dimension_slice STRING,
  root_cause_label STRING,
  reason          STRING,              -- 'acknowledged_by_owner' | 'known_business_decision' | 'wontfix'
  created_by      STRING,              -- user_email or 'critic_auto'
  created_at      TIMESTAMP,
  expires_at      TIMESTAMP,           -- TTL; NULL = permanent
)
CLUSTER BY metric, root_cause_label;
```

**Write path:** a stakeholder acknowledges a finding (via the brief's action UI) â†’ row inserted with `reason` and an `expires_at` TTL (default 30d so a re-emerging issue eventually re-surfaces); the Critic may also auto-suppress within a run. **Retrieval path:** Diagnose and Critic both consult the suppression list before promoting a leaf â€” a candidate matching an unexpired `(metric, dimension_slice, root_cause_label)` is auto-DROPPED with reason "suppressed". **TTL/decay:** rows with `expires_at < now()` are ignored by reads (a scheduled job soft-archives them). This closes the **"stop nagging me"** loop.

## 22.3 Business glossary & context

Definitions, known seasonality, and the launch calendar â€” the priors the Critic uses to refute "this is just seasonal/expected".

```
CREATE TABLE IF NOT EXISTS helios_memory.glossary (
  term          STRING NOT NULL,      -- e.g. 'cart_abandonment_rate'
  definition     STRING,
  canonical_name STRING,               -- maps synonym → canonical metric/dim
  embedding      ARRAY<FLOAT64>
);

CREATE TABLE IF NOT EXISTS helios_memory.seasonality_calendar (
  event_label STRING,                 -- 'Black Friday', 'Christmas dip', 'New Year'
  start_date  DATE, end_date DATE,
  metric      STRING,                 -- affected metric, NULL = all
  expected_dir STRING,                 -- up|down
  expected_mag FLOAT64,                -- typical delta, for confound subtraction
);
```

```

notes          STRING
);

CREATE TABLE IF NOT EXISTS helios_memory.launch_calendar (
  launch_id     STRING,
  launch_date   DATE,
  description    STRING, -- 'new /sale landing page', 'checkout redesign'
  affected_dims STRING,  -- e.g. landing_page=/sale
  affected_metric STRING
);

```

**Write path:** seeded once from the FOUNDATION canonical names (glossary) and the GA4 sample window's known events (the dataset spans 2020-11-01 to 2021-01-31, so Black Friday 2020 and the December peak/January trough are pre-loaded into `seasonality_calendar`); thereafter updated by analysts. **Retrieval path:** the Critic queries `seasonality_calendar` and `launch_calendar` overlapping `[t0,t1]` for the finding's metric/slice; if the observed change is within `expected_mag` of a calendar entry, the finding is `DROP` ped or `DOWNGRADE` d as "explained by known seasonality/launch". The glossary resolves any analyst-typed synonym in the drill-down chat back to a canonical name before it reaches `semantic-mcp` (enforcing G5). **TTL:** glossary/calendar entries are durable (no decay).

## 22.4 Action tracking

Which prescribed experiments shipped, and whether the fix actually worked â€” the **did-the-fix-work** loop.

```

CREATE TABLE IF NOT EXISTS helios_memory.action_tracking (
  experiment_id  STRING NOT NULL,
  finding_id     STRING NOT NULL, -- FK → diagnosis_history
  hypothesis     STRING,
  target_metric  STRING,          -- canonical
  baseline_value FLOAT64,
  mde            FLOAT64,         -- minimum detectable effect from power_analysis
  required_n     INT64,
  est_runtime_days INT64,
  status         STRING,          -- proposed|shipped|running|completed|abandoned
  shipped_at     TIMESTAMP,
  result_lift    FLOAT64,         -- observed effect once completed
  result_p_value FLOAT64,
  outcome        STRING,          -- win|flat|loss|inconclusive
  updated_at    TIMESTAMP
)
CLUSTER BY target_metric, status;

```

**Write path:** Prescribe inserts a `proposed` row per test card (with `required_n`, `mde`, `est_runtime_days` from `experiment-mcp`); a webhook/manual update from the experimentation platform advances `status` and writes `result_lift` / `outcome` on completion. **Retrieval path:** at the start of each run, the Orchestrator calls `recall_prior` which joins `action_tracking` so the brief reports "Exp-001 for the /sale finding completed: +1.4pp conv, p=0.02, WIN" and so Diagnose down-weights a root cause whose fix already shipped and won. **Loop closure:** an `outcome = win` validates the prior diagnosis (raises confidence on that `root_cause_label` in future similarity scoring); an `outcome = loss/flat` flags the original finding as a likely false diagnosis, lowering its prior weight â€” Helios literally learns which of its diagnoses were correct.

## 22.5 Run state / audit trail

Full reconstructability of every autonomous run â€” the determinism and verify-then-trust principles depend on it.

```

CREATE TABLE IF NOT EXISTS helios_memory.run_state (
  run_id          STRING NOT NULL,
  started_at      TIMESTAMP, ended_at TIMESTAMP,
  trigger         STRING,      -- 'scheduler' | 'manual'
  window_t0       DATE, window_t1 DATE,
  state           STRING,      -- PLAN|MONITOR|...|END|ABORTED
  bytes_scanned   INT64,       -- vs per-run byte budget
  total_cost_usd  FLOAT64,
  n_findings      INT64, n_passed INT64, n_dropped INT64,
  brief_uri       STRING
);

CREATE TABLE IF NOT EXISTS helios_memory.audit_log (
  run_id          STRING, step_seq INT64,
  agent           STRING,          -- Orchestrator|Monitor|...
  mcp_tool        STRING,          -- e.g. semantic-mcp.build_query
  args_hash       STRING,          -- sha256 of normalized args
  sql_text        STRING,          -- governed SQL actually run (NULL for non-query tools)
  bytes_scanned   INT64,
  latency_ms      INT64,

```

```

verdict      STRING,          -- for Critic steps
ts           TIMESTAMP
)
PARTITION BY DATE(ts);

```

**Write path:** the Orchestrator opens a `run_state` row at `PLAN` and updates `state` /budget counters at each transition; every MCP call from any agent appends an `audit_log` row (the control plane wraps all tool invocations). **Retrieval path:** internal `â€` used to enforce the per-run BigQuery byte budget mid-run (if `bytes_scanned` nears the cap the Orchestrator tightens scope), to verify the "0 hallucinated metrics / 100% governed SQL" target by proving every `sql_text` came from `semantic-mcp`, and to power the offline eval harness that grades root-cause accuracy (target  $\geq 85\%$  vs  $\leq 45\%$  naive baseline). **TTL:** durable; partitioned by day for cheap retention management.

## 22.6 How memory closes the loop

```

RUN N    diagnose F → save_diagnosis (history + embedding)
          → prescribe Exp → action_tracking(proposed)
stakeholder ack → suppression_list (TTL 30d)
Exp ships+completes → action_tracking(outcome=win/loss)
RUN N+1  recall_prior(metric, segment):
  - similar prior F via vector ANN → "seen before, here is the trend"
  - suppression hit → auto-DROP, don't re-nag
  - seasonality/launch overlap → Critic refutes as expected
  - action outcome=win → down-weight already-fixed cause
  - action outcome=loss → lower prior confidence (was a bad diagnosis)

```

Each memory type therefore feeds a distinct learning signal: history gives continuity and similarity, suppression prevents repetition, glossary/calendar supplies the confound priors the Critic needs, action tracking validates or invalidates past diagnoses, and the audit trail guarantees every run is governed, budgeted, and reproducible.

## 20. Evaluation Framework

The Evaluation Framework is the trust centerpiece of Helios. Because the LLM agents (Orchestrator, Monitor, Decompose, Diagnose, Prescribe, Narrator, Critic) make causal-sounding claims about WHY the funnel moved, those claims are worthless unless we can prove the system finds the *right* root cause more reliably than a naive analyst. We prove this with an **offline, labeled benchmark**: we inject synthetic-but-known anomalies into a frozen copy of the GA4 data, run the full Helios pipeline against the perturbed copy, and grade its diagnosis against ground truth we recorded at injection time. The headline contract: **root-cause segment accuracy  $\geq 85\%$  on the labeled benchmark vs  $\leq 45\%$  for the naive baseline, with 0 hallucinated columns/metrics and 100% of findings carrying a significance test and a dollar revenue-at-risk.**

### 20.1 Why offline injection (and not the live data)

`bigquery-public-data.ga4_obfuscated_sample_ecommerce` is a fixed historical export (~2020-11-01 to 2021-01-31). It contains real anomalies, but they are **unlabeled** `â€` we do not know their true root causes, so we cannot grade against them. We therefore construct a *counterfactual*: take a real baseline period, clone it, and surgically perturb one or more (`metric`, `dimension-segment`, `time`) cells by a **known amount**, recording the perturbation as ground truth. The pipeline must rediscover what we hid. This is the only way to compute decomposition error and root-cause accuracy with confidence intervals.

### 20.2 Injection mechanism

Injection operates on a **scenario fixture table** materialized in the eval dataset `helios_eval`. We never mutate the public source. The flow:

- `warehouse-mcp.run_query` extracts a baseline window into `helios_eval.fct_daily_funnel_base` (the canonical `fct_daily_funnel` grain: one row per `day` x dimension cell with `sessions`, `view_item_sessions`, `add_to_cart_sessions`, `begin_checkout_sessions`, `purchasing_sessions`, `revenue`, `transactions`).
- A deterministic Python injector (seeded) reads a **scenario spec** (YAML below) and produces `helios_eval.fct_daily_funnel_perturbed` by applying one of two perturbation primitives at the specified `inject_at` date onward:
  - rate perturbation:** multiply a segment's *conversion rate* at a funnel step by `rate_multiplier`. Operationally, for a segment cell we recompute the numerator (e.g. `purchasing_sessions`) as `round(sessions * base_rate * rate_multiplier)` while holding `sessions` (the volume/weight `w_i`) fixed. This changes `r_i` only  $\rightarrow$  ground-truth = **rate-change**.
  - volume/mix perturbation:** multiply a segment's `sessions` by `volume_multiplier` while holding that segment's per-step rates fixed, then renormalize so total sessions is conserved within tolerance. This changes `w_i` only  $\rightarrow$  ground-truth = **mix-shift**.
- The injector writes a **ground-truth label record** to `helios_eval.labels` capturing: `scenario_id`, `anomaly_type` (`mix` | `rate` | `mixed` | `none`), affected metric, affected segment key(s) and dimension, `inject_at`, the *true* mix/rate/interaction contributions computed analytically

by the same decomposition algebra Helios is graded on (see Foundation CORE ALGORITHM), and the **true dollar-at-risk** =  $(\text{counterfactual\_revenue\_without\_perturbation} - \text{perturbed\_revenue})$  summed over the post-injection window.

Because perturbations are applied to aggregates with conserved totals, the analytic mix/rate/interaction split is exact and serves as the gold target for decomposition MAPE.

#### Scenario-spec example

```
# helios_eval/scenarios/S017_paid_search_mobile_rate_drop.yaml
scenario_id: S017
title: "Paid Search x mobile checkout-to-purchase rate collapse"
anomaly_type: rate # mix | rate | mixed | none
seed: 1759 # deterministic injector seed
baseline_window: { start: "2020-12-01", end: "2020-12-20" }
inject_at: "2020-12-21"
eval_window: { start: "2020-12-21", end: "2021-01-10" }
target_metric: checkout_to_purchase_rate
funnel_step: { numerator: purchasing_sessions, denominator: begin_checkout_sessions }
perturbation:
  dimension: [channel_group, device_category]
  segment: { channel_group: "Paid Search", device_category: "mobile" }
  rate_multiplier: 0.55 # 45% relative rate drop in this cell only
expected_ground_truth:
  root_cause_segment: { channel_group: "Paid Search", device_category: "mobile" }
  dominant_effect: rate # graded against Decompose output
  is_seasonality_decay: false
  dollar_at_risk_used: null # filled by injector post-materialization
controls:
  hold_constant: [sessions] # volume fixed → isolates rate effect
```

### 20.3 The eval dataset (>= 30 scenarios)

The benchmark ships **50 scenarios** spanning the required coverage axes. Each must be reproducible from its seed.

| Bucket                                     | Count | What it tests                                                                                                                                                        |
|--------------------------------------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Single-segment rate-change                 | 10    | One dimension cell's step rate moves; Decompose must attribute to <b>rate</b> effect and pin the segment.                                                            |
| Single-segment mix-shift                   | 10    | One segment's <code>sessions</code> share moves; must attribute to <b>mix</b> effect (Simpson's-paradox guard).                                                      |
| Multi-segment rate-change                  | 6     | Several cells move together; top-3 must contain all true cells.                                                                                                      |
| Multi-segment mixed (mix+rate+interaction) | 6     | Both <code>w_i</code> and <code>r_i</code> move; tests interaction-term handling.                                                                                    |
| Seasonality decoys                         | 6     | A <i>real</i> seasonal swing (e.g. post-holiday week-over-week dip) is present but is <b>not</b> the injected anomaly; system must NOT flag it (Critic must refute). |
| No-anomaly controls                        | 6     | Zero perturbation; tests false-positive rate of Monitor ( <code>detect_anomaly</code> ).                                                                             |
| Data-quality / confound                    | 6     | Injected NULL spikes, transaction_id duplication, late-arriving shard; Critic must catch as data quality, not behavior.                                              |

### 20.4 Metrics

Each metric below is computed by the harness in deterministic Python (never the LLM) and aggregated across scenarios.

| Metric                                     | Definition                                                                                                        | Target           |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------|------------------|
| <b>Root-cause segment accuracy (top-1)</b> | fraction of scenarios where Diagnose's #1 root-cause segment == labeled segment                                   | <b>&gt;= 85%</b> |
| Root-cause segment accuracy (top-3)        | labeled segment appears in Diagnose's top-3 ranked candidates                                                     | >= 95%           |
| <b>Decomposition error (MAPE)</b>          | mean abs % error between Decompose's estimated mix/rate/interaction contributions and analytic ground truth       | <= 10%           |
| Anomaly precision / recall / F1            | over all (scenario, day, metric) cells flagged by Monitor vs labeled injections; controls contribute to precision | F1 >= 0.85       |

| Metric                          | Definition                                                                                            | Target    |
|---------------------------------|-------------------------------------------------------------------------------------------------------|-----------|
| Dollar-at-risk estimation error | abs % error of estimated revenue-at-risk vs label <code>dollar_at_risk_usd</code>                     | <= 15%    |
| <b>Hallucination rate</b>       | any column/metric in emitted SQL not present in the semantic-mcp registry or GA4 schema (AST-checked) | <b>0%</b> |
| Faithfulness                    | does Narrator's prose claim match the SQL evidence + stats-mcp outputs (entailment check)             | >= 0.95   |

**Top-1 accuracy** is the headline. **Hallucination rate** is a hard gate (any non-zero value fails CI regardless of accuracy) because grounding-over-generation is a first principle.

20.5 Naive baseline ("largest absolute segment delta")

The baseline a competent-but-unsophisticated analyst would use: for the anomalous metric, compute each segment's `delta = value_at(t1) - value_at(t0)`, rank by `|delta|`, and declare the single largest-magnitude segment the root cause. It performs **no mix-vs-rate decomposition**, so it is systematically fooled by mix-shift (a high-volume segment whose *rate* barely moved still shows the largest absolute delta). On our 50-scenario benchmark this baseline scores **~45% top-1** (it gets pure single-segment rate cases right and most mix cases wrong). Helios must clear **85%**, a near-doubling, which is the project's central empirical claim.

20.6 Harness architecture

```
helios/eval/
injector.py      # seeded perturbation → fct_daily_funnel_perturbed + labels
runner.py       # for each scenario: point pipeline at perturbed copy, run all 7 agents
scorers/
  rootcause.py  # top-1 / top-3 segment accuracy
  decomposition.py # MAPE on mix/rate/interaction
  detection.py  # precision/recall/F1 for Monitor
  dollars.py    # dollar-at-risk error
  hallucination.py # SQL AST vs semantic-mcp registry + GA4 schema
  faithfulness.py # narrative↔evidence entailment (Critic-as-judge + rule checks)
report.py       # aggregates → results table + per-scenario JSON + markdown
scenarios/*.yaml # the 50 specs
```

```
# runner.py (core loop, abbreviated)
def run_benchmark(scenarios, pipeline):
    results = []
    for spec in scenarios:
        inject(spec) # materialize perturbed copy + label
        ctx = PipelineContext(dataset="helios_eval",
                              table="fct_daily_funnel_perturbed",
                              window=spec.eval_window)
        diag = pipeline.run(ctx) # Orchestrator..Narrator+Critic
        label = load_label(spec.scenario_id)
        results.append(score_all(diag, label, scorers))
    return aggregate(results) # → results table
```

The harness pins random seeds, freezes the dbt model SHA, and records the semantic-mcp registry hash so a run is fully reproducible. Each scenario emits a per-scenario JSON artifact (predicted vs label, every sub-score) for debugging.

20.7 Scoring details

- **Root-cause matching** is on the normalized segment key (sorted dimension=value pairs). A predicted segment matches the label iff dimension set and values are equal; partial matches (right dimension, wrong value) count as miss for top-1.
- **Decomposition MAPE** is computed only on scenarios where a true effect exists (controls excluded), comparing the three contribution buckets element-wise, then averaging.
- **Faithfulness** runs two checks: (1) a rule check that every numeric claim in the brief has a backing `run_query` result hash and a `significance_test` p-value attached; (2) a Critic-as-judge entailment pass that flags any sentence not entailed by the evidence bundle. Both must pass.

20.8 Regression gating in CI

The benchmark runs in **GitHub Actions** as a required check on every PR that touches `models/`, `semantic/`, `agents/`, or `eval/`. Gating thresholds (stored in `eval/gates.yaml`):

```

gates:
  rootcause_top1_min: 0.85
  decomposition_mape_max: 0.10
  hallucination_rate_max: 0.00      # hard zero
  detection_f1_min: 0.85
  dollar_error_max: 0.15
  faithfulness_min: 0.95
  regression_tolerance: 0.02      # top1 may not drop >2pts vs main baseline

```

CI fails the PR if any gate is breached OR if top-1 regresses more than `regression_tolerance` against the committed `main` baseline ( `eval/baselines/main.json` ). To bound cost (Foundation byte-budget target), the harness runs every scenario through `warehouse-mcp.dry_run` first and aborts if total scanned bytes exceed the per-run budget; a 12-scenario **smoke subset** runs on every push, the full 50 on PRs to `main` .

## 20.9 Results-table template

```

Helios Eval Report â€” run 2026-06-03  model_sha=ab12cd  registry=9f3e

```

| Metric               | Helios | Baseline | Target      | Pass |
|----------------------|--------|----------|-------------|------|
| Root-cause top-1     | 0.882  | 0.441    | $\geq 0.85$ | PASS |
| Root-cause top-3     | 0.971  | 0.618    | $\geq 0.95$ | PASS |
| Decomposition MAPE   | 0.073  | n/a      | $\leq 0.10$ | PASS |
| Anomaly detection F1 | 0.901  | 0.560    | $\geq 0.85$ | PASS |
| Dollar-at-risk error | 0.118  | n/a      | $\leq 0.15$ | PASS |
| Hallucination rate   | 0.000  | 0.000    | $== 0.00$   | PASS |
| Faithfulness         | 0.962  | n/a      | $\geq 0.95$ | PASS |

```

Scenarios: 50  Cost: 2.1 GB scanned (budget 5 GiB (~5.37 GB))  Time: 4m12s/run

```

This table is rendered to the PR comment and archived under `eval/history/` , giving a longitudinal record of whether the 85%-vs-45% claim continues to hold as the system evolves.

## 21. Experimentation Framework

Helios does not stop at diagnosis. The **Prescribe** agent turns each surviving finding into a **statistically-defensible experiment design**: a hypothesis card, a powered sample size, a runtime estimate, and a prioritization score. The honest constraint stated up front: **this GA4 dataset is OBSERVATIONAL and historical â€” there is no live traffic to A/B test against**. Helios therefore (a) *designs and sizes* experiments a team could run, and (b) for already-occurred changes, runs **quasi-experimental readbacks** (pre/post and difference-in-differences) using `stats-mcp` . Every prescribed experiment ties back to a `dollar-at-risk` from the diagnosis, so the backlog is ranked by money.

### 21.1 Hypothesis-card schema

Each prescription is a governed object produced by `experiment-mcp.design_experiment(hypothesis, metric)` and persisted via `report-mcp.save_diagnosis` .

```

hypothesis_card:
  card_id: H-2021-0042
  source_finding_id: F-2021-0042      # link to the diagnosis it came from
  hypothesis: >
    Because mobile begin_checkout_sessions on Paid Search convert at a 45%
    lower checkout_to_purchase_rate post 2020-12-21, simplifying the mobile
    payment step will recover purchasing_sessions.
  target_metric: checkout_to_purchase_rate # canonical metric name
  segment: { channel_group: "Paid Search", device_category: "mobile" }
  expected_mechanism: "reduce form friction at add_payment_info → purchase"
  variant_description: "one-tap wallet + autofill on mobile checkout"
  primary_metric: checkout_to_purchase_rate
  guardrail_metrics: {aov, cart_abandonment_rate, revenue_per_session, net_revenue}
  baseline_rate: 0.061                # observed in control segment
  mde_relative: 0.10                  # detect a 10% relative lift
  alpha: 0.05
  power: 0.80
  test: two_proportion_z
  sample_size_per_arm: null           # filled by power_analysis
  runtime_days: null                  # filled by runtime_estimate
  ice_score: null
  lifecycle_state: proposed

```



## 21.2 Statistics

All math is delegated to `stats-mcp` and `experiment-mcp`; the LLM only supplies parameters. For a binomial primary metric (a conversion rate at a funnel step), the required sample size per arm for a **two-proportion z-test** is:

```
p1 = baseline_rate
p2 = baseline_rate * (1 + mde_relative)      # treatment under H1
pbar = (p1 + p2) / 2
n_per_arm = ( z_(1-alpha/2) * sqrt(2*pbar*(1-pbar))
             + z_(1-beta) * sqrt(p1*(1-p1) + p2*(1-p2)) )^2
             / (p2 - p1)^2
```

with  $z_{(1-\alpha/2)}=1.96$  at  $\alpha=0.05$  (two-sided) and  $z_{(1-\beta)}=0.84$  at  $\text{power}=0.80$ . `experiment-mcp.power_analysis(baseline, mde, alpha, power)` returns `n_per_arm`; `runtime_estimate(n, traffic)` divides total required  $n$  ( $2 * n\_per\_arm$ ) by the **observed eligible traffic rate** for the segment (sessions/day reaching the funnel step) to yield `runtime_days`.

```
# worked example for card H-2021-0042
p1 = 0.061; mde = 0.10
n = power_analysis(baseline=p1, mde=mde, alpha=0.05, power=0.80)["n_per_arm"]
# ~ 14,800 begin_checkout_sessions per arm
traffic = run_query("select avg(begin_checkout_sessions) "
                   "from fct_daily_funnel "
                   "where channel_group='Paid Search' and device_category='mobile'")
# observed ~ 95 begin_checkout_sessions/day in this segment
runtime = runtime_estimate(n_total=2*n, traffic_per_day=95) # ~ 311 days → UNDERPOWERED
```

This example deliberately surfaces the dataset's core limitation: thin per-segment traffic makes many fine-grained tests impractically long. The Prescribe agent must report `runtime_days` honestly and, when it exceeds a threshold, recommend **rolling the test up to a coarser segment** (e.g. all `mobile` rather than `Paid Search x mobile`) or relaxing `mde_relative`, re-sizing, and noting the trade-off.

### Sequential / peeking caveats

Naively checking a running test repeatedly inflates the false-positive rate far above  $\alpha$ . Helios designs are **fixed-horizon by default** (decide  $n$  up front, read once at `runtime_days`). When continuous monitoring is desired, `experiment-mcp.design_experiment` applies an **alpha-spending / group-sequential** correction (O'Brien-Fleming-style boundaries) or specifies an **always-valid sequential test** (mSPRT), and the card records the chosen procedure so the readout uses the matching stopping rule. The Critic explicitly checks every readout for peeking violations.

### Multiple-comparison handling

A single run can generate many hypothesis cards. When several primaries are tested in one program, Helios applies **Benjamini-Hochberg FDR control** across the family of p-values (preferred for backlog screening) or Bonferroni for a small confirmatory set, and records the adjusted threshold on each card. Guardrail metrics are evaluated as one-sided non-inferiority checks at their own  $\alpha$ .

## 21.3 Mapping onto the GA4 dataset

| Testable primary metric   | Funnel step                     | Typical daily volume (store-wide)          | Testability                          |
|---------------------------|---------------------------------|--------------------------------------------|--------------------------------------|
| view_to_cart_rate         | view_item -> add_to_cart        | high (thousands of view_item_sessions/day) | strong "€" fast tests                |
| cart_to_checkout_rate     | add_to_cart -> begin_checkout   | medium                                     | moderate                             |
| checkout_to_purchase_rate | begin_checkout -> purchase      | low (hundreds/day store-wide)              | weak at segment grain                |
| session_conversion_rate   | sessions -> purchasing_sessions | medium denominator, low numerator          | moderate store-wide, weak by segment |
| aov / revenue_per_session | continuous, per transaction     | low transaction count                      | weak (high variance)                 |

Rule of thumb encoded in Prescribe: tests on **upper-funnel rate metrics** (`view_to_cart_rate`) are well-powered within weeks; **lower-funnel and revenue metrics** are usually only powerable at coarse grain or store-wide. Continuous metrics (`aov`, `revenue_per_session`) use a Welch t-test (via `significance_test`) rather than the z-test and need variance estimates from the data.

## 21.4 Prioritization model (ICE / PIE)

Helios scores every card with **ICE** and ranks the backlog descending:

```
Impact    = dollar_at_risk (from diagnosis) x expected_relative_lift (mde or modeled)
Confidence = evidence_strength in [0,1]
```

```

= f(significance p-value, decomposition cleanliness, sample adequacy,
    Critic-survival, prior-similar-finding outcomes from memory)
Effort    = engineering estimate in [1..10] (variant complexity + instrumentation)

ICE_score = (Impact_normalized * Confidence) / Effort

```

Impact is grounded in **real dollars** because every finding carries a `dollar-at-risk`, so the backlog is literally sorted by recoverable revenue per unit effort. `Confidence` downweights cards whose `runtime_days` is impractical or whose decomposition had high interaction (ambiguous attribution). A PIE variant (Potential, Importance, Ease) is available as an alternate weighting; both are deterministic and recorded on the card for auditability.

## 21.5 Experiment lifecycle and memory tracking

```

proposed â€œâ€œ> designed â€œâ€œ> running â€œâ€œ> readout â€œâ€œ> (archived: won | lost | inconclusive)
â€œ,      â€œ,      â€œ,      â€œ,
Prescribe power+runtime team runs stats-mcp readback
emits card sized & ICE'd (external) + Critic verifies

```

- **proposed:** Prescribe creates the card from a surviving finding.
- **designed:** `power_analysis` + `runtime_estimate` fill `sample_size_per_arm` and `runtime_days`; ICE computed; Critic checks the design (powered? guardrails sensible? peeking rule set?).
- **running:** external to Helios (no live traffic here) â€œ state tracked only.
- **readout:** Helios computes results. For real online tests, `significance_test(a,b)`. For this **observational dataset**, Helios runs **quasi-experimental readbacks**: a **pre/post** comparison of the segment around `inject_at`, and a **difference-in-differences** that nets out a comparable control segment to remove seasonality ( $DiD = (treated\_post - treated\_pre) - (control\_post - control\_pre)$ ), with the parallel-trends assumption checked on the pre-period and reported. The Critic attempts to refute (confounding, non-parallel pre-trends, mix-shift in the control).

Every state transition and result is persisted by `report-mcp.save_diagnosis` into the **Memory store**, and `report-mcp.recall_prior(metric, segment)` lets future runs see whether a similar experiment already won or lost â€œ feeding the `Confidence` term and preventing the backlog from re-proposing settled questions. This closes the loop from autonomous diagnosis to a living, money-ranked, statistically-defensible experiment backlog.

## 23. Future Roadmap

Helios is delivered along a five-phase maturity ladder. Each phase is independently demoable and shippable; each maps to a maturity level (L1 Intern MVP, L2 Strong Portfolio, L3 Top-1% Undergrad) plus two production-frontier phases beyond. The discipline is: never start a phase until the prior phase's exit criteria are green in CI (dbt build + tests + eval harness all passing).

### 23.1 Phase 0 â€œ MVP (L1, Intern-level)

**Scope.** Single-tenant, batch, read-only against `bigquery-public-data.ga4_obfuscated_sample_ecommerce`. Prove the grounding spine works end to end on one diagnosis path.

#### Key deliverables.

- dbt staging + intermediate + facts: `stg_ga4__events`, `stg_ga4__event_params`, `int_ga4__sessionized`, `int_ga4__funnel_steps`, `fct_sessions`, `fct_daily_funnel`. Sessionization on (`user_pseudo_id`, `ga_session_id`).
- The canonical macro funnel computed for the full window: `sessions`  $\rightarrow$  `view_item_sessions`  $\rightarrow$  `add_to_cart_sessions`  $\rightarrow$  `begin_checkout_sessions`  $\rightarrow$  `purchasing_sessions`, plus `session_conversion_rate`, `view_to_cart_rate`, `cart_to_checkout_rate`, `checkout_to_purchase_rate`.
- `warehouse-mcp` (`list_tables`, `describe_table`, `dry_run`, `run_query`, `reconcile`) and a minimal `semantic-mcp` (`get_metric`, `build_query`) exposing the canonical metric names.
- Monitor agent runs `stats-mcp.detect_anomaly` on `session_conversion_rate` daily series; Narrator emits a plain-text brief via `report-mcp.render_brief`.

**Exit criteria.** `reconcile('revenue', 'day')` matches a hand-written control query to the cent; 0 hallucinated columns (every column traces to the GA4 schema); a single anomaly produces a brief in under 5 minutes. **Skill demonstrated:** analytics-engineering fundamentals + governed-SQL grounding.

### 23.2 Phase 1 â€œ Strong Portfolio (L2)

**Scope.** The full seven-agent plan-execute-critique loop and the core decomposition algorithm, statistically defended.

**Key deliverables.**

- All seven agents wired on the Claude Agent SDK: Orchestrator -> Monitor -> Decompose -> Diagnose -> Prescribe -> Narrator, with the Critic refuting every finding.
- `stats-mcp.decompose_change` implementing mix effect / rate effect / interaction across canonical dimensions ( `device_category` , `channel_group` , `country` , `landing_page` ). This is the technical centerpiece: it distinguishes mix-shift from rate-change and resolves Simpson's paradox.
- `experiment-mcp` ( `power_analysis` , `runtime_estimate` , `design_experiment` ) producing a prioritized, statistically-defensible backlog; every finding carries `significance_test` results AND a dollar revenue-at-risk AND a recommended action.
- Offline eval harness with a labeled benchmark in GitHub Actions CI; memory store persisting prior diagnoses + suppression list + business glossary.

**Exit criteria.** Diagnosis root-cause accuracy >= 85% on the labeled benchmark vs <= 45% naive baseline; 100% of findings carry significance + dollar impact; query cost per run under the fixed BigQuery byte budget. **Skill demonstrated:** causal-style root-cause analysis, experiment design, trustworthy multi-agent orchestration.

23.3 Phase 2 “ Top-1% Undergrad (L3)

**Scope.** Autonomy, depth, and adversarial rigor that read as production thinking.

**Key deliverables.**

- Scheduler (Cloud Scheduler/cron) for proactive autonomous runs; `report-mcp.recall_prior` + action-tracking close the loop on prior recommendations.
- Forecasting ( `stats-mcp.forecast` via prophet/pmdarima) for expected-vs-actual deltas; `cohort_retention` and `rfm_segment` for behavioral segmentation feeding the Diagnose hypothesis tree.
- Hardened GA4-style default channel grouping honoring the `traffic_source` gotcha (session-scoped `event_params` source/medium first, user first-touch fallback).
- Critic expanded to a full refutation battery: mix-shift confound, insufficient sample, seasonality, data quality.

**Exit criteria.** Time-to-diagnosis < 5 min/run sustained on schedule; Critic catch-rate measured on injected adversarial cases; eval accuracy holds >= 85% across all canonical dimensions. **Skill demonstrated:** autonomous systems design, statistical depth, adversarial verification.

23.4 Phase 3 “ Productionization

**Scope.** Multi-tenant, real-time streaming, warehouse-agnostic.

**Key deliverables.** Tenant isolation (per-tenant semantic layer + byte budgets); streaming ingestion of GA4 events (intraday) feeding `fct_daily_funnel` near-real-time; warehouse-agnostic adapters behind `warehouse-mcp` (Snowflake/Databricks/DuckDB) so the semantic layer is the only SQL author regardless of dialect; observability (per-agent latency, token cost, cache hit-rate). **Exit criteria.** Two warehouses pass identical reconcile tests; p95 run latency under SLA at N tenants. **Skill demonstrated:** platform engineering, multi-tenancy, infra abstraction.

23.5 Phase 4 “ Frontier

**Scope.** True causal inference and closed-loop experimentation.

**Key deliverables.** Uplift modeling / causal inference (difference-in-differences, synthetic control, double-ML) replacing correlational decomposition where data supports it; automated experiment execution via integrations (push test cards to an experimentation platform, read back results, auto-update the backlog); multi-dataset (join GA4 with CRM/cost data for true CAC/ROAS). **Exit criteria.** A causal estimate validated against a held-out randomized experiment. **Skill demonstrated:** causal ML, full autonomous experimentation loop.

23.6 Capability-maturity table

| Capability                      | P0 (L1)   | P1 (L2)      | P2 (L3)           | P3 Prod         | P4 Frontier     |
|---------------------------------|-----------|--------------|-------------------|-----------------|-----------------|
| Governed SQL (semantic-mcp)     | basic     | full         | full              | multi-warehouse | + multi-dataset |
| Anomaly detection (Monitor)     | threshold | statistical  | forecast-residual | streaming       | causal triggers |
| Mix-shift vs rate decomposition | “         | core         | all dims          | per-tenant      | + uplift        |
| Experiment backlog              | “         | power+design | prioritized       | “               | auto-executed   |

| Capability           | P0 (L1) | P1 (L2) | P2 (L3)      | P3 Prod          | P4 Frontier   |
|----------------------|---------|---------|--------------|------------------|---------------|
| Critic refutation    | â€”     | basic   | full battery | â€”              | causal checks |
| Autonomy             | manual  | manual  | scheduled    | multi-tenant SLA | closed-loop   |
| Revenue-at-risk (\$) | â€”     | yes     | yes          | yes              | causal \$     |

23.7 Explicitly deferred items (with rationale)

| Deferred item         | Phase deferred to | Rationale                                                                                                                                                    |
|-----------------------|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| True customer LTV     | P4 + new data     | Dataset window is ~3 months (2020-11-01 to 2021-01-31); too short to observe lifetime value or long retention curves. Report 30/60-day proxy retention only. |
| Cross-device identity | P4                | <code>user_id</code> is almost always NULL; <code>user_pseudo_id</code> is device/cookie-scoped, so identity stitching is unsound on this data.              |
| True causal inference | P4                | No experiment assignment in the public data; P1-P2 ship correlational decomposition with explicit confound caveats from the Critic.                          |
| Real-time streaming   | P3                | MVP value is in diagnosis quality, not latency; batch is sufficient for a 3-month static sample.                                                             |
| ML-based attribution  | P4                | Requires session-scoped channel + cost data the sample lacks; use GA4 default channel grouping until then.                                                   |

24. Interview Narrative

24.1 The 60-second pitch

"Helios is an autonomous AI Growth Analyst. Instead of another 'ask your data' chatbot, it runs on a schedule and proactively diagnoses *why* an e-commerce funnel is moving. Its centerpiece is a mix-shift-versus-rate-change decomposition that tells you whether conversion dropped because *behavior* changed or because your *traffic composition* changed â€” that's how it dodges Simpson's paradox. Every finding ships with a significance test, a dollar revenue-at-risk number, and a prioritized experiment to fix it. The trust story is grounding: the LLM never writes raw SQL or does math â€” it composes governed metrics through a semantic layer and calls deterministic stats tools over MCP, and an adversarial Critic agent tries to refute every finding before it ships. On a labeled benchmark it hits 85%+ root-cause accuracy versus 45% for a naive baseline."

24.2 The 2-minute deep-dive

"The architecture is seven agents in a plan-execute-critique loop on the Claude Agent SDK. The Orchestrator plans; Monitor runs anomaly detection on canonical series like `session_conversion_rate`; Decompose runs the mix/rate/interaction math; Diagnose builds a hypothesis tree and verifies each branch with governed SQL; Prescribe designs experiments with power analysis; Narrator writes the executive Decision Brief. The Critic reviews everything adversarially â€” it checks for mix-shift confounds, insufficient sample, seasonality, and data-quality issues â€” and a finding only ships if it survives.

The grounding spine is five MCP servers: warehouse-mcp for execution and reconciliation, semantic-mcp as the *only* path to SQL, stats-mcp as the *only* path to math, experiment-mcp for design, and report-mcp for briefs and memory. That boundary is the whole trust argument: hallucinated columns are structurally impossible because the model can only reference governed metrics like `view_to_cart_rate` or `aov`, and statistics are deterministic code, not generated tokens.

The data is the GA4 obfuscated sample â€” event-level, date-sharded, with the classic gotchas: `traffic_source` is user first-touch not session source, and sessions are keyed on `(user_pseudo_id, ga_session_id)`. dbt models it cleanly from `stg_ga4__events` up to `fct_daily_funnel`, and an offline eval harness in CI grades root-cause accuracy on every commit."

24.3 STAR stories

**(a) The mix-shift-vs-rate-change insight.** *Situation:* overall `session_conversion_rate` fell week-over-week and the obvious read was "checkout is broken." *Task:* find the real cause before prescribing a fix. *Action:* I built `stats-mcp.decompose_change`, decomposing the aggregate rate  $R = \sum_i w_i * r_i$  into mix effect  $\sum_i (\Delta w_i * r_{i\_at\_t0})$ , rate effect  $\sum_i (w_{i\_at\_t0} * \Delta r_i)$ , and interaction. *Result:* the per-segment rates were flat â€” the drop was almost entirely *mix*: a surge of low-converting mobile/Paid Social traffic shifted the weights. The prescription flipped from "fix checkout" to "fix acquisition mix," and the decomposition is auditable.

**(b) Trustworthy AI via the eval harness.** *Situation:* "the AI says so" is not shippable. *Task:* make trust measurable. *Action:* built a labeled benchmark of seeded funnel changes with known root causes and wired it into GitHub Actions so every commit grades diagnosis accuracy; added the adversarial Critic. *Result:* 85%+ accuracy vs a 45% naive baseline, with regression caught automatically in CI.

**(c) MCP tool-boundary design.** *Situation:* LLMs hallucinate columns and miscompute stats. *Task:* eliminate both structurally. *Action:* split capability across five MCP servers and enforced that semantic-mcp is the only SQL author and stats-mcp the only math path; the model composes metrics, never raw SQL. *Result:* 0 hallucinated columns/metrics and 100% governed SQL and every number is reproducible.

**(d) A hard SQL/data-quality problem.** *Situation:* channel attribution looked wrong. *Task:* correct it. *Action:* discovered the `traffic_source` struct is user first-touch, not session source; rebuilt `channel_group` on session-scoped `event_params` source/medium with first-touch fallback, and handled the `event_params` ARRAY-of-STRUCT unnesting and date-sharded scans within the byte budget. *Result:* correct GA4-style channel grouping and query cost held under the fixed BigQuery budget.

## 24.4 "Why you can trust this AI"

Four pillars: **grounding over generation** (LLM composes governed metrics, never authors SQL or stats); **verify-then-trust** (dry-run cost + schema validation + result reconciliation + adversarial Critic before any findings); **determinism where it matters** (all math is real code in stats-mcp); and **accountability** (every finding carries a significance test, a dollar revenue-at-risk, and a recommended action – no naked assertions).

## 24.5 Anticipated interviewer questions

1. *Why not just a SQL chatbot?* Chatbots are reactive and hallucinate; Helios is proactive, governed, and graded.
2. *How do you prevent hallucinated columns?* semantic-mcp is the only SQL path; the model references metric names, not schema.
3. *Mix-shift vs rate-change – explain.* The `w_i / r_i` decomposition; mix = composition change, rate = behavior change, plus interaction.
4. *How is accuracy measured?* Labeled benchmark in CI: 85%+ vs 45% baseline.
5. *What does the Critic actually check?* Mix-shift confound, sample size, seasonality, data quality.
6. *Why MCP and five servers?* Capability isolation = enforceable trust boundaries.
7. *Opus vs Sonnet split?* Opus orchestrates/critiques; Sonnet handles high-volume sub-tasks for cost.
8. *Biggest GA4 gotcha?* `traffic_source` is first-touch, not session-scoped; sessions key on `(user_pseudo_id, ga_session_id)`.
9. *How do you quantify revenue-at-risk?* Affected sessions x rate delta x revenue\_per\_session, attributed by segment.
10. *Why is LTV deferred?* The ~3-month window can't support lifetime curves; only proxy retention.
11. *Cost control?* dry\_run estimates bytes; runs are capped under a fixed byte budget.
12. *How do you avoid acting on noise?* significance\_test gating + Critic + suppression list in memory.
13. *Behavioral – disagreement with a stakeholder?* The decomposition gave an auditable answer that overrode the gut call on checkout.
14. *Behavioral – biggest mistake?* Initially trusted `traffic_source` as session source; the eval harness surfaced the attribution error.

## 24.6 Whiteboard talking points

Draw the seven-agent loop with the Critic as a gate; the five MCP servers as the trust boundary; the funnel `session_start → view_item → add_to_cart → begin_checkout → add_shipping_info → add_payment_info → purchase`; and the decomposition identity `deltaR = mix + rate + interaction`.

# 25. Resume Narrative

## 25.1 One-line headline

**Helios – Autonomous AI Growth Analyst (BigQuery + dbt + Claude Agent SDK + MCP):** a self-running diagnosis engine that distinguishes mix-shift from rate-change, quantifies revenue-at-risk in dollars, and prescribes a statistically-defensible experiment backlog from governed SQL.

## 25.2 Quantified achievement bullets

- Built an autonomous seven-agent diagnosis engine (Claude Agent SDK) that hit **85%+ root-cause accuracy** on a labeled benchmark vs a **45% naive baseline**, graded automatically in CI.
- Engineered a **mix-shift vs rate-change decomposition** (mix/rate/interaction) over GA4 funnel metrics, resolving Simpson's paradox and turning ambiguous conversion drops into segment-attributed root causes.
- Enforced **100% governed SQL with 0 hallucinated columns** by routing all SQL through a semantic layer and all statistics through deterministic stats tools over five MCP servers.
- Delivered **time-to-diagnosis under 5 minutes per run** under a fixed BigQuery byte budget via `dry_run` cost gating and date-shard pruning.

- Shipped findings where **100% carry a significance test, a dollar revenue-at-risk, and a recommended action**, vetted by an adversarial Critic agent before release.
- Modeled the GA4 export in **dbt** from `stg_ga4__events` to `fact_daily_funnel`, correctly handling the `traffic_source` first-touch gotcha and `event_params` ARRAY-of-STRUCT unnesting.

25.3 Tailored bullet variants

**Product / Growth Analyst:** "Diagnosed e-commerce funnel movement with a mix-shift-vs-rate-change framework, quantified revenue-at-risk in dollars, and produced a prioritized, power-analyzed experiment backlog and executive Decision Brief."

**Data / Analytics Engineer:** "Built the dbt analytics layer (staging -> intermediate -> facts/dims) on the GA4 BigQuery export with a governed semantic/metrics layer, reconciliation tests, and CI (dbt build + tests + eval harness) under a fixed byte budget."

**AI / ML Engineer:** "Architected a seven-agent plan-execute-critique system on the Claude Agent SDK with MCP tool boundaries (semantic-mcp as sole SQL author, stats-mcp as sole math path) and an adversarial Critic, achieving 85%+ benchmarked accuracy and 0 hallucinated metrics."

25.4 Skills / ATS keyword block

SQL BigQuery dbt GA4 / Google Analytics 4 data modeling semantic / metrics layer Model Context Protocol (MCP) multi-agent systems Claude Agent SDK LLM orchestration root-cause analysis (RCA) mix-shift decomposition A/B testing / experimentation power analysis statistical significance cohort / retention analysis RFM segmentation forecasting (Prophet, pmdarima) anomaly detection Python scipy / statsmodels / pandas GitHub Actions / CI revenue analytics conversion funnel

25.5 LinkedIn project blurb

"Helios is an autonomous AI Growth Analyst I built on BigQuery, dbt, and the Claude Agent SDK. It continuously diagnoses *why* an e-commerce funnel moves â€” separating mix-shift from rate-change, quantifying revenue-at-risk in dollars, and prescribing a statistically-defensible experiment backlog. The trust model is what makes it different: the LLM never writes raw SQL or computes statistics â€” it composes governed metrics and calls deterministic tools over five MCP servers, and an adversarial Critic refutes every finding before it ships. Benchmarked at 85%+ root-cause accuracy vs a 45% naive baseline."

25.6 Portfolio case-study structure

| Section               | What to show                                                               |
|-----------------------|----------------------------------------------------------------------------|
| Problem               | Funnel drops are ambiguous; BI dashboards don't say <i>why</i> .           |
| Anti-product          | Not a dashboard or SQL chatbot â€” autonomous, governed, graded.           |
| Architecture          | Seven-agent loop + five MCP servers diagram (trust boundary).              |
| Technical centerpiece | Mix/rate/interaction decomposition with worked Simpson's-paradox example.  |
| Trust & evals         | Grounding, Critic, and the labeled benchmark (85% vs 45%).                 |
| Sample Decision Brief | A real rendered brief with significance + dollar revenue-at-risk + action. |
| Results & roadmap     | Success metrics hit; the five-phase maturity ladder.                       |